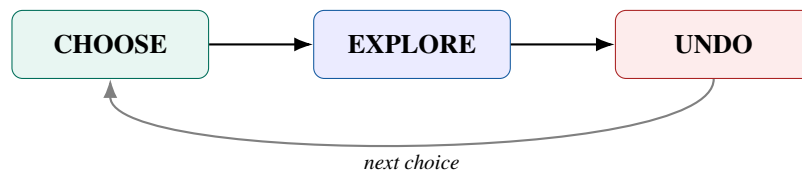


BACKTRACKING



From “I can’t see the recursion” to
*“I can define the state, derive the recursion,
prune the search space, and analyse it.”*

Python 3 • Java 17

Recursion Trees · Pruning · 12 LeetCode Patterns · Practice Set · Cheat Sheet

Study Edition

Contents

I	Foundations	1
1	Foundations of Backtracking	2
1.1	What is Backtracking?	2
1.2	Why Backtracking Exists	3
1.3	Backtracking vs Recursion	3
1.4	Backtracking vs DFS	4
1.5	Backtracking vs Brute Force	4
1.6	Backtracking vs Dynamic Programming	4
1.7	State-Space Search	5
1.8	Decision Trees	5
1.9	The Fundamental Cycle: CHOOSE → EXPLORE → UNDO	6
1.10	How to Identify a Backtracking Problem	6
1.11	The Anatomy of a Backtracking Solution	7
1.11.1	State	7
1.11.2	Base case	7
1.11.3	Choices	8
1.11.4	Constraints	8
1.11.5	Pruning	8
1.11.6	Undo operation	8
1.11.7	Result collection	8
1.12	Complexity Analysis: First Look	8
2	The Universal Backtracking Template	10
2.1	The template	10
2.2	Line-by-line	10
2.3	What changes, what stays, what is restored	11
2.4	How the call stack behaves	11
2.5	The generic recursion tree	12
2.6	Two dialects of the same template	13
II	The Core Patterns	14
3	Pattern 1 — Take / Don't Take	15
3.1	A. Intuition	15
3.2	B. Real-world analogy	15
3.3	C. State definition	15
3.4	D. Choices	15
3.5	E. Base case	15
3.6	F. Constraint	15
3.7	G. Pruning	15
3.8	H–J. Choose / Explore / Undo	16
3.9	The warm-up problem: all binary strings of length n	16
3.9.1	K. Recursion tree	16
3.9.2	L. Python implementation	17

3.9.3	M. Java implementation	17
3.9.4	N. Dry run ($n = 2$)	18
3.9.5	O. Complexity	18
3.9.6	P. Common mistakes	18
3.9.7	Q. Similar problem to try	19
3.9.8	R. LeetCode to solve independently	19
4	LeetCode 78 — Subsets	20
4.1	A. Deriving the solution without writing code	20
4.2	B. The decision tree for <code>nums = [1, 2, 3]</code>	20
4.3	C–G. The seven ingredients	21
4.4	L. Python solution	21
4.5	M. Java solution	22
4.6	The equivalent loop formulation	22
4.7	N. Dry run of the loop version, <code>nums = [1, 2]</code>	23
4.8	O. Complexity	23
4.9	P. Why the copies and the pop are mandatory	24
4.10	Common mistakes	24
4.11	Q. Try Yourself	25
4.12	R. LeetCode next	25
5	Pattern 2 — For Loop + Start Index	26
5.1	A. Why a loop instead of take/don't-take?	26
5.2	B. Real-world analogy	26
5.3	C. The ordering invariant	26
5.4	Subsets vs Combinations	27
5.5	A warm-up: choose 2 from <code>[a, b, c]</code>	27
5.6	LeetCode 77 — Combinations	27
5.6.1	C–G. Ingredients	27
5.6.2	K. Recursion tree for $n = 4, k = 2$	27
5.6.3	L. Python	28
5.6.4	M. Java	28
5.6.5	Why <code>backtrack(i + 1)</code> ?	29
5.6.6	N. Dry run, $n = 4, k = 2$	29
5.6.7	O. Complexity	29
5.6.8	P. Common mistakes	30
5.6.9	Q. Similar problems	30
5.6.10	R. Solve independently	30
6	Pattern 3 — Reusable Choices (i vs $i+1$)	31
6.1	The most consequential single character in backtracking	31
6.2	A minimal illustration	31
6.3	LeetCode 39 — Combination Sum	31
6.3.1	A. Intuition	32
6.3.2	C. State	32
6.3.3	D–G. Choices, base cases, pruning	32
6.3.4	K. Recursion tree, <code>candidates=[2, 3, 6, 7], target=7</code>	32
6.3.5	L. Python	33
6.3.6	M. Java	33
6.3.7	N. Dry run, <code>[2, 3, 6, 7], target 7</code>	34
6.3.8	O. Complexity	34
6.3.9	P. Common mistakes	34

6.3.10	Q. Similar problems	35
6.3.11	R. Solve independently	35
7	Pattern 4 — Permutations	36
7.1	A. Combination vs permutation: what actually changes	36
7.2	Why the start-index technique cannot work here	36
7.3	LeetCode 46 — Permutations	36
7.3.1	C–G. Ingredients	36
7.3.2	K. Recursion tree for <code>nums = [1, 2, 3]</code>	37
7.3.3	H–J. Choose / Explore / Undo, in detail	37
7.3.4	L. Python	37
7.3.5	M. Java	38
7.3.6	The swap variant (no extra memory)	38
7.3.7	N. Dry run, <code>nums=[1, 2]</code>	39
7.3.8	O. Complexity and the call stack	39
7.3.9	P. Common mistakes	39
7.3.10	Q. Similar problems	40
7.3.11	R. Solve independently	40
8	Pattern 5 — Handling Duplicates	41
8.1	A. The problem duplicates create	41
8.2	Why “dedupe with a set at the end” is inferior	41
8.3	The canonical-representative idea	41
8.3.1	Reading the condition piece by piece	42
8.4	LeetCode 90 — Subsets II	42
8.4.1	K. The tree, <code>nums = [1, 2, 2]</code> (sorted)	42
8.4.2	L. Python	43
8.4.3	M. Java	43
8.5	LeetCode 40 — Combination Sum II	43
8.5.1	L. Python	44
8.5.2	M. Java	44
8.5.3	N. Dry run, <code>[1, 1, 2]</code> , target 3	44
8.6	Permutations II (LeetCode 47): the same idea, different guard	45
8.7	P. Common mistakes recap	46
8.8	Q/R. Practice	46
III	Applied Backtracking	47
9	String Backtracking	48
9.1	A. Why strings deserve their own chapter	48
9.2	LeetCode 17 — Letter Combinations of a Phone Number	48
9.2.1	A–C. Intuition and state	48
9.2.2	D–G	48
9.2.3	K. Recursion tree for “23”	49
9.2.4	L. Python	49
9.2.5	M. Java	49
9.2.6	O. Complexity	50
9.2.7	P. Common mistakes	50
9.3	LeetCode 22 — Generate Parentheses	50
9.3.1	A. Intuition: constraints as the whole problem	50
9.3.2	C–G	50

9.3.3	K. Recursion tree for $n = 2$	51
9.3.4	L. Python	51
9.3.5	M. Java	51
9.3.6	N. Dry run, $n = 2$	52
9.3.7	O. Complexity	52
9.3.8	P. Common mistakes	53
9.3.9	Q/R. Practice	53
10	Partitioning Backtracking	54
10.1	A. Intuition: choosing where to cut	54
10.2	LeetCode 131 — Palindrome Partitioning	54
10.2.1	B. Real-world analogy	54
10.2.2	C–G. Ingredients	54
10.2.3	K. Recursion tree for "aab"	55
10.2.4	L. Python	55
10.2.5	M. Java	55
10.2.6	N. Dry run, "aab"	56
10.2.7	O. Complexity	56
10.3	Optimisation 1 — precomputed palindrome table	57
10.4	Optimisation 2 — when memoisation actually applies	57
10.5	P. Common mistakes	58
10.6	Q/R. Practice	58
11	Grid Backtracking	59
11.1	A. Intuition	59
11.2	LeetCode 79 — Word Search	59
11.2.1	B. Real-world analogy	59
11.2.2	C–G. Ingredients	59
11.2.3	Two ways to mark a cell	59
11.2.4	K. Visualising a search	60
11.2.5	L. Python	60
11.2.6	M. Java	61
11.2.7	N. Dry run, board $\begin{bmatrix} A & B \\ C & D \end{bmatrix}$, word "ABD"	62
11.2.8	O. Complexity	62
11.2.9	P. Common mistakes	62
11.2.10	Useful prunes	62
11.2.11	Q/R. Practice	63
12	Constraint Backtracking — N-Queens	64
12.1	A. Constraint satisfaction as a search pattern	64
12.2	LeetCode 51 — N-Queens	64
12.2.1	The first modelling decision (and it is the whole problem)	64
12.2.2	C–G. Ingredients	65
12.3	The diagonal formulas	65
12.3.1	A 4-queens solution board	65
12.3.2	K. The search tree for $n = 4$	66
12.3.3	L. Python	66
12.3.4	M. Java	66
12.3.5	The bitmask version (for the follow-up)	67
12.3.6	N. Dry run, $n = 4$, first branch	68
12.3.7	O. Complexity	68
12.3.8	P. Common mistakes	68

12.3.9 Q/R. Practice	69
13 Sudoku Solver	70
13.1 Why Sudoku is not “simply brute force”	70
13.2 LeetCode 37 — Sudoku Solver	70
13.2.1 The box index formula	70
13.2.2 C–G. Ingredients	71
13.2.3 L. Python	71
13.2.4 M. Java	72
13.2.5 N. Dry run (conceptual)	72
13.3 Optimisation techniques	73
13.3.1 1. Sets or boolean arrays instead of scanning	73
13.3.2 2. Bitmasks	73
13.3.3 3. MRV — Minimum Remaining Values	73
13.3.4 4. Constraint propagation	74
13.3.5 O. Complexity	74
13.3.6 P. Common mistakes	74
13.3.7 Q/R. Practice	74
IV Mastery	75
14 Pruning	76
14.1 What is pruning?	76
14.2 Why it matters: the arithmetic	76
14.3 Visualising it	76
14.4 Correctness vs optimisation vs pruning	76
14.5 The four kinds of prune	77
14.6 Worked examples	77
14.6.1 Combination Sum (LC 39)	77
14.6.2 Generate Parentheses (LC 22)	77
14.6.3 N-Queens (LC 51)	77
14.6.4 Sudoku (LC 37)	78
14.6.5 Word Search (LC 79)	78
14.7 How to invent a prune	78
14.8 Over-pruning and under-pruning	78
14.9 Pruning checklist	79
15 Backtracking + Memoisation	80
15.1 State-space tree vs state graph	80
15.2 When memoisation applies (and when it does not)	80
15.3 Example 1 — Subset sum: from backtracking to DP	81
15.3.1 The backtracking version	81
15.3.2 Spotting the repetition	81
15.3.3 The memoised version	81
15.4 Example 2 — Word Break (LC 139): the classic	82
15.5 Example 3 — Partition to K Equal Sum Subsets (LC 698)	83
15.6 Backtracking, memoisation and DP: one continuum	84
15.7 Common mistakes	84

16 Complexity Analysis	85
16.1 The master formula	85
16.2 The six quantities to identify	85
16.3 Worked analyses	85
16.3.1 Subsets (LC 78)	85
16.3.2 Combinations (LC 77)	86
16.3.3 Permutations (LC 46)	86
16.3.4 Combination Sum (LC 39)	86
16.3.5 Generate Parentheses (LC 22)	86
16.3.6 Palindrome Partitioning (LC 131)	86
16.3.7 Word Search (LC 79)	86
16.3.8 N-Queens (LC 51)	86
16.3.9 Sudoku (LC 37)	87
16.4 Summary table	87
16.5 Why exact complexity is often hard	87
16.6 Space complexity: three components	87
16.7 Complexity checklist	88
17 Pattern Cheat Sheet	89
17.1 The master table	89
17.2 Which template should I use?	90
17.3 The three recursion calls, side by side	90
17.4 Combination vs permutation, at a glance	90
17.5 Recognition signals	91
18 Python vs Java	92
18.1 The translation table	92
18.2 Side-by-side: the same solution twice	92
18.3 Java pitfalls	93
18.4 Python pitfalls	94
18.5 Performance notes	95
19 Why My Backtracking Solution Doesn't Work	96
19.1 Missing base case	96
19.2 Wrong base case	96
19.3 Forgetting the undo	97
19.4 Wrong start index	97
19.5 Using i when you need $i + 1$	97
19.6 Using $i + 1$ when you need i	97
19.7 Forgetting to reset <code>visited</code>	98
19.8 Not copying the path	98
19.9 Duplicate results	98
19.10 Over-pruning	98
19.11 Under-pruning	99
19.12 Mutating shared state incorrectly	99
19.13 Wrong recursion state	99
19.14 Incorrect loop bounds	100
19.15 Incorrect diagonal calculation	100
19.16 Infinite recursion	100
19.17 A five-minute debugging routine	100

20 How to Derive a Backtracking Solution	102
20.1 The twelve questions	102
20.2 Worked derivation: a problem you have not seen	102
20.3 The interview script	104
20.4 One-page interview checklist	105
21 Practice Set	106
21.1 Level 1 — Warm-ups (custom problems)	106
21.2 Level 2 — LeetCode core	106
21.3 Level 3 — Hard	108
21.4 Suggested progression	108
22 Diagrams and Animations	109
22.1 Sequential frames: animating the cycle on paper	109
22.2 The same idea as a growing tree	109
22.3 Beamer overlays — a compilable presentation	110
22.4 Notation used throughout this book	111
23 Final Summary	112
23.1 Backtracking in 10 rules	112
23.2 Template — Python	112
23.3 Template — Java	113
23.4 Pattern recognition table	113
23.5 i vs $i + 1$ — the one-liner	113
23.6 Combination vs permutation	113
23.7 Pruning checklist	114
23.8 Complexity checklist	114
23.9 Interview checklist	114
23.10 Complete problem progression	115
23.11 One-page cheat sheet	116
A Solutions	117
A.1 Level 1 — Warm-ups	117
A.2 Level 2 — LeetCode core	119
A.3 Level 3 — Hard	120
A.4 Answers to the “Try Yourself” prompts	120

Part I

Foundations

Chapter 1

Foundations of Backtracking

1.1 What is Backtracking?

Intuition

Imagine you are walking through a maze holding a piece of chalk. At every junction you pick a corridor, draw an arrow on the floor, and walk on. If you hit a dead end, you walk back to the last junction, *rub out your arrow*, and try the next corridor. That erasing step — restoring the world to exactly the way it was before you made the choice — is backtracking. Everything else is ordinary recursion.

Formal definition. Backtracking is a systematic method for exploring a *state-space tree* of partial candidate solutions. It builds a candidate incrementally, one *choice* at a time, and abandons (“backtracks” from) a partial candidate as soon as it determines that this candidate cannot possibly be extended to a valid complete solution.

Three properties define it:

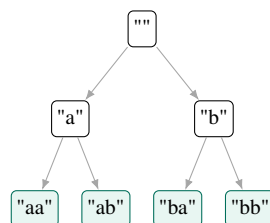
1. **Incremental construction.** A solution is built one decision at a time.
2. **Constraint checking (pruning).** A partial candidate that already violates a constraint is abandoned immediately, cutting off an entire subtree.
3. **State restoration (the undo).** After returning from a recursive branch, the shared state is restored so the next sibling branch starts from a clean slate.

Key Insight

Backtracking = **DFS over a state-space tree + constraint pruning + state restoration**. If you remove pruning you get exhaustive search. If you remove restoration you get a bug.

A tiny example

Generate all ways to place the letters a and b into 2 slots. At slot 0 you choose a or b; at slot 1 you again choose a or b. Four leaves: aa, ab, ba, bb. That is a state-space tree of depth 2 and branching factor 2.



Common Mistake

Beginners think “backtracking” means “the return statement”. It does not. The return happens automatically; the *undo* does not. If your language shares a mutable `path` list across the whole recursion, forgetting the undo corrupts every sibling branch.

1.2 Why Backtracking Exists

Some problems have no clever formula. If a problem asks you to *enumerate* all objects satisfying a property, or to *find one* object satisfying a hard combinatorial constraint, the honest answer is: you must search. Backtracking exists because:

- The answer set itself may be exponential (all 2^n subsets). No algorithm can be faster than the output size.
- Even when only *one* answer is needed (Sudoku, N-Queens), no polynomial algorithm is known — these are NP-complete style problems.
- Naive brute force generates *all* candidates then filters. Backtracking never generates a candidate whose prefix is already invalid, which in practice removes astronomically large parts of the space.

Intuition

Brute force for 8-Queens: try all $\binom{64}{8} \approx 4.4 \times 10^9$ placements. Backtracking with row-by-row placement and column/diagonal checks explores about 15,000 nodes. Same answer, six orders of magnitude cheaper. Pruning is not a micro-optimisation; it is the whole point.

1.3 Backtracking vs Recursion

Recursion is a *mechanism* (a function calling itself). Backtracking is a *strategy* that uses that mechanism.

	Plain recursion	Backtracking
Purpose	Compute a value	Enumerate / search candidates
Return value	Usually meaningful (<code>fib(n)</code>)	Often <code>void</code> ; answers pushed into a result list
State	Usually passed by value in parameters	Often a shared mutable structure
Undo step	Not needed	Essential
Shape	One recursive call per level is typical	A loop of many calls per level

```

1 def fact(n):
2     if n == 0:
3         return 1
4     return n * fact(n - 1)      # exactly one branch, value returned

```

Listing 1.1: Plain recursion: no choices, no undo.

```

1 def gen(path, n, out):
2     if len(path) == n:
3         out.append("".join(path))
4         return
5     for ch in "ab":
6         path.append(ch)      # CHOOSE
7         gen(path, n, out)    # EXPLORE
8         path.pop()          # UNDO

```

Listing 1.2: Backtracking: several branches, shared state, undo.

Remember This

Every backtracking function is recursive. Not every recursive function is backtracking. The tell-tale signs are: *a loop over choices* and *a mutation followed by its inverse*.

1.4 Backtracking vs DFS

Backtracking *is* DFS — but on a tree that does not exist in memory.

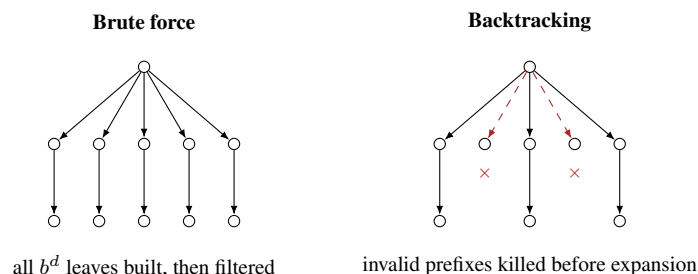
	Graph DFS	Backtracking
Tree/graph visited	Given explicitly (adjacency list) Global, never unmarked (we do not want to revisit a node ever)	Implicit; generated by the choice loop Often unmarked on the way out (we may revisit a cell on a <i>different</i> path)
Goal	Reach / count / traverse nodes	Build complete candidates along root-to-leaf paths
Memory	$O(V)$ for the graph	$O(\text{depth})$ for the call stack + path

Key Insight

The single sharpest distinction: in connectivity DFS, `visited[v]=True` is *permanent*. In backtracking (e.g. Word Search) it is *temporary* — it means “this cell is used by the path I am currently standing on”. So it must be undone when the path retreats.

1.5 Backtracking vs Brute Force

Brute force: *generate all, then test*. Backtracking: *test while generating*.



Note the crucial asymmetry: cutting a node at depth k removes b^{d-k} leaves. Pruning *early* (near the root) is worth exponentially more than pruning late.

Common Mistake

Writing brute force dressed as backtracking: building the full candidate, then checking validity only at the leaf. It is correct but wastes the entire benefit. Always ask: “can I detect failure at depth k instead of depth n ?”

1.6 Backtracking vs Dynamic Programming

Both explore a space of decisions. The difference is whether *different decision paths collapse into the same subproblem* — and whether you care about *all* solutions or an *optimal/counted* one.

	Backtracking	Dynamic Programming
Output	The objects themselves (all subsets, all permutations, a valid board)	An aggregate: min/max/count/boolean
Structure	State-space <i>tree</i> (paths are distinct)	State <i>graph</i> /DAG (paths merge)
Reuse	Rarely possible: the answer depends on the whole path	Essential: the answer depends only on the state
Complexity	Exponential, often $\Theta(\text{output size})$	Polynomial ($\#states \times \text{transition cost}$)

Key Insight

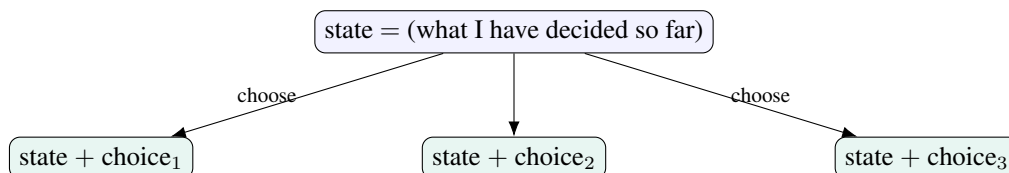
Memoisation is legal only if the value of a state does *not* depend on how you got there. “Can I make sum t using items from index i ?” depends only on (i, t) — so memoise. “List all subsets from index i that sum to t ” depends on the prefix you already picked, so the *answers* cannot be shared verbatim, only re-prefixed (Chapter 15).

1.7 State-Space Search

A backtracking algorithm is a walk over a rooted tree T :

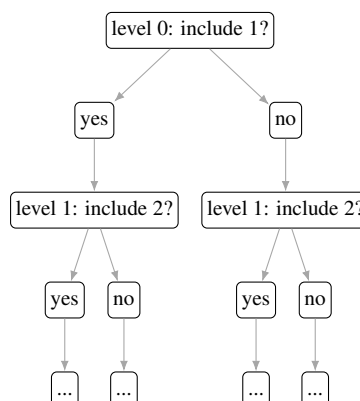
- **Nodes** = partial candidates (states). The root is the empty candidate.
- **Edges** = choices. A node at depth k has made k decisions.
- **Leaves** = complete candidates (or dead ends).
- **Branching factor** b = number of choices available at a node.
- **Depth** d = number of decisions in a complete candidate.

The size of the tree is at most $\frac{b^{d+1}-1}{b-1} = \Theta(b^d)$; hence the classic mental model $O(b^d)$.



1.8 Decision Trees

A *decision tree* is the state-space tree drawn with the *question* at each level made explicit. For subsets of $[1, 2, 3]$ the question at level i is: “do I include `nums[i]`?”

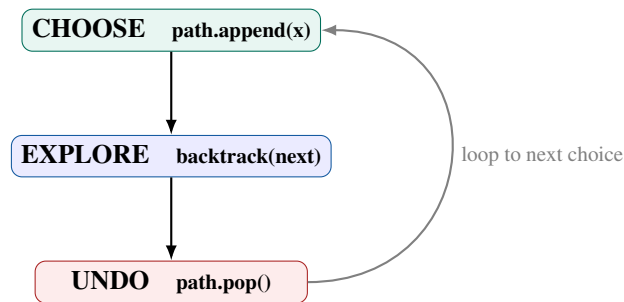


Drawing the decision tree *before* writing code is the single highest-yield habit in this entire book. If you can draw two levels correctly, the code writes itself.

Interview Tip

In an interview, say out loud: “Let me draw the decision tree for a size-3 input.” Then draw it on the whiteboard. Interviewers grade communication, and the tree also prevents you from writing the wrong recursion.

1.9 The Fundamental Cycle: CHOOSE → EXPLORE → UNDO



The three lines must be *adjacent* and *symmetric*. Whatever CHOOSE mutates, UNDO reverses — exactly, completely, and only that.

Remember This

Symmetry rule. For every mutation in CHOOSE there is exactly one inverse in UNDO. `append ↔ pop`, `visited[i]=True ↔ False`, `board[r][c]=v ↔ '.'`, `sb.append(ch) ↔ sb.deleteCharAt(len-1)`.

When you can skip the undo

If you pass a *new immutable object* down instead of mutating shared state, there is nothing to undo:

```

1 def gen(path, i, nums, out):
2     if i == len(nums):
3         out.append(path); return
4     gen(path + [nums[i]], i + 1, nums, out) # new list each call
5     gen(path, i + 1, nums, out)
  
```

Listing 1.3: No shared mutation, therefore no undo — but $O(n)$ copying per node.

This is legal and often cleaner, but costs an $O(n)$ copy at every node instead of only at the leaves. Interviewers expect you to know both and to justify your choice.

1.10 How to Identify a Backtracking Problem

Ask these questions. Two or more “yes” answers means backtracking.

1. Does the statement say “**all**”, “**every**”, “**list all**”, “**generate**”, “**count the number of ways**” (and n is small)?
2. Is the answer a **collection of sequences** (subsets, combinations, permutations, partitions, paths, boards)?
3. Is n suspiciously small ($n \leq 20$ for subsets, $n \leq 10$ for permutations, board $\leq 9 \times 9$)? Small constraints scream exponential search.

4. Can a candidate be built **one element at a time**, with validity checkable on the *prefix*?
5. Is there no obvious greedy or DP formulation because the output is the objects themselves?

Constraint you see	What it usually means
$n \leq 8$	permutations $O(n!)$ are fine
$n \leq 20$	subsets $O(2^n)$ are fine
$n \leq 40$	meet-in-the-middle $O(2^{n/2})$
$n \leq 10^5$	not backtracking — think greedy/DP/sorting

1.11 The Anatomy of a Backtracking Solution

Every solution in this book is assembled from exactly seven parts. Learn the parts, not the problems.

1. **State** — the variables that describe a partial candidate

2. **Base case** — when is the candidate complete (or dead)?

3. **Choices** — what can I append at this level?

4. **Constraints** — which choices are illegal right now?

5. **Pruning** — which legal choices are provably hopeless?

6. **Undo** — how do I restore the state exactly?

7. **Result collection** — where and how do I record a solution?

1.11.1 State

The state is *everything a recursive call needs to continue correctly*. Two flavours:

- **Explicit state** — passed as parameters: `index`, `remaining target`, `row`.
- **Implicit / shared state** — mutated in place: `path`, `visited[]`, `board`, `cols/diag sets`.

Shared state is what makes the undo necessary. A good heuristic: *scalars go in parameters, collections go in shared state*.

Common Mistake

Under-specified state. If your function needs to know “which elements are still available” but you only pass `path`, you will either recompute it every call ($O(n)$ waste) or get wrong answers. Write the state down before coding.

1.11.2 Base case

Two distinct kinds, and mixing them up is a classic bug:

- **Accepting base case:** the candidate is complete $\Rightarrow$ record it and return. (`if len(path)==k`, `if index==n`, `if target==0`.)

- **Rejecting base case:** the candidate is dead $\Rightarrow$ return without recording. (if `target < 0`.)

For subsets, *every* node is a solution, so the “base case” is only about stopping (`index == n`) while recording happens at every node — an important special case we dissect in Chapter 4.

1.11.3 Choices

The loop body. The choice set is one of:

- binary: take / don’t take (Chapter 3);
- positional: for `i` in `range(start, n)` (Chapter 5);
- full alphabet: for `ch` in `"0123456789"` or for `d` in `1..9` (Sudoku);
- directional: 4 neighbours in a grid (Chapter 11).

1.11.4 Constraints

A predicate that says “this choice is illegal *right now*”. Constraints are about **correctness**: without them you emit wrong answers.

1.11.5 Pruning

A predicate that says “this choice is legal but cannot lead to any solution”. Pruning is about **speed**: without it you still emit the right answers, just slowly. See the dedicated Chapter 14.

1.11.6 Undo operation

The exact inverse of CHOOSE, placed immediately after the recursive call.

1.11.7 Result collection

Append a *snapshot* (a copy) of the path — never the live object.

Common Mistake

`res.append(path)` in Python or `res.add(path)` in Java stores a *reference*. Since `path` keeps mutating, you end up with N pointers to one list, which at the end is empty. Always `list(path)` / `new ArrayList<>(path)`.

1.12 Complexity Analysis: First Look

The universal skeleton:

$$T(n) = \underbrace{(\text{number of nodes in the explored tree})}_{\leq O(b^d)} \times \underbrace{(\text{work done per node})}_{\text{loop, validity check, copying}}$$

Typical results, derived properly in Chapter 16:

Problem	Time	Space (excl. output)
Subsets	$O(n \cdot 2^n)$	$O(n)$
Combinations $\binom{n}{k}$	$O(k \binom{n}{k})$	$O(k)$
Permutations	$O(n \cdot n!)$	$O(n)$
Generate Parentheses	$O\left(\frac{4^n}{\sqrt{n}}\right)$	$O(n)$
N-Queens	$O(n!)$ upper bound	$O(n)$
Word Search	$O(m \cdot n \cdot 3^L)$	$O(L)$
Sudoku	$O(9^{\text{empty cells}})$ worst case	$O(1)$

Interview Tip

When asked for complexity, always separate “the number of nodes” from “the work per node”. Saying “ 2^n subsets, and each costs $O(n)$ to copy, so $O(n2^n)$ ” shows far more understanding than blurting “exponential”.

Chapter 2

The Universal Backtracking Template

2.1 The template

```
1 def backtrack(state):
2     if is_solution(state):           # 1. base case
3         save(copy_of(state))        # 2. record a SNAPSHOT
4         return                      # 3. stop descending
5
6     for choice in choices(state):    # 4. enumerate this level's options
7         if invalid(choice):         # 5. constraint / pruning
8             continue
9
10        make_choice(choice)          # 6. CHOOSE (mutate state)
11        backtrack(state)             # 7. EXPLORE (recurse deeper)
12        undo_choice(choice)          # 8. UNDO (restore state)
```

Listing 2.1: The universal template — Python

```
1 private void backtrack(State state) {
2     if (isSolution(state)) {        // 1. base case
3         save(new Snapshot(state));  // 2. record a COPY
4         return;                     // 3. stop descending
5     }
6
7     for (Choice choice : choices(state)) { // 4. options at this level
8         if (invalid(choice))         // 5. constraint / pruning
9             continue;
10
11        makeChoice(choice);          // 6. CHOOSE
12        backtrack(state);             // 7. EXPLORE
13        undoChoice(choice);          // 8. UNDO
14    }
15 }
```

Listing 2.2: The universal template — Java

2.2 Line-by-line

Line 1 — `if is_solution(state)`. This asks “is the partial candidate now complete?”. It is a question about the *state*, not about the loop. Depending on the problem it reads `index == n`, `len(path) == k`, `target == 0`, or `row == n`. It must be checked *before* the loop, because a complete candidate has nothing left to choose.

Line 2 — `save(copy_of(state))`. The snapshot. `path` is a single object that is being mutated as we walk the tree; if we store the object itself, all stored results alias the same list. In Python: `res.append(path[:])` or `list(path)`. In Java: `res.add(new ArrayList<>(path))`. For strings, Python’s `"".join(path)` and Java’s `sb.toString()` already produce new immutable objects, so they are safe.

Line 3 — `return`. Cuts the descent. Omitting it is not always fatal (the loop may be empty anyway) but it is sloppy and in problems like Combination Sum it causes infinite recursion.

Line 4 — the loop. This is the *branching factor*. The number of iterations here is b ; the depth of recursion is d ; the tree has $\Theta(b^d)$ nodes. Everything about performance is decided by how you write this loop and how aggressively line 5 kills iterations.

Line 5 — `if invalid(choice): continue`. Two different jobs live here:

- **Constraint** — the choice is illegal (queen attacks, cell already used, digit already in row). Skipping is required for correctness.
- **Pruning** — the choice is legal but hopeless (`candidate > target`). Skipping is required only for speed.

Sometimes `continue` can be upgraded to `break` (Chapter 6) when the candidate list is sorted, because if this one is hopeless, all later ones are too.

Line 6 — CHOOSE. Mutate the shared state to reflect the decision.

Line 7 — EXPLORE. Recurse. The parameters you pass here encode the *most important design decision in backtracking*: `i` (reuse allowed) versus `i+1` (reuse forbidden) versus `start` unchanged (permutations).

Line 8 — UNDO. Restore exactly what line 6 changed.

2.3 What changes, what stays, what is restored

Category	Examples	Lifetime
Changes per call (parameters)	<code>index, start, target, row, open, close</code>	Each stack frame has its own copy; automatically restored on return
Shared & mutated	<code>path, visited[], board, cols/diag sets, StringBuilder</code>	One object for the whole recursion; you must restore it
Constant (read-only)	<code>nums, n, k, grid, digitMap</code>	Never touched; safe to close over or pass once
Output	<code>res</code>	Grows monotonically; never undone

Key Insight

Parameters are undone *for free* by the call stack. Shared mutable state is not. That is the entire reason the UNDO line exists. If you make every piece of state a parameter (passing copies), you can drop the undo — at the cost of $O(n)$ copying per node.

2.4 How the call stack behaves

Consider generating binary strings of length 2. Time flows downward; indentation is stack depth.

```

bt(path="") depth 0
  choose '0' -> path="0"
    bt(path="0") depth 1
      choose '0' -> path="00"
        bt(path="00") -> SAVE "00", return
      undo -> path="0"
      choose '1' -> path="01"
        bt(path="01") -> SAVE "01", return
      undo -> path="0"
    return
  undo -> path=""
  choose '1' -> path="1"
    bt(path="1") ... SAVE "10", "11"
  undo -> path=""

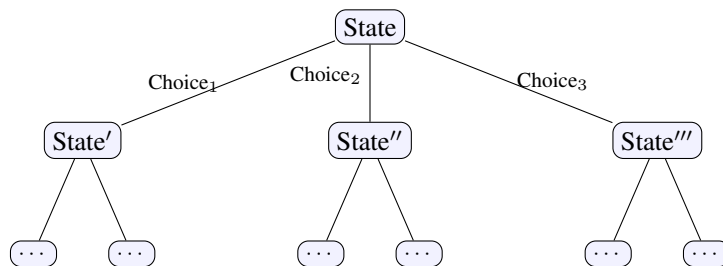
```

Observe the pattern: the path grows going down, shrinks going up, and at the moment the recursion returns to the root the path is *exactly as it started*. That invariant — *a backtracking call leaves the state as it found it* — is the property you should verify mentally for every function you write.

Remember This

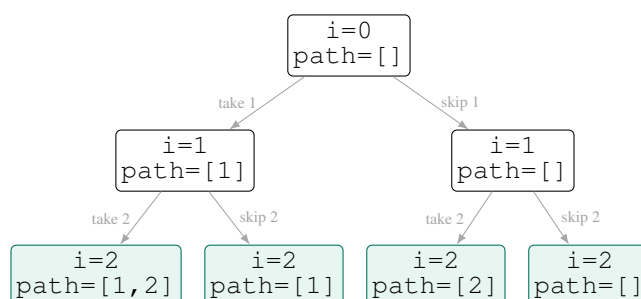
The invariant. `backtrack(...)` must return with `path`, `visited`, `board` in *exactly* the state they had on entry. If you can prove that for one call, induction gives it for all of them.

2.5 The generic recursion tree



A concrete annotated tree

Here is the same shape with real content: subsets of $[1, 2]$ using the take/don't-take formulation. Each node is annotated `idx | path`.



Green nodes are the base case $i == \text{len}(\text{nums})$; there are exactly $2^2 = 4$ of them, and they are the four subsets.

Interview Tip

Annotate every node of your whiteboard tree with the *parameters* of the call $(i, \text{target}, \text{path})$. Interviewers can then follow your reasoning, and you will catch off-by-one errors before you write a single line.

2.6 Two dialects of the same template

You will meet the template in two syntactic disguises. They are equivalent; choosing the right one is a matter of what the problem asks.

Dialect A — binary (take/don't take)

```
def bt(i, path):
    if i == n:
        res.append(path[:]); return
    path.append(nums[i])
    bt(i + 1, path)
    path.pop()
    bt(i + 1, path)
```

Tree is binary, depth exactly n , 2^n leaves.
Natural for: include/exclude, knapsack-shaped problems.

Dialect B — loop + start index

```
def bt(start, path):
    res.append(path[:])
    for i in range(start, n):
        path.append(nums[i])
        bt(i + 1, path)
        path.pop()
```

Tree is n -ary, depth $\leq n$, every *node* is an answer.
Natural for: combinations, “pick k ”, partitions, reuse control.

We spend Chapter 3 on Dialect A and Chapter 5 on Dialect B, then show they generate the same set for Subsets.

Common Mistake

Mixing the dialects: writing a `for` loop *and* an explicit “skip” branch. That double-counts and produces duplicate subsets. Pick one dialect per function.

Part II

The Core Patterns

Chapter 3

Pattern 1 — Take / Don't Take

3.1 A. Intuition

You are packing for a trip. On the bed lie n items. You walk down the row and, for each item independently, you answer one yes/no question: *does it go in the bag?* When you reach the end of the row, whatever is in the bag is one possible packing. Every distinct sequence of yes/no answers gives a distinct packing, and there are 2^n of them.

Intuition

The essential move is that you never look backwards and never look ahead. At index i you answer one question about `nums[i]` and then hand the rest of the row to a smaller copy of yourself. That is what makes the recursion binary and the depth exactly n .

3.2 B. Real-world analogy

A yes/no questionnaire with n questions. Each completed questionnaire is a leaf; the answer sheet is the path from the root.

3.3 C. State definition

$$\text{state} = (i, \text{path})$$

- i — the index of the element we are currently deciding about. It is *position in the input*, not size of the output.
- path — the decisions already committed (the partial answer).

3.4 D. Choices

Exactly two, always: *take* `nums[i]` or *skip* `nums[i]`. Branching factor $b = 2$.

3.5 E. Base case

$i == n$. We have run out of questions; the questionnaire is complete. Record it.

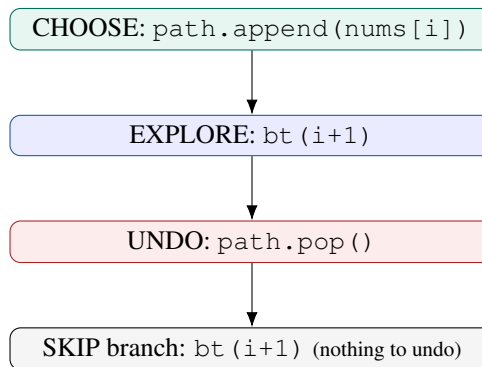
3.6 F. Constraint

For the pure enumeration problem, none. Both choices are always legal. Constraints appear as soon as we add a target sum or a size limit.

3.7 G. Pruning

None yet — and that is important. If you must output all 2^n answers, no pruning is possible, because the output itself is exponential. Pruning becomes possible only when a filter exists (“subsets summing to T ”).

3.8 H–J. Choose / Explore / Undo



Key Insight

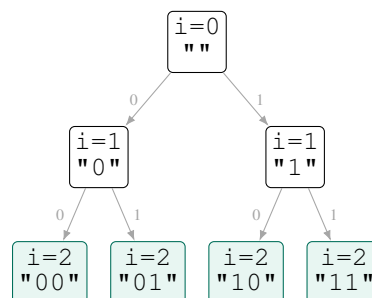
The “don’t take” branch requires **no undo** because it made **no mutation**. Undo is not a ritual you perform after every recursive call — it is the inverse of a mutation. No mutation, no inverse.

3.9 The warm-up problem: all binary strings of length n

Problem. Print all strings of length n over the alphabet $\{0, 1\}$. For $n = 2$: 00, 01, 10, 11.

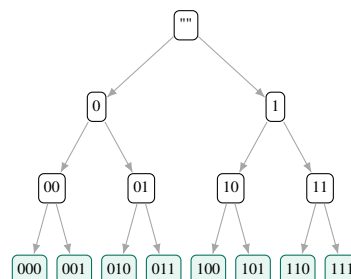
3.9.1 K. Recursion tree

$n = 2$.



$2^2 = 4$ leaves, $2^3 - 1 = 7$ nodes, depth 2.

$n = 3$.



$2^3 = 8$ leaves, 15 nodes. Notice that the tree *doubles* with each increment of n : this is the visual meaning of $O(2^n)$.

3.9.2 L. Python implementation

```

1  def binary_strings(n):
2      res = []
3      path = []
4
5      def backtrack(i):
6          if i == n:                # base case: n decisions made
7              res.append("".join(path)) # join creates a NEW string → safe
8              return
9          for ch in ("0", "1"):      # two choices
10             path.append(ch)         # CHOOSE
11             backtrack(i + 1)         # EXPLORE
12             path.pop()              # UNDO
13
14     backtrack(0)
15     return res

```

Listing 3.1: All binary strings of length n — Python

3.9.3 M. Java implementation

```

1  import java.util.*;
2
3  class Solution {
4      public List<String> binaryStrings(int n) {
5          List<String> res = new ArrayList<>();
6          backtrack(0, n, new StringBuilder(), res);
7          return res;
8      }
9
10     private void backtrack(int i, int n, StringBuilder sb, List<String> res) {
11         if (i == n) {                // base case
12             res.add(sb.toString());  // toString() copies → safe
13             return;
14         }
15         for (char ch : new char[]{'0', '1'}) {
16             sb.append(ch);            // CHOOSE
17             backtrack(i + 1, n, sb, res); // EXPLORE
18             sb.deleteCharAt(sb.length() - 1); // UNDO
19         }
20     }
21 }

```

Listing 3.2: All binary strings of length n — Java

Common Mistake

In Java, `res.add(sb)` instead of `res.add(sb.toString())` would store a reference to the *same* `StringBuilder`. Every entry would then print the same (final, empty) content. This is the same class of bug as forgetting `new ArrayList<>(path)`.

3.9.4 N. Dry run ($n = 2$)

Dry Run

Step	Call / action	path	res
1	bt (0)	[]	[]
2	choose '0'	[0]	[]
3	bt (1)	[0]	[]
4	choose '0'	[0, 0]	[]
5	bt (2) base, save	[0, 0]	["00"]
6	undo	[0]	["00"]
7	choose '1'	[0, 1]	["00"]
8	bt (2) base, save	[0, 1]	["00", "01"]
9	undo	[0]	
10	return to depth 0, undo	[]	
11	choose '1'	[1]	
12	symmetric subtree		["00", "01", "10", "11"]

Observe step 10: the path is empty again *before* the second top-level choice. That is the invariant working.

3.9.5 O. Complexity

- **Nodes:** $2^{n+1} - 1 = O(2^n)$.
- **Work per leaf:** $O(n)$ for `"".join / toString`.
- **Time:** $O(n \cdot 2^n)$ — optimal, since the output contains $n2^n$ characters.
- **Space:** $O(n)$ recursion depth + $O(n)$ path, excluding output.

Key Insight

Why depth is $O(n)$ but time is $O(2^n)$. Depth counts one root-to-leaf path (memory). Time counts *all* paths (work). Students who conflate the two conclude “the recursion is $O(n)$ ”, which is wrong by an exponential factor.

3.9.6 P. Common mistakes

Common Mistake

1. Using `i` as both index and length. Here they coincide because we always append. In Subsets they do *not*: `i` counts elements *considered*, `len(path)` counts elements *taken*.

Common Mistake

2. Forgetting return in the base case. The loop then runs at `i == n`, calls `bt(n+1)`, and `i == n` never matches again, giving `RecursionError / StackOverflowError`.

Common Mistake

3. Appending path instead of `"".join(path)`. Stores a live reference.

3.9.7 Q. Similar problem to try

Generate all strings of length n over the alphabet $\{a, b, c\}$ (branching factor 3, so 3^n leaves). Then generate all binary strings of length n with *no two consecutive 1s* — your first taste of a constraint (`if path and path[-1] == '1' and ch == '1': continue`) and therefore of pruning.

3.9.8 R. LeetCode to solve independently

LeetCode 78 — Subsets — exactly this pattern with a twist: instead of appending a fixed character, you append `nums[i]` on the “take” branch and *nothing* on the “skip” branch. That is the subject of the next chapter.

Interview Tip

If an interviewer asks a generation problem and you freeze, start by saying: “The simplest version of this is generating all binary strings of length n ; let me build up from there.” Anchoring on the warm-up almost always reveals the real structure.

Chapter 4

LeetCode 78 — Subsets

Problem. Given an integer array `nums` of *unique* elements, return all possible subsets (the power set). The solution set must not contain duplicate subsets; order does not matter.

4.1 A. Deriving the solution without writing code

Do not reach for code. Reach for a question. The only question that generates subsets is:

For every `nums[i]`: should I take it, or should I skip it?

Ask it once for each index, independently. A subset is a sequence of n yes/no answers. So:

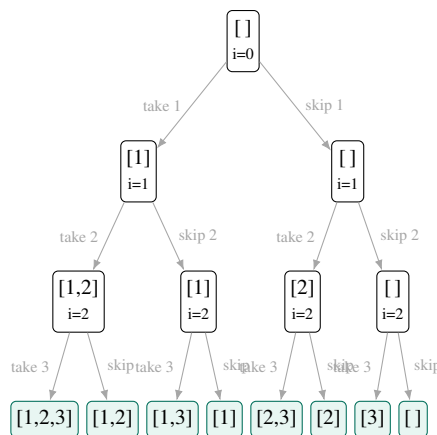
- There are 2^n answer-sequences, hence 2^n subsets. This matches the known size of a power set — a good sanity check that our model is right.
- The decision at index i does not depend on decisions at other indices, so the recursion is a clean binary tree of depth n .
- Because the elements are unique and we always move forward, no two leaves can produce the same subset. *Uniqueness is free* — we never need a `set`.

Key Insight

Subsets is Pattern 1 with a variable-length path. On “take” we append; on “skip” we append nothing. The path length therefore varies between 0 and n , while the index i always advances by exactly one.

4.2 B. The decision tree for `nums = [1, 2, 3]`

Each node shows `path`; the label on each edge is the decision about `nums[i]` where i is the depth of the parent.



The 8 green leaves, read left to right, are: `[1, 2, 3]`, `[1, 2]`, `[1, 3]`, `[1]`, `[2, 3]`, `[2]`, `[3]`, `[]`. Exactly 2^3 subsets, no duplicates.

Intuition

Read a root-to-leaf path as a bit string: *take* = 1, *skip* = 0. The leftmost leaf is 111 $\rightarrow$ [1, 2, 3]; the rightmost is 000 $\rightarrow$ []. Subsets and binary strings of length n are literally the same tree with different labels. This bijection is why the answer count is 2^n .

4.3 C–G. The seven ingredients

Ingredient	For Subsets
State	(i, path): index under consideration, elements taken so far
Base case	$i == \text{len}(\text{nums}) \Rightarrow$ record path and return
Choices	take <code>nums[i]</code> ; skip <code>nums[i]</code>
Constraints	none — both branches always legal
Pruning	none possible; the output itself is 2^n
Undo	<code>path.pop()</code> after the take branch only
Result	<code>res.append(path[:])</code> — a snapshot

4.4 L. Python solution

```

1  class Solution:
2      def subsets(self, nums):
3          res, path = [], []
4          n = len(nums)
5
6          def backtrack(i):
7              if i == n:                # all n decisions made
8                  res.append(path[:])  # SNAPSHOT (path[:] == list(path))
9                  return
10
11             # ---- choice 1: TAKE nums[i] ----
12             path.append(nums[i])      # CHOOSE
13             backtrack(i + 1)          # EXPLORE (i+1: never revisit index i)
14             path.pop()               # UNDO
15
16             # ---- choice 2: SKIP nums[i] ----
17             backtrack(i + 1)          # no mutation  $\rightarrow$  no undo needed
18
19         backtrack(0)
20         return res

```

Listing 4.1: LeetCode 78 — take/don't-take formulation

Line-by-line

- `res, path = [], []` — `res` accumulates snapshots; `path` is the single mutable buffer shared by the whole recursion.
- `if i == n` — the questionnaire is finished. Note we record *only at the leaves* in this formulation; every leaf is a distinct subset.
- `res.append(path[:])` — `path[:]` builds a new list with the same elements. Without the slice we would store 2^n references to one list that ends up empty.
- `path.append(nums[i])` then `backtrack(i+1)` then `path.pop()` — the **CHOOSE** $\rightarrow$ **EXPLORE** $\rightarrow$ **UNDO** cycle. `i+1` says: *index i is now settled forever*.

- the final `backtrack(i+1)` — the skip branch. Nothing was appended, so nothing is popped.

4.5 M. Java solution

```

1  import java.util.*;
2
3  class Solution {
4      public List<List<Integer>> subsets(int[] nums) {
5          List<List<Integer>> res = new ArrayList<>();
6          backtrack(0, nums, new ArrayList<>(), res);
7          return res;
8      }
9
10     private void backtrack(int i, int[] nums,
11                             List<Integer> path, List<List<Integer>> res) {
12         if (i == nums.length) {
13             res.add(new ArrayList<>(path)); // SNAPSHOT (deep-enough copy)
14             return;
15         }
16
17         path.add(nums[i]); // CHOOSE (take)
18         backtrack(i + 1, nums, path, res); // EXPLORE
19         path.remove(path.size() - 1); // UNDO → remove(int index)
20
21         backtrack(i + 1, nums, path, res); // SKIP
22     }
23 }

```

Listing 4.2: LeetCode 78 — Java

Common Mistake

`remove(Object)` vs `remove(int)`. For a `List<Integer>`, `path.remove(5)` removes the element *at index 5*, not the value 5. Always undo with `path.remove(path.size() - 1)`. If you ever truly need to remove by value, write `path.remove(Integer.valueOf(5))`.

4.6 The equivalent loop formulation

The same problem in Dialect B (Chapter 2), which we will need for Combinations. Here *every node* is a subset, not only the leaves.

```

1  class Solution:
2      def subsets(self, nums):
3          res, path = [], []
4
5          def backtrack(start):
6              res.append(path[:]) # EVERY node is a valid subset
7              for i in range(start, len(nums)):
8                  path.append(nums[i]) # CHOOSE
9                  backtrack(i + 1) # EXPLORE (forward only)
10                 path.pop() # UNDO
11
12         backtrack(0)
13         return res

```

Listing 4.3: LeetCode 78 — loop + start index

```

1  private void backtrack(int start, int[] nums,
2                          List<Integer> path, List<List<Integer>> res) {

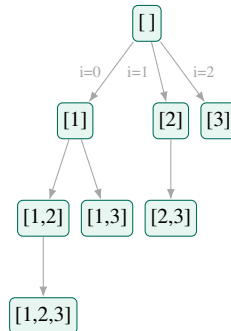
```

```

3     res.add(new ArrayList<>(path));           // every node is an answer
4     for (int i = start; i < nums.length; i++) {
5         path.add(nums[i]);                   // CHOOSE
6         backtrack(i + 1, nums, path, res);   // EXPLORE
7         path.remove(path.size() - 1);        // UNDO
8     }
9 }

```

Listing 4.4: LeetCode 78 — loop + start index, Java



Eight nodes, eight subsets — the same eight as before, produced in a different order and by a shallower, wider tree. Both are correct; the loop version has fewer nodes (8 vs 15) because it does not spend nodes on “skip” decisions.

Key Insight

The two trees are duals. Take/don’t-take has $2^{n+1} - 1$ nodes and records only at 2^n leaves. Loop+start has exactly 2^n nodes and records at *every* node. Same answers, same asymptotics, different shape. Being able to switch between them on demand is a strong signal in interviews.

4.7 N. Dry run of the loop version, `nums = [1, 2]`

Dry Run

#	Action	path	res
1	bt(0): record	[]	[[]]
2	i=0: append 1	[1]	
3	bt(1): record	[1]	[[], [1]]
4	i=1: append 2	[1, 2]	
5	bt(2): record; loop empty	[1, 2]	[[], [1], [1, 2]]
6	pop	[1]	
7	loop over, return; pop	[]	
8	i=1: append 2	[2]	
9	bt(2): record	[2]	[[], [1], [1, 2], [2]]
10	pop; done	[]	

Note step 7: the pop at depth 0 is what allows step 8 to start from [] instead of [1]. Delete that pop() and you get [1, 2] as the fourth answer — a silent wrong answer, not a crash.

4.8 O. Complexity

- Number of subsets: 2^n .

- **Copying cost:** each snapshot costs $O(\text{len}(\text{path})) = O(n)$ worst case. More precisely $\sum_{k=0}^n \binom{n}{k} k = n2^{n-1}$.
- **Time:** $\Theta(n \cdot 2^n)$. Optimal — the output has that many integers.
- **Space:** $O(n)$ auxiliary (path + call stack); $O(n2^n)$ for the output.

4.9 P. Why the copies and the pop are mandatory

Why `path[:]` in Python

```
res.append(path)    # stores a REFERENCE to the one shared list
```

Listing 4.5: Broken

At the end, `path` has been popped back to `[]`, and since all 2^n entries of `res` point at that same object, the result prints as `[[], [], [], [], [], [], [], []]`. The fix is any snapshot: `path[:]`, `list(path)`, or `copy.copy(path)`.

Why `new ArrayList<>(path)` in Java

Identical reasoning. `res.add(path)` adds the reference. `new ArrayList<>(path)` allocates a fresh backing array and copies the element references into it. Because `Integer` is immutable, a shallow copy is sufficient here; if the elements were mutable objects you would need a deep copy.

Why `pop()` / `remove()`

`path` is a single object walking the tree with us. After exploring the “take 1” subtree we are *back at the root* logically, but `path` still physically contains 1. Every subsequent subset would then wrongly include it. The `pop` re-synchronises the physical state with the logical position in the tree.

Without pop
after “take 1” subtree, `path=[1]`
⇒ “skip 1” subtree produces
`[1], [1, 2], [1, 3], [1, 2, 3]` again

With pop
after “take 1” subtree, `path=[]`
⇒ “skip 1” subtree produces
`[], [2], [3], [2, 3]`

4.10 Common mistakes

Common Mistake

Recording at every node and using take/don’t-take. That yields duplicates, because the “skip” chain re-records the same path repeatedly. Rule: record at leaves in Dialect A, record at every node in Dialect B — never both.

Common Mistake

Using `backtrack(start)` instead of `backtrack(i+1)` in the loop version. Then `nums[i]` can be picked again ⇒ infinite recursion / repeated elements. `i+1` is what encodes “each element is considered at most once”.

Common Mistake

Sorting or de-duplicating at the end with a `set`. Lists are unhashable in Python; you would need tuples, and the whole exercise is unnecessary because unique inputs plus forward-only movement already guarantee uniqueness. If you feel the need for a `set`, your recursion is wrong.

4.11 Q. Try Yourself

1. **Count subsets whose sum is even.** Same tree; instead of collecting, carry a running `s` and increment a counter at the leaf when `s % 2 == 0`. What is your state now? (Answer: `(i, s)` — and notice this one *is* memoisable, see Chapter 15.)
2. **Print subsets in increasing order of size.** Hint: run the loop version once per target size `k`, or sort at the end. Which is better and why?
3. **Subsets of a string's characters** — identical, but practise with `StringBuilder / "" .join`.

4.12 R. LeetCode next

LeetCode 77 — Combinations (next chapter) and then **LeetCode 90 — Subsets II** (Chapter 8).

Interview Tip

The one-line iterative solution — start with `res=[]` and for each `x` do `res += [s + [x] for s in res]` — is worth knowing as a follow-up, but lead with the backtracking version: the interviewer is testing recursion, not list comprehensions.

Chapter 5

Pattern 2 — For Loop + Start Index

5.1 A. Why a loop instead of take/don't-take?

Take/don't-take answers a question *per element*. But many problems are not naturally phrased that way. Consider: “choose 2 numbers out of 4”. The natural question is not “do I take 1?” but rather:

*Which element do I place **next**, given that I must pick from the ones I have not yet passed?*

That is a loop over candidates, not a binary switch. Formally, we replace the state (i, path) by $(\text{start}, \text{path})$ where *start* is a *lower bound on which indices are still allowed*, and the loop body enumerates all of them.

```
def backtrack(start, path):
    if <path is complete>:
        res.append(path[:]); return
    for i in range(start, n):          # every still-allowed candidate
        path.append(nums[i])          # CHOOSE
        backtrack(i + 1, path)        # EXPLORE: move past i
        path.pop()                   # UNDO
```

Listing 5.1: The Dialect B skeleton

5.2 B. Real-world analogy

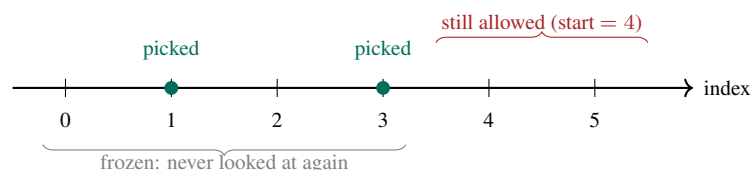
Picking a committee from a list of names printed in a fixed order. Once you have chosen Priya (row 3), you only look at rows 4 and below. You never go back up — if you did, you would produce the same committee in a different order and count it twice.

5.3 C. The ordering invariant

Key Insight

Invariant: the indices in `path` are strictly increasing. $i_1 < i_2 < \dots < i_k$. Each *set* of k indices has exactly *one* increasing arrangement, so each combination is generated exactly once. This single invariant is what removes duplicates — no `set`, no sorting, no post-filtering.

Why does `start` enforce it? Because a call `backtrack(start)` may only append an index $\geq \text{start}$, and when it appends index i it recurses with $i+1$. So every appended index is strictly greater than its predecessor. Induction does the rest.



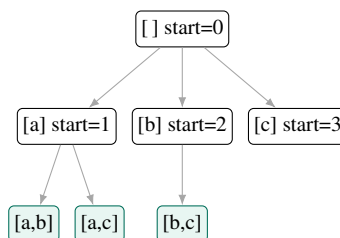
5.4 Subsets vs Combinations

	Subsets (78)	Combinations (77)
Answer size	any length $0..n$	exactly k
Where recorded	every node	only nodes with <code>len(path) == k</code>
Count	2^n	$\binom{n}{k}$
Recursion	<code>backtrack(i+1)</code>	<code>backtrack(i+1)</code> (identical!)
Difference	only the base case / recording rule	

Key Insight

Combinations is Subsets with a length filter. The tree is the same; we just stop recording at nodes of the wrong size (and, with pruning, stop *visiting* them too).

5.5 A warm-up: choose 2 from [a, b, c]



Green nodes are complete. Note the last branch `[c]`: it can never reach size 2, so it is a wasted subtree — our first candidate for pruning.

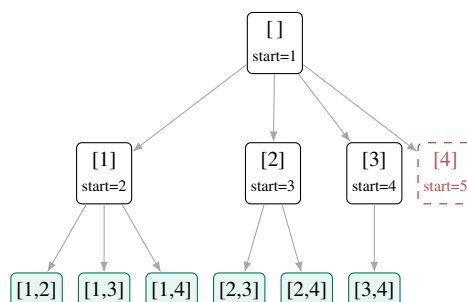
5.6 LeetCode 77 — Combinations

Problem. Given n and k , return all combinations of k numbers chosen from $[1, n]$.

5.6.1 C–G. Ingredients

State	<code>(start, path)</code> — smallest number still allowed, numbers chosen so far
Base case	<code>len(path) == k</code> $\Rightarrow$ record, return
Choices	every number i with <code>start <= i <= n</code>
Constraint	implicit: numbers must increase (encoded by <code>start</code>)
Pruning	stop the loop when not enough numbers remain (see below)
Undo	<code>path.pop()</code>

5.6.2 K. Recursion tree for $n = 4, k = 2$



$\binom{4}{2} = 6$ green leaves: $[1, 2], [1, 3], [1, 4], [2, 3], [2, 4], [3, 4]$. The red node $[4]$ is generated by the naive code and immediately dies — the pruning below removes it before it is created.

Intuition

Look at the widths: the first child has 3 grandchildren, the second 2, the third 1, the fourth 0. This staircase is exactly why the count is $\binom{4}{2} = 3 + 2 + 1$. Backtracking trees often *are* the combinatorial identity.

5.6.3 L. Python

```

1  class Solution:
2      def combine(self, n, k):
3          res, path = [], []
4
5          def backtrack(start):
6              if len(path) == k:           # base case: enough numbers
7                  res.append(path[:])
8                  return
9
10             need = k - len(path)        # how many more we must pick
11             # last valid starting number: n - need + 1
12             for i in range(start, n - need + 2):
13                 path.append(i)          # CHOOSE
14                 backtrack(i + 1)        # EXPLORE: strictly forward
15                 path.pop()              # UNDO
16
17         backtrack(1)
18         return res

```

Listing 5.2: LeetCode 77 — Combinations, with pruning

Deriving the pruning bound. Suppose we still need `need` numbers and we are considering starting at i . The numbers available from i upward are $n - i + 1$. We need $n - i + 1 \geq \text{need}$, i.e. $i \leq n - \text{need} + 1$. Python’s `range` is exclusive at the top, hence $n - \text{need} + 2$. Without this the loop runs to $n+1$ and creates doomed nodes like $[4]$ above.

5.6.4 M. Java

```

1  import java.util.*;
2
3  class Solution {
4      public List<List<Integer>> combine(int n, int k) {
5          List<List<Integer>> res = new ArrayList<>();
6          backtrack(1, n, k, new ArrayList<>(), res);
7          return res;
8      }
9
10     private void backtrack(int start, int n, int k,
11                             List<Integer> path, List<List<Integer>> res) {
12         if (path.size() == k) {
13             res.add(new ArrayList<>(path));
14             return;
15         }
16         int need = k - path.size();
17         for (int i = start; i ≤ n - need + 1; i++) {
18             path.add(i);                // CHOOSE
19             backtrack(i + 1, n, k, path, res); // EXPLORE
20             path.remove(path.size() - 1); // UNDO

```

```

21     }
22     }
23 }
```

Listing 5.3: LeetCode 77 — Combinations, Java

5.6.5 Why `backtrack(i + 1)`?

Remember This

We pass `i + 1`, not `start + 1` and not `i`, because:

- `i`: would allow re-picking the *same* number $\Rightarrow [1, 1]$, and in an unbounded problem, infinite recursion.
- `start + 1`: ignores which element we actually took. If the loop is at `i=3` but `start` was 1, we would re-offer 2 and 3, producing $[3, 2]$ — a permutation, not a combination, and a duplicate of $[2, 3]$.
- `i + 1`: “I just consumed index *i*; the next pick must come strictly after it.” This is the ordering invariant in code.

5.6.6 N. Dry run, $n = 4, k = 2$

Dry Run

#	Call	path	note
1	<code>bt(1)</code>	<code>[]</code>	<code>need=2</code> , loop <code>i=1..3</code>
2	<code>i=1, push</code>	<code>[1]</code>	
3	<code>bt(2)</code>	<code>[1]</code>	<code>need=1</code> , loop <code>i=2..4</code>
4	<code>i=2, push</code>	<code>[1, 2]</code>	
5	<code>bt(3)</code>	<code>[1, 2]</code>	<code>size = k</code> $\Rightarrow$ record
6	<code>pop</code>	<code>[1]</code>	
7	<code>i=3, push, record, pop</code>	<code>[1]</code>	<code>[1, 3]</code> recorded
8	<code>i=4, push, record, pop</code>	<code>[1]</code>	<code>[1, 4]</code> recorded
9	<code>return, pop</code>	<code>[]</code>	
10	<code>i=2, push</code>	<code>[2]</code>	yields <code>[2, 3]</code> , <code>[2, 4]</code>
11	<code>i=3, push</code>	<code>[3]</code>	yields <code>[3, 4]</code>
12	loop ends at <code>i = 3</code>	<code>[]</code>	<code>i = 4</code> pruned

5.6.7 O. Complexity

- Leaves: $\binom{n}{k}$; each costs $O(k)$ to copy.
- Internal nodes: $O\left(\binom{n}{k}\right)$ as well with pruning (each internal node has at least one descendant leaf), so **time** $= O\left(k \binom{n}{k}\right)$.
- Without the pruning bound the tree also contains dead branches; the bound is then loosely $O\left(k \cdot \binom{n}{k} \cdot (n - k + 1)\right)$ — still exponential-ish but measurably worse.
- **Space**: $O(k)$.

5.6.8 P. Common mistakes

Common Mistake

Starting the loop at 0 or at 1 inconsistently. LeetCode 77 uses numbers $1..n$, so `backtrack(1)` and `i <= n`. Array problems use indices $0..n - 1$, so `backtrack(0)` and `i < n`. Mixing the conventions gives off-by-one errors that compile fine.

Common Mistake

Checking `if len(path) > k:` return instead of `== k`. Harmless but useless: the path can never exceed k if you return at k .

Common Mistake

Over-pruning. Writing `n - k + 1` instead of `n - need + 1` uses the *original* k rather than the *remaining* count and drops valid combinations. Always recompute `need = k - len(path)`.

5.6.9 Q. Similar problems

“All increasing subsequences of length k ”, “all ways to choose k pizza toppings”, and **LeetCode 216 — Combination Sum III** (choose k distinct digits $1..9$ summing to n) — which is Combinations plus a sum constraint, and an excellent bridge to the next chapter.

5.6.10 R. Solve independently

LeetCode 77 — Combinations without looking, then **LeetCode 216 — Combination Sum III**.

Chapter 6

Pattern 3 — Reusable Choices (i vs $i+1$)

6.1 The most consequential single character in backtracking

Every loop-based backtracking function ends with a recursive call. What you pass as the new `start` decides the entire shape of the answer set:

`backtrack(i)`
element i may be chosen **again**
⇒ multisets / unbounded repetition

`backtrack(i + 1)`
element i is **consumed**
⇒ each element used at most once

Remember This

i = “stay on this element, I may want it again.”

$i+1$ = “move past this element, I am done with it forever.”

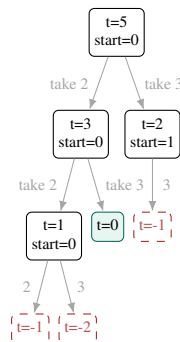
`start` (unchanged) = “I am not restricting anything” — used for permutations, where a `visited` array does the restricting instead.

6.2 A minimal illustration

Let `nums = [2, 3]` and ask for all ways to reach a sum of exactly 5.

With `backtrack(i+1)` the only sequences are `[2, 3]` — each number used once.

With `backtrack(i)` we also get, for target 4, `[2, 2]`; for target 6, `[2, 2, 2]` and `[3, 3]`. Repetition unlocked.



Green = solution `[2, 3]`. Red = pruned because the remaining target went negative.

6.3 LeetCode 39 — Combination Sum

Problem. Given an array of *distinct* positive integers `candidates` and a target, return all *unique* combinations that sum to target. **Each number may be used an unlimited number of times.** Two combinations are different if the multiset of chosen numbers differs.

6.3.1 A. Intuition

Think of coins of distinct denominations and a bill you must pay exactly. You may reuse a denomination as often as you like, but paying $2 + 3$ and $3 + 2$ is the same payment. To avoid counting it twice we impose the same ordering invariant as in Chapter 5 — non-decreasing indices — except now “non-decreasing” rather than “strictly increasing”, because repeats are allowed.

Key Insight

Uniqueness of *multisets* $\Leftrightarrow$ indices are **non-decreasing** $\Leftrightarrow$ recurse with i (not start , not $i+1$).

6.3.2 C. State

$\text{state} = (\text{start}, \text{remaining target } t, \text{path})$

Carrying the *remaining* target rather than the running sum is a small but real simplification: the base case becomes $t == 0$ and the prune becomes $t < 0$, both independent of the original target.

6.3.3 D–G. Choices, base cases, pruning

Choices	<code>candidates[i]</code> for i from <code>start</code> to <code>n-1</code>
Accept	$t == 0 \Rightarrow$ record path, return
Reject	$t < 0 \Rightarrow$ return (dead end)
Prune	after sorting: if <code>candidates[i] > t</code> then break
Undo	<code>path.pop()</code>

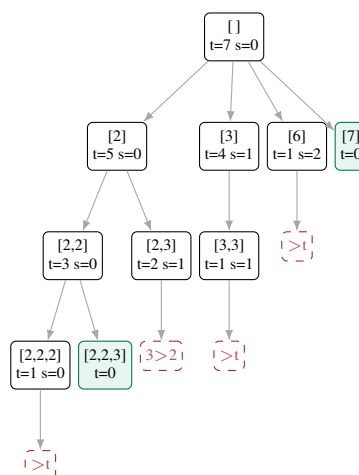
Key Insight

Why $t < 0$ may terminate a branch. All candidates are strictly positive, so the remaining target is non-increasing along any path. Once it is negative it can never return to 0. This monotonicity is what licenses the prune — with negative numbers allowed, the same prune would be *incorrect*.

Key Insight

Why **break can replace **continue** after sorting.** If `candidates` is sorted ascending and `candidates[i] > t`, then every later candidate is also $> t$. There is nothing left to try at this level, so we abandon the whole loop, not just this iteration. `continue` would still be correct, merely slower.

6.3.4 K. Recursion tree, `candidates=[2, 3, 6, 7]`, `target=7`



Solutions: $[2, 2, 3]$ and $[7]$. Every red node is killed by $\text{candidates}[i] > t$. Note that $[3]$ does *not* branch back to 2: the non-decreasing invariant forbids it, which is precisely how $[2, 3]$ and $[3, 2]$ are prevented from both appearing.

6.3.5 L. Python

```

1  class Solution:
2      def combinationSum(self, candidates, target):
3          candidates.sort()          # enables the break-prune
4          res, path = [], []
5
6          def backtrack(start, t):
7              if t == 0:              # ACCEPT
8                  res.append(path[:])
9                  return
10             for i in range(start, len(candidates)):
11                 c = candidates[i]
12                 if c > t:             # sorted => all later ones are too big
13                     break           # PRUNE the rest of this level
14                 path.append(c)       # CHOOSE
15                 backtrack(i, t - c)  # EXPLORE: 'i', not 'i+1' => reuse allowed
16                 path.pop()          # UNDO
17
18         backtrack(0, target)
19         return res

```

Listing 6.1: LeetCode 39 — Combination Sum

6.3.6 M. Java

```

1  import java.util.*;
2
3  class Solution {
4      public List<List<Integer>> combinationSum(int[] candidates, int target) {
5          Arrays.sort(candidates);          // enables break-prune
6          List<List<Integer>> res = new ArrayList<>();
7          backtrack(0, target, candidates, new ArrayList<>(), res);
8          return res;
9      }
10
11     private void backtrack(int start, int t, int[] cand,
12                             List<Integer> path, List<List<Integer>> res) {
13         if (t == 0) {
14             res.add(new ArrayList<>(path));
15             return;
16         }
17         for (int i = start; i < cand.length; i++) {
18             if (cand[i] > t) break;          // PRUNE whole level
19             path.add(cand[i]);              // CHOOSE
20             backtrack(i, t - cand[i], cand, path, res); // reuse: i
21             path.remove(path.size() - 1);    // UNDO
22         }
23     }
24 }

```

Listing 6.2: LeetCode 39 — Combination Sum, Java

6.3.7 N. Dry run, [2, 3, 6, 7], target 7

Dry Run

#	call	path	t	event
1	bt(0, 7)	[]	7	try 2
2	bt(0, 5)	[2]	5	try 2
3	bt(0, 3)	[2, 2]	3	try 2
4	bt(0, 1)	[2, 2, 2]	1	$2 > 1 \Rightarrow$ break
5	undo	[2, 2]	3	try 3
6	bt(1, 0)	[2, 2, 3]	0	record
7	undo, undo	[2]	5	try 3
8	bt(1, 2)	[2, 3]	2	$3 > 2$ break
9	undo, undo	[]	7	try 3
10	bt(1, 4)	[3]	4	try 3
11	bt(1, 1)	[3, 3]	1	break
12	...	[]	7	try 6 $\rightarrow$ t=1 break ; try 7 $\rightarrow$ t=0 record

Note step 8: after taking 3 we recursed with `start=1`, so 2 is no longer offered — that is why `[3, 2]` never appears.

6.3.8 O. Complexity

Let T be the target and m the smallest candidate. The deepest path has $d = \lfloor T/m \rfloor$ levels, and each level branches at most n ways, so a loose bound is $O(n^{T/m})$; copying costs $O(T/m)$ per solution. A commonly quoted form is

$$O(n^{T/m+1}) \text{ time, } O(T/m) \text{ space (excluding output).}$$

The true count is the number of restricted multisets summing to T — much smaller in practice thanks to the two prunes. This is a good example of a problem where the exact complexity is genuinely hard to state; see Chapter 16.

6.3.9 P. Common mistakes

Common Mistake

Passing $i + 1$. You then solve a different problem (each candidate used at most once — that is LeetCode 40, and even there it needs duplicate handling). Symptom: missing answers like `[2, 2, 3]`.

Common Mistake

Passing `start` instead of i . Now every level may pick *any* candidate, so `[2, 3]` and `[3, 2]` both appear — duplicates. Symptom: too many answers, and permutations of the same multiset.

Common Mistake

Forgetting that the prune requires positivity. If candidates could be 0, the recursion `backtrack(i, t-0)` never decreases $t \Rightarrow$ infinite recursion. If they could be negative, $t < 0$ would no longer be terminal. Always state the positivity assumption out loud.

Common Mistake

Using `break` without sorting. `break` assumes the remaining candidates are all at least as large. On an unsorted array it silently drops valid answers. Either sort, or use `continue`.

6.3.10 Q. Similar problems

LeetCode 40 — Combination Sum II (each element once, duplicates in input — Chapter 8); **LeetCode 216 — Combination Sum III**; **LeetCode 377 — Combination Sum IV** (where order *does* matter, so it becomes a counting DP, not a backtracking enumeration — a great contrast to discuss in an interview).

6.3.11 R. Solve independently

LeetCode 39 — Combination Sum, then try modifying your own code by changing i to $i+1$ and predicting the output before running it.

Interview Tip

When you write the recursive call, say the reason aloud: “I pass i because after choosing `candidates[i]` I am still allowed to choose it again, but I must never go back to a smaller index or I would generate the same multiset in a different order.” That one sentence covers reuse *and* deduplication.

Chapter 7

Pattern 4 — Permutations

7.1 A. Combination vs permutation: what actually changes

	Combination	Permutation
Is <code>[1, 2]</code> the same as <code>[2, 1]</code> ?	Yes — one answer	No — two answers
Count from n items, all used	1	$n!$
Enforcement device	start index (ordering invariant)	<code>visited[]</code>
Loop range	<code>range(start, n)</code>	<code>range(0, n)</code>

Intuition

A combination is a *bag*; a permutation is a *queue*. When order matters, you must be able to reach backwards — position 3 of the output may hold `nums[0]`. But the moment you allow the loop to reach backwards, “have I already used this element?” stops being answerable from the index alone. Something must remember it. That something is `visited`.

7.2 Why the start-index technique cannot work here

The whole point of `start` was to *forbid* going back, which is exactly what kills duplicate orderings. Permutations *want* those orderings. So:

- If we keep `start`, we can only ever produce the increasing arrangement — i.e. combinations. We lose $n! - 1$ of the $n!$ answers.
- If we drop `start` and loop `0..n-1` with no memory, we produce *sequences with repetition*: `[1, 1]`, `[1, 1, 1]`, n^n of them instead of $n!$.

Neither is right, so a new state component is forced upon us.

Key Insight

`visited[i]` means: “*nums[i]* is already sitting somewhere in the path I am currently building.” It is a property of the current root-to-node path, not of the whole search. Therefore it must be cleared on the way back up — unlike graph-DFS `visited`, which is permanent.

7.3 LeetCode 46 — Permutations

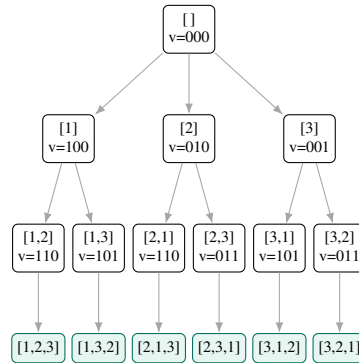
Problem. Given an array `nums` of distinct integers, return all possible permutations.

7.3.1 C–G. Ingredients

State	<code>(path, visited)</code> — note: no index . The “level” is <code>len(path)</code> .
Base case	<code>len(path) == len(nums) ⇒ record, return</code>
Choices	every index <code>i</code> in <code>0..n-1</code>
Constraint	<code>visited[i]</code> must be false
Pruning	none in the vanilla problem (all $n!$ leaves are answers)
Undo	<code>path.pop()</code> and <code>visited[i] = False</code> — two mutations, two inverses

Common Mistake

Undoing only one of the two mutations. If you `pop()` but forget `visited[i]=False`, the search silently loses branches and returns far fewer than $n!$ permutations. If you clear `visited` but forget `pop()`, the path grows forever. Write the two undo lines together, immediately after the recursive call.

7.3.2 K. Recursion tree for `nums = [1, 2, 3]`

The annotation `v=110` is the `visited` bitmask. Observe the shrinking branching factor: 3 at the root, 2 at depth 1, 1 at depth 2. That is $3 \times 2 \times 1 = 3! = 6$ leaves — the factorial appears directly in the tree's shape.

Intuition

Compare with the subsets tree, where the branching factor stayed at 2 the whole way down (2^n). Here it decays because each level consumes one element. *The shape of the tree is the formula.*

7.3.3 H–J. Choose / Explore / Undo, in detail

CHOOSE `visited[i]=True; path.append(nums[i])`

EXPLORE `backtrack()` (no index argument!)

UNDO `path.pop(); visited[i]=False`

Undo in the mirror order of choose (pop first, then unmark) is good hygiene, though for independent mutations the order does not matter.

7.3.4 L. Python

```

1 class Solution:
2     def permute(self, nums):
3         n = len(nums)
4         res, path = [], []
5         visited = [False] * n
6
7         def backtrack():
8             if len(path) == n:           # base case: path is full
9                 res.append(path[:])
10                return
11            for i in range(n):           # EVERY index, not range(start, n)
12                if visited[i]:           # constraint: already in this path
13                    continue

```

```

14         visited[i] = True           # CHOOSE (mutation 1)
15         path.append(nums[i])        # CHOOSE (mutation 2)
16         backtrack()                # EXPLORE
17         path.pop()                  # UNDO (inverse of 2)
18         visited[i] = False          # UNDO (inverse of 1)
19
20     backtrack()
21     return res

```

Listing 7.1: LeetCode 46 — Permutations, visited array

7.3.5 M. Java

```

1  import java.util.*;
2
3  class Solution {
4      public List<List<Integer>> permute(int[] nums) {
5          List<List<Integer>> res = new ArrayList<>();
6          boolean[] visited = new boolean[nums.length];
7          backtrack(nums, visited, new ArrayList<>(), res);
8          return res;
9      }
10
11     private void backtrack(int[] nums, boolean[] visited,
12                             List<Integer> path, List<List<Integer>> res) {
13         if (path.size() == nums.length) {
14             res.add(new ArrayList<>(path));
15             return;
16         }
17         for (int i = 0; i < nums.length; i++) {
18             if (visited[i]) continue;           // constraint
19             visited[i] = true;                   // CHOOSE
20             path.add(nums[i]);
21             backtrack(nums, visited, path, res); // EXPLORE
22             path.remove(path.size() - 1);        // UNDO
23             visited[i] = false;                   // UNDO
24         }
25     }
26 }

```

Listing 7.2: LeetCode 46 — Permutations, Java

7.3.6 The swap variant (no extra memory)

An elegant alternative fixes the prefix by swapping elements into place:

```

1  def permute(nums):
2      res = []
3      def backtrack(k):                # positions 0..k-1 are fixed
4          if k == len(nums):
5              res.append(nums[:]); return
6          for i in range(k, len(nums)):
7              nums[k], nums[i] = nums[i], nums[k] # CHOOSE
8              backtrack(k + 1)                  # EXPLORE
9              nums[k], nums[i] = nums[i], nums[k] # UNDO (swap back)
10     backtrack(0)
11     return res

```

Listing 7.3: Permutations by swapping

Key Insight

Here *the input array itself is the state*. The swap is the choose; swapping back is the undo. It saves the `visited` array and the separate `path`, but the output order differs and it is harder to extend to duplicate handling. Know both; lead with `visited` because it generalises.

7.3.7 N. Dry run, `nums=[1, 2]`**Dry Run**

#	action	path	visited	res
1	<code>bt()</code>	<code>[]</code>	<code>[F, F]</code>	<code>[]</code>
2	<code>i=0: mark, push</code>	<code>[1]</code>	<code>[T, F]</code>	
3	<code>bt(): i=0 skipped (visited)</code>			
4	<code>i=1: mark, push</code>	<code>[1, 2]</code>	<code>[T, T]</code>	
5	<code>bt(): full ⇒ record</code>			<code>[[1, 2]]</code>
6	<code>undo: pop, unmark</code>	<code>[1]</code>	<code>[T, F]</code>	
7	<code>return; undo: pop, unmark</code>	<code>[]</code>	<code>[F, F]</code>	
8	<code>i=1: mark, push</code>	<code>[2]</code>	<code>[F, T]</code>	
9	<code>i=0: mark, push, record</code>	<code>[2, 1]</code>	<code>[T, T]</code>	<code>[[1, 2], [2, 1]]</code>
10	<code>unwind to [], [F, F]</code>	<code>[]</code>	<code>[F, F]</code>	

Line 7 is the crucial one. Had we forgotten `visited[0]=False`, step 9 would find index 0 still marked and `[2]` would never grow — the algorithm would return only one permutation.

7.3.8 O. Complexity and the call stack

- **Leaves:** $n!$.
- **Internal nodes:** $\sum_{k=0}^n \frac{n!}{(n-k)!} \leq e \cdot n!$, so $\Theta(n!)$.
- **Work per node:** the loop is $O(n)$ (we scan all n indices even though only $n - k$ are usable); each leaf copies $O(n)$.
- **Time:** $O(n \cdot n!)$.
- **Space:** call stack depth exactly $n + 1$ frames; `path` $O(n)$; `visited` $O(n)$. Total $O(n)$ auxiliary.

Call stack picture. At any instant the stack holds one frame per element placed so far. Frame k is “deciding what goes in position k ”, and its local variable `i` remembers where its loop had got to. When frame $k + 1$ returns, frame k resumes its loop at the next `i`. The `path` is, quite literally, the concatenation of the choices currently live on the stack.

7.3.9 P. Common mistakes**Common Mistake**

Using `if nums[i] in path`: `continue` instead of `visited`. It works for distinct values but costs $O(n)$ per check (raising the total to $O(n^2 \cdot n!)$) and breaks immediately if the array contains duplicate *values* — because then two distinct positions look identical.

Common Mistake

Making `visited` a local variable inside the loop (Java: `boolean[] visited = new boolean[n];` inside `backtrack`). Each call then gets a fresh array and nothing is remembered. Symptom: n^n outputs with repeats.

Common Mistake

Keeping a `start` parameter out of habit. `for i in range(start, n)` turns permutations back into combinations. There is no `start` in a permutation.

Common Mistake

Declaring `res` or `path` as a `static` field in Java. Two test cases in the same JVM then share state and the second one is polluted. Pass them as parameters or make them instance fields reset per call.

7.3.10 Q. Similar problems

LeetCode 47 — Permutations II (with duplicates — Chapter 8), **LeetCode 31 — Next Permutation** (not backtracking, but the same combinatorial object), **LeetCode 784 — Letter Case Permutation** (a hybrid: binary choice per character).

7.3.11 R. Solve independently

LeetCode 46 — Permutations. Then implement the swap variant and verify both produce the same $n!$ results (as sets).

Interview Tip

Interviewers love the follow-up “what if the array has duplicates?”. Have the answer ready: sort, then add the same-level skip `if i > 0 and nums[i] == nums[i-1] and not visited[i-1]: continue`. We derive exactly why in the next chapter.

Chapter 8

Pattern 5 — Handling Duplicates

8.1 A. The problem duplicates create

Take `nums = [1, 2, 2]` and run the ordinary Subsets algorithm. The tree happily produces `[1, 2]` twice — once using the first 2 and once using the second. As *index sequences* `(0, 1)` and `(0, 2)` are different, so the algorithm is behaving exactly as designed. As *value sequences* they are identical, and the problem asks for distinct value sequences.

Key Insight

Duplicates break the correspondence *one index-sequence* $\Leftrightarrow$ *one answer*. Our previous guarantee of uniqueness came from that correspondence. To restore it we must choose, among all index-sequences that give the same value-sequence, exactly **one canonical representative** — and prune all the others.

8.2 Why “dedupe with a set at the end” is inferior

You can convert each subset to a tuple, dump them into a `set`, and convert back. It produces the right answer, and interviewers will not fail you for mentioning it. But:

- **Work is not saved.** You still *explore* every duplicate branch. With `[2]*20` the correct answer has 21 subsets; the naive search visits $2^{20} \approx 10^6$ nodes. In-tree skipping visits 21.
- **Memory.** You must materialise all duplicates before discarding them.
- **It does not generalise.** For Combination Sum II the duplicate branches may be enormous subtrees; for N-Queens-style problems there is nothing to hash.
- **It hides understanding.** The skip condition is a two-line idiom that interviewers specifically look for.

8.3 The canonical-representative idea

Sort the array first. Now all equal values are adjacent. Among the several positions holding the same value, agree on this rule:

*When several identical values are available at the same level of the tree, use only the **first** of them.*

Equivalently: if you decide to *skip* a value at a level, you must skip *all* copies of it at that level. That is exactly what the famous condition encodes:

```
for i in range(start, n):
    if i > start and nums[i] == nums[i - 1]:
        continue           # same value, same level, not the first copy
```

8.3.1 Reading the condition piece by piece

<code>nums[i] == nums[i-1]</code>	“this candidate has the same <i>value</i> as the previous candidate” — meaningful only because we sorted.
<code>i > start</code>	“and the previous candidate was offered <i>at this same level</i> ”. When <code>i == start</code> , <code>nums[i-1]</code> belongs to an earlier level (it is the element our parent chose), so this is the <i>first</i> copy at this level and must be kept. skip this branch entirely; its subtree is a duplicate of the one rooted at <code>i-1</code> .
<code>continue</code>	

Key Insight

Same-level duplicates vs different-level duplicates.

- *Same level* (siblings): choosing the 1st or the 2nd copy gives identical subtrees. Keep one, kill the rest. This is what `i > start` detects.
- *Different levels* (ancestor–descendant): `[2, 2]` means “take a 2 and then take another 2”. That is a genuinely different, valid answer. It must be allowed. This is why the condition is `i > start` and **not** `i > 0`.

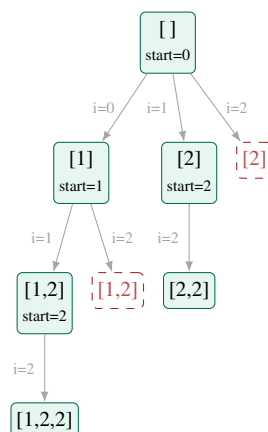
Common Mistake

Writing `if i > 0 and nums[i] == nums[i-1]: continue`. This also forbids taking two equal values on consecutive levels, so `[2, 2]` disappears from Subsets II and `[1, 2, 2]` disappears from Combination Sum II. Symptom: too few answers.

8.4 LeetCode 90 — Subsets II

Problem. `nums` may contain duplicates; return all possible subsets, no duplicate subsets.

8.4.1 K. The tree, `nums = [1, 2, 2]` (sorted)



The two red nodes are killed by `i > start` and `nums[i] == nums[i-1]`:

- At the root, level candidates are $i = 0, 1, 2$. At $i = 2$ we have `nums[2] == nums[1]` and $2 > start = 0 \Rightarrow$ skip. (Taking the second 2 first would rebuild the same `[2]` subtree.)
- Inside `[1]` (`start=1`), at $i = 2$ again $2 > 1$ and values equal $\Rightarrow$ skip.
- But inside `[1, 2]` we have `start=2` and $i = 2$, so $i \not> start$: the branch is **kept**, giving `[1, 2, 2]`. Different level, genuinely new answer.

Result: [], [1], [1,2], [1,2,2], [2], [2,2] — 6 distinct subsets.

8.4.2 L. Python

```

1  class Solution:
2      def subsetsWithDup(self, nums):
3          nums.sort()                # equal values become adjacent
4          res, path = [], []
5
6          def backtrack(start):
7              res.append(path[:])    # every node is a subset
8              for i in range(start, len(nums)):
9                  if i > start and nums[i] == nums[i - 1]:
10                     continue        # skip duplicate SIBLING
11                 path.append(nums[i]) # CHOOSE
12                 backtrack(i + 1)     # EXPLORE (each element once)
13                 path.pop()          # UNDO
14
15         backtrack(0)
16         return res

```

Listing 8.1: LeetCode 90 — Subsets II

8.4.3 M. Java

```

1  import java.util.*;
2
3  class Solution {
4      public List<List<Integer>> subsetsWithDup(int[] nums) {
5          Arrays.sort(nums);
6          List<List<Integer>> res = new ArrayList<>();
7          backtrack(0, nums, new ArrayList<>(), res);
8          return res;
9      }
10
11     private void backtrack(int start, int[] nums,
12                             List<Integer> path, List<List<Integer>> res) {
13         res.add(new ArrayList<>(path));
14         for (int i = start; i < nums.length; i++) {
15             if (i > start && nums[i] == nums[i - 1]) continue; // dup sibling
16             path.add(nums[i]); // CHOOSE
17             backtrack(i + 1, nums, path, res); // EXPLORE
18             path.remove(path.size() - 1); // UNDO
19         }
20     }
21 }

```

Listing 8.2: LeetCode 90 — Subsets II, Java

8.5 LeetCode 40 — Combination Sum II

Problem. candidates may contain duplicates; each element may be used **at most once**; return all unique combinations summing to target.

This is the intersection of two ideas we already have:

- *each element once* $\Rightarrow$ `backtrack(i + 1)` (contrast with LC 39, which used `i`);
- *duplicates in input* $\Rightarrow$ sort + the same-level skip.

8.5.1 L. Python

```

1  class Solution:
2      def combinationSum2(self, candidates, target):
3          candidates.sort()
4          res, path = [], []
5
6          def backtrack(start, t):
7              if t == 0:
8                  res.append(path[:])
9                  return
10             for i in range(start, len(candidates)):
11                 if i > start and candidates[i] == candidates[i - 1]:
12                     continue # duplicate sibling
13                 if candidates[i] > t:
14                     break # sorted → prune rest of level
15                 path.append(candidates[i]) # CHOOSE
16                 backtrack(i + 1, t - candidates[i]) # i+1: use once
17                 path.pop() # UNDO
18
19         backtrack(0, target)
20         return res

```

Listing 8.3: LeetCode 40 — Combination Sum II

8.5.2 M. Java

```

1  import java.util.*;
2
3  class Solution {
4      public List<List<Integer>> combinationSum2(int[] candidates, int target) {
5          Arrays.sort(candidates);
6          List<List<Integer>> res = new ArrayList<>();
7          backtrack(0, target, candidates, new ArrayList<>(), res);
8          return res;
9      }
10
11     private void backtrack(int start, int t, int[] c,
12                             List<Integer> path, List<List<Integer>> res) {
13         if (t == 0) { res.add(new ArrayList<>(path)); return; }
14         for (int i = start; i < c.length; i++) {
15             if (i > start && c[i] == c[i - 1]) continue; // duplicate sibling
16             if (c[i] > t) break; // prune
17             path.add(c[i]);
18             backtrack(i + 1, t - c[i], c, path, res); // use once
19             path.remove(path.size() - 1);
20         }
21     }
22 }

```

Listing 8.4: LeetCode 40 — Combination Sum II, Java

8.5.3 N. Dry run, [1, 1, 2], target 3

Dry Run

Sorted: [1, 1, 2].

#	level (start)	i	decision	path/t
1	0	0	take 1	[1], t=2
2	1	1	$i = \text{start}$, keep; take 1	[1, 1], t=1
3	2	2	$2 > 1 \Rightarrow \text{break}$	—
4	1	2	take 2	[1, 2], t=0 $\Rightarrow$ record
5	0	1	$i > \text{start}$ and $c[1] == c[0] \Rightarrow$ skip	
6	0	2	take 2	[2], t=1
7	3	—	loop empty	no answer

Answer: `[[1, 2]]`. Step 2 shows the different-level duplicate being *allowed* (so `[1, 1]` is reachable when the target permits); step 5 shows the same-level duplicate being *killed*, which is what stops `[1, 2]` appearing twice.

8.6 Permutations II (LeetCode 47): the same idea, different guard

With `visited`, “same level” is no longer expressible as $i > \text{start}$. The equivalent canonical rule is: *among equal values, always consume the leftmost unused copy*. In code:

```

1 class Solution:
2     def permuteUnique(self, nums):
3         nums.sort()
4         n, res, path = len(nums), [], []
5         visited = [False] * n
6
7         def backtrack():
8             if len(path) == n:
9                 res.append(path[:]); return
10            for i in range(n):
11                if visited[i]:
12                    continue
13                # if the previous equal value is NOT used, then this copy is
14                # not the leftmost available one  $\rightarrow$  its subtree is a duplicate
15                if i > 0 and nums[i] == nums[i - 1] and not visited[i - 1]:
16                    continue
17                visited[i] = True; path.append(nums[i])
18                backtrack()
19                path.pop(); visited[i] = False
20
21        backtrack()
22        return res

```

Listing 8.5: LeetCode 47 — Permutations II

Key Insight

`not visited[i-1]` means the previous equal copy is still on the shelf — so we are about to pick the second copy *before* the first, which is the non-canonical ordering. Skip it. If `visited[i-1]` is true, the first copy is already in the path (an ancestor), so this is a legitimate “two equal values in one permutation”.

8.7 P. Common mistakes recap

Common Mistake

Forgetting to sort. The condition compares `nums[i]` with `nums[i-1]`; without sorting, equal values are not adjacent and the skip catches almost nothing. Symptom: duplicates still appear.

Common Mistake

Using `i > start` with a visited array (or `visited[i-1]` with a `start` index). The two guards belong to two different frameworks. Mixing them is a classic interview stumble.

Common Mistake

Sorting inside the recursion. Sort once, before the first call. Sorting at every node adds an $O(n \log n)$ factor per node for no benefit.

8.8 Q/R. Practice

LeetCode 90 — Subsets II, LeetCode 40 — Combination Sum II, LeetCode 47 — Permutations II. Then explain to yourself, out loud, why LC 90 uses `i > start` while LC 47 uses `not visited[i-1]`. If you can do that, you own this pattern.

Part III

Applied Backtracking

Chapter 9

String Backtracking

9.1 A. Why strings deserve their own chapter

Nothing about the algorithm changes — the tree, the choose/explore/undo cycle and the base case are identical. What changes is the *data structure for the path*, and that change is where the bugs live.

	Python	Java
str	immutable: <code>s += c</code> builds a new string, $O(\text{len})$	String is immutable too; <code>s + c</code> is $O(\text{len})$
Recommended path	list of chars + <code>"".join(path)</code> at the leaf	StringBuilder
CHOOSE	<code>path.append(c)</code>	<code>sb.append(c)</code>
UNDO	<code>path.pop()</code>	<code>sb.deleteCharAt(sb.length()-1)</code>
Snapshot	<code>"".join(path)</code> (new object)	<code>sb.toString()</code> (new object)

Key Insight

There are two legitimate styles. (a) *Mutable buffer + undo* — list/StringBuilder: $O(1)$ per choice, one $O(L)$ copy at each leaf. This is the interview-standard. (b) *Immutable parameter, no undo* — pass `s + c` down: no undo needed, but $O(L)$ allocation at *every node*. Perfectly acceptable for short strings; say why you chose it.

9.2 LeetCode 17 — Letter Combinations of a Phone Number

Problem. Given a string of digits 2–9, return all letter combinations the number could spell, using the classic phone keypad.

9.2.1 A–C. Intuition and state

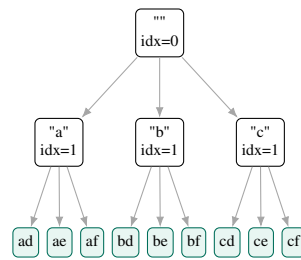
The input *is* the tree. Digit 0 picks the first letter, digit 1 the second, and so on. The branching factor is not constant: it is 3 for most digits and 4 for 7 and 9. Depth is exactly `len(digits)`.

state = (index into digits, path)

9.2.2 D–G

Choices	each letter mapped to <code>digits[index]</code>
Base case	<code>index == len(digits) ⇒ record "".join(path)</code>
Edge case	empty input must return <code>[]</code> , not <code>[""]</code> — handle before recursing
Constraint / prune	none: every branch reaches a valid leaf
Undo	<code>path.pop()</code>

9.2.3 K. Recursion tree for "23"



$3 \times 3 = 9$ leaves. For "79" it would be $4 \times 4 = 16$.

9.2.4 L. Python

```

1 class Solution:
2     def letterCombinations(self, digits):
3         if not digits:
4             return []                # edge case: [] not [""]
5
6         keypad = {"2": "abc", "3": "def", "4": "ghi", "5": "jkl",
7                  "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz"}
8         res, path = [], []
9
10        def backtrack(index):
11            if index == len(digits):
12                res.append("".join(path))    # join → new immutable string
13                return
14            for ch in keypad[digits[index]]:
15                path.append(ch)              # CHOOSE
16                backtrack(index + 1)         # EXPLORE (always +1: fixed length)
17                path.pop()                  # UNDO
18
19        backtrack(0)
20        return res

```

Listing 9.1: LeetCode 17 — Letter Combinations

9.2.5 M. Java

```

1 import java.util.*;
2
3 class Solution {
4     private static final String[] KEYPAD = {
5         "", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"
6     };
7
8     public List<String> letterCombinations(String digits) {
9         List<String> res = new ArrayList<>();
10        if (digits == null || digits.isEmpty()) return res;
11        backtrack(0, digits, new StringBuilder(), res);
12        return res;
13    }
14
15    private void backtrack(int index, String digits,
16                           StringBuilder sb, List<String> res) {
17        if (index == digits.length()) {
18            res.add(sb.toString());          // snapshot
19            return;
20        }

```

```

21     String letters = KEYPAD[digits.charAt(index) - '0'];
22     for (int i = 0; i < letters.length(); i++) {
23         sb.append(letters.charAt(i));           // CHOOSE
24         backtrack(index + 1, digits, sb, res); // EXPLORE
25         sb.deleteCharAt(sb.length() - 1);      // UNDO
26     }
27 }
28 }

```

Listing 9.2: LeetCode 17 — Letter Combinations, Java

9.2.6 O. Complexity

With m digits mapping to 3 letters and p digits mapping to 4: $O(4^p \cdot 3^m \cdot (m + p))$ time (the trailing factor is the join at each leaf), $O(m + p)$ auxiliary space.

9.2.7 P. Common mistakes

Common Mistake

Returning `[]` for empty input. LeetCode expects `[]`. Guard first.

Common Mistake

Reusing one `StringBuilder` but forgetting `deleteCharAt`. Output becomes `["ad", "adae", ...]` — monotonically growing garbage. The tell-tale symptom of a missing undo on a buffer.

9.3 LeetCode 22 — Generate Parentheses

Problem. Given n pairs of parentheses, generate all combinations of *well-formed* parentheses.

9.3.1 A. Intuition: constraints as the whole problem

Naively there are 2^{2n} strings of length $2n$ over $\{ (,) \}$, of which only $C_n = \frac{1}{n+1} \binom{2n}{n}$ are valid ($n = 3$: 64 candidates, 5 valid). Generating all and filtering is legal but wasteful. Instead we ask: *when is a prefix still extendable?* Two conditions, and they are enough:

open < n	we may add ' (' only if we have not already used all n opening brackets.
close < open	we may add ')' only if there is an unmatched ' (' to close. If close == open, adding ')' makes the prefix invalid forever.

Key Insight

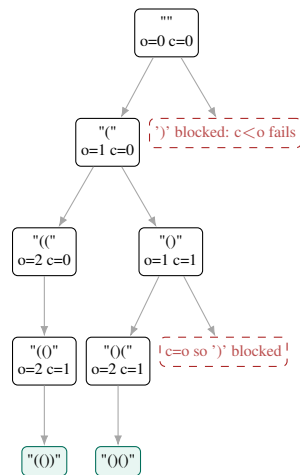
This is pruning, not filtering. A prefix with more ')' than '(' can never be repaired by any suffix — invalidity is *monotone*. So the moment the condition fails we can delete the entire subtree. Recognising monotone invalidity is the single most useful pruning skill; it is the same reasoning as $t < 0$ in Combination Sum.

9.3.2 C–G

state = (open, close, path)

Base case: `len(path) == 2n` (equivalently `close == n`). Choices: two, each guarded. Undo: `path.pop()`.

9.3.3 K. Recursion tree for $n = 2$



Only 2 leaves for $n = 2$ ($C_2 = 2$), and the two red nodes are never even created.

Intuition

Draw the string as a mountain path: ' (' steps up, ') ' steps down. Valid strings are paths that never go below sea level and end exactly at sea level after $2n$ steps. `close < open` is “don’t go below sea level”; `open < n` is “don’t climb higher than n ”. These are Dyck paths, counted by the Catalan numbers.

9.3.4 L. Python

```

1 class Solution:
2     def generateParenthesis(self, n):
3         res, path = [], []
4
5     def backtrack(open_count, close_count):
6         if len(path) == 2 * n:           # base case: full length
7             res.append("".join(path))
8             return
9
10        if open_count < n:                 # PRUNE: only if capacity left
11            path.append("(")               # CHOOSE
12            backtrack(open_count + 1, close_count)
13            path.pop()                     # UNDO
14
15        if close_count < open_count:       # PRUNE: only if something to close
16            path.append(")")               # CHOOSE
17            backtrack(open_count, close_count + 1)
18            path.pop()                     # UNDO
19
20    backtrack(0, 0)
21    return res

```

Listing 9.3: LeetCode 22 — Generate Parentheses

9.3.5 M. Java

```
1 import java.util.*;
2
3 class Solution {
4     public List<String> generateParenthesis(int n) {
```

```

5      List<String> res = new ArrayList<>();
6      backtrack(0, 0, n, new StringBuilder(), res);
7      return res;
8  }
9
10     private void backtrack(int open, int close, int n,
11                             StringBuilder sb, List<String> res) {
12         if (sb.length() == 2 * n) {
13             res.add(sb.toString());
14             return;
15         }
16         if (open < n) { // capacity for '('
17             sb.append('(');
18             backtrack(open + 1, close, n, sb, res);
19             sb.deleteCharAt(sb.length() - 1);
20         }
21         if (close < open) { // something to close
22             sb.append(')');
23             backtrack(open, close + 1, n, sb, res);
24             sb.deleteCharAt(sb.length() - 1);
25         }
26     }
27 }

```

Listing 9.4: LeetCode 22 — Generate Parentheses, Java

9.3.6 N. Dry run, $n = 2$

Dry Run

#	path	open	close	event
1	" "	0	0	) blocked ($0 < 0$ false); take (
2	" ("	1	0	both allowed; take (
3	" (("	2	0	(blocked ($2 < 2$); take)
4	" (() "	2	1	take)
5	" (()) "	2	2	length 4 $\Rightarrow$ record
6	backtrack to " ("	1	0	take)
7	" () "	1	1	) blocked ($1 < 1$ false); take (
8	" () ("	2	1	take)
9	" () () "	2	2	record

Answer [" (()) ", " () () "]. Steps 1 and 7 are the pruned branches; without those guards we would have generated ") "-prefixed garbage and wasted 62 of 64 candidates.

9.3.7 O. Complexity

The number of leaves is the n -th Catalan number $C_n = \frac{1}{n+1} \binom{2n}{n} \sim \frac{4^n}{n^{3/2}\sqrt{\pi}}$. Each leaf costs $O(n)$ to build the string, and because the pruning admits *only* valid prefixes, every internal node lies on a path to at least one leaf. Hence

$$\text{Time} = O\left(\frac{4^n}{\sqrt{n}}\right), \quad \text{Space} = O(n) \text{ auxiliary.}$$

Interview Tip

Saying “the answer count is the Catalan number, so the time is $\Theta(n \cdot C_n) = O(4^n / \sqrt{n})$ ” is a genuinely strong answer. If you cannot recall Catalan, say “at most $O(2^{2n})$ candidates, but pruning restricts us to valid prefixes only, so the work is proportional to the output size.” That is honest and correct.

9.3.8 P. Common mistakes**Common Mistake**

Using `close < n` instead of `close < open`. Then `) ) ( (` is generated: you have bounded the *count* of closers but not their *legality*.

Common Mistake

Base case `open == n` and `close == n` written as `open == n` only. Records incomplete strings such as `" (`.

Common Mistake

Validating at the leaf instead of pruning at each node. Correct, but $12\times$ slower at $n = 8$ and it throws away the whole lesson of the problem.

9.3.9 Q/R. Practice

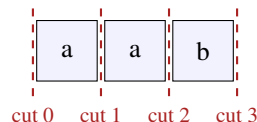
LeetCode 784 — Letter Case Permutation, **LeetCode 93 — Restore IP Addresses** (string cuts + validity constraint), **LeetCode 301 — Remove Invalid Parentheses** (hard: pruning by minimum removals).

Chapter 10

Partitioning Backtracking

10.1 A. Intuition: choosing where to cut

So far a “choice” has been *which element to append*. In partitioning problems a choice is *where to place the next cut*. Given $s = \text{"aab"}$, the gaps between characters are the cut positions:



A partition of a length- n string is a subset of the $n - 1$ interior cut positions, so there are 2^{n-1} partitions — exponential, hence backtracking.

Key Insight

Reframe: at position `start`, the choice is *how long the next piece is*. Piece lengths $1, 2, \dots, n - \text{start}$ correspond to the loop `for end in range(start+1, n+1)`, and the recursion continues at `end`. This is Dialect B with “prefix length” as the choice.

10.2 LeetCode 131 — Palindrome Partitioning

Problem. Partition s such that every substring of the partition is a palindrome. Return all possible partitionings.

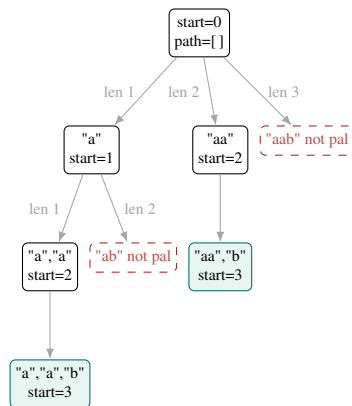
10.2.1 B. Real-world analogy

Cutting a ribbon printed with letters into pieces, where a piece is only allowed if its printing reads the same from both ends. You cut left to right; once a cut is made you never revisit it.

10.2.2 C–G. Ingredients

State	$(\text{start}, \text{path})$ — <code>start</code> is the first uncut index; <code>path</code> holds the pieces already cut
Base case	<code>start == len(s)</code> — the whole string is consumed $\Rightarrow$ record
Choices	<code>end</code> in <code>start+1 .. n</code> ; the piece is <code>s[start:end]</code>
Constraint	<code>s[start:end]</code> must be a palindrome
Pruning	the constraint <i>is</i> the pruning: a non-palindromic first piece kills the entire subtree of ways to cut the rest
Undo	<code>path.pop()</code>

10.2.3 K. Recursion tree for "aab"



Two solutions, matching the expected output:

a | a | b and aa | b

10.2.4 L. Python

```

1 class Solution:
2     def partition(self, s):
3         n = len(s)
4         res, path = [], []
5
6         def is_pal(lo, hi):
7             # s[lo:hi] inclusive-exclusive
8             hi -= 1
9             while lo < hi:
10                 if s[lo] != s[hi]:
11                     return False
12                 lo += 1
13                 hi -= 1
14             return True
15
16         def backtrack(start):
17             if start == n:
18                 # consumed the whole string
19                 res.append(path[:])
20                 return
21             for end in range(start + 1, n + 1):
22                 # piece = s[start:end]
23                 if not is_pal(start, end):
24                     continue
25                 # CONSTRAINT / PRUNE
26                 path.append(s[start:end])
27                 # CHOOSE
28                 backtrack(end)
29                 # EXPLORE from the cut
30                 path.pop()
31                 # UNDO
32
33         backtrack(0)
34         return res
  
```

Listing 10.1: LeetCode 131 — Palindrome Partitioning

10.2.5 M. Java

```

1 import java.util.*;
2
3 class Solution {
4     public List<List<String>> partition(String s) {
5         List<List<String>> res = new ArrayList<>();
  
```

```

6      backtrack(0, s, new ArrayList<>(), res);
7      return res;
8  }
9
10     private void backtrack(int start, String s,
11                             List<String> path, List<List<String>> res) {
12         if (start == s.length()) {
13             res.add(new ArrayList<>(path));
14             return;
15         }
16         for (int end = start + 1; end ≤ s.length(); end++) {
17             if (!isPal(s, start, end - 1)) continue; // constraint
18             path.add(s.substring(start, end)); // CHOOSE
19             backtrack(end, s, path, res); // EXPLORE
20             path.remove(path.size() - 1); // UNDO
21         }
22     }
23
24     private boolean isPal(String s, int lo, int hi) {
25         while (lo < hi) {
26             if (s.charAt(lo++) ≠ s.charAt(hi--)) return false;
27         }
28         return true;
29     }
30 }

```

Listing 10.2: LeetCode 131 — Palindrome Partitioning, Java

Common Mistake

Recurring with `start + 1` instead of `end`. The next piece must begin where this one ended. `start+1` produces overlapping pieces and nonsense output. Say it to yourself: “the cut is at `end`, so the next call starts at `end`.”

10.2.6 N. Dry run, "aab"**Dry Run**

#	call	piece tried	outcome
1	bt (0)	"a"	palindrome ⇒ push, bt (1)
2	bt (1)	"a"	palindrome ⇒ push, bt (2)
3	bt (2)	"b"	palindrome ⇒ push, bt (3)
4	bt (3)	—	start==n ⇒ record ["a", "a", "b"]
5	unwind	"ab" (at bt (1))	not a palindrome ⇒ skip
6	unwind to bt (0)	"aa"	palindrome ⇒ push, bt (2)
7	bt (2)	"b"	push, bt (3) ⇒ record ["aa", "b"]
8	unwind	"aab"	not a palindrome ⇒ skip; done

10.2.7 O. Complexity

- There are at most 2^{n-1} partitions, so $O(2^n)$ leaves.
- Each palindrome check costs $O(n)$ and each leaf copy costs $O(n)$.
- **Time:** $O(n \cdot 2^n)$.
- **Space:** $O(n)$ recursion depth (plus output).

10.3 Optimisation 1 — precomputed palindrome table

Notice we may test `is_pal(2, 5)` many times along different branches. Precompute a boolean table once in $O(n^2)$ and every check becomes $O(1)$:

$$\text{pal}[i][j] = (s_i = s_j) \wedge (j - i < 2 \vee \text{pal}[i + 1][j - 1])$$

```

1 class Solution:
2     def partition(self, s):
3         n = len(s)
4         pal = [[False] * n for _ in range(n)]
5         # expand by increasing length so pal[i+1][j-1] is already known
6         for i in range(n - 1, -1, -1):
7             for j in range(i, n):
8                 if s[i] == s[j] and (j - i < 2 or pal[i + 1][j - 1]):
9                     pal[i][j] = True
10
11         res, path = [], []
12
13         def backtrack(start):
14             if start == n:
15                 res.append(path[:]); return
16             for end in range(start, n):
17                 if pal[start][end]: # O(1) constraint check
18                     path.append(s[start:end + 1])
19                     backtrack(end + 1)
20                     path.pop()
21
22         backtrack(0)
23         return res

```

Listing 10.3: Palindrome Partitioning with an $O(1)$ palindrome test

```

1 boolean[][] pal = new boolean[n][n];
2 for (int i = n - 1; i ≥ 0; i--)
3     for (int j = i; j < n; j++)
4         pal[i][j] = s.charAt(i) == s.charAt(j)
5             && (j - i < 2 || pal[i + 1][j - 1]);

```

Listing 10.4: Palindrome table in Java

Key Insight

This is **not** memoisation of the search. It is memoisation of the *constraint check*. The tree still has 2^{n-1} leaves — unavoidable, they are the output — but the per-node cost drops from $O(n)$ to $O(1)$. Distinguishing “cheaper nodes” from “fewer nodes” is exactly the kind of precision interviewers reward.

10.4 Optimisation 2 — when memoisation actually applies

If the question were changed to “*what is the minimum number of cuts?*” (**LeetCode 132 — Palindrome Partitioning II**), the answer for suffix `s[start:]` no longer depends on how we got to `start`. Then `memo[start]` is legitimate and the complexity collapses to $O(n^2)$.

```

1 from functools import lru_cache
2
3 def min_cut(s):
4     n = len(s)
5     pal = [[False] * n for _ in range(n)]
6     for i in range(n - 1, -1, -1):

```

```

7         for j in range(i, n):
8             if s[i] == s[j] and (j - i < 2 or pal[i + 1][j - 1]):
9                 pal[i][j] = True
10
11     @lru_cache(maxsize=None)
12     def cuts(start):
13         if start == n:
14             return -1
15         best = float("inf")
16         for end in range(start, n):
17             if pal[start][end]:
18                 best = min(best, 1 + cuts(end + 1))
19         return best
20
21     return cuts(0)

```

Listing 10.5: Min-cuts variant: the state is memoisable

Key Insight

The rule. Memoise when the return value depends only on the state, not on the path. “All partitions of the suffix” depends only on `start` too — but the *answers* must each be prefixed with the current path, so caching them saves re-enumeration at the cost of a copy per reuse. Caching an *aggregate* (a number, a boolean) is always safe and always cheap. See Chapter 15.

10.5 P. Common mistakes

Common Mistake

Off-by-one between end as exclusive and inclusive. Python slices are exclusive (`s[start:end]`); a two-pointer `is_pal(lo, hi)` is usually inclusive. Pick one convention per function and write it in a comment.

Common Mistake

Base case if `start >= n` combined with a loop that can overshoot. Use `==` and make sure `end <= n` so it can never be exceeded — an assertion-style base case catches bugs instead of hiding them.

Common Mistake

Building substrings before checking the palindrome property. `s.substring` in Java allocates $O(n)$; check first, allocate second.

10.6 Q/R. Practice

LeetCode 131 — Palindrome Partitioning, **LeetCode 93 — Restore IP Addresses** (partition into exactly 4 pieces with a numeric constraint), **LeetCode 139 — Word Break** / **LeetCode 140 — Word Break II** (partition with a dictionary constraint — and the classic example where memoisation turns an exponential search into a polynomial one).

Chapter 11

Grid Backtracking

11.1 A. Intuition

On a grid the “path” is a walk, and the “choices” are the four directions. Everything else is the same template. The two new ideas are:

1. **Boundary checks:** a choice may leave the board.
2. **Path-local visited:** a cell may not be reused *by the walk currently in progress*, but it is perfectly reusable by a different walk started elsewhere.

Key Insight

This is the sharpest place to see the difference from graph DFS. In connectivity DFS you mark a cell visited and never unmark it, because visiting it twice is pointless. In Word Search you *must* unmark it on the way out, because a different starting cell may legitimately need to walk through it.

11.2 LeetCode 79 — Word Search

Problem. Given an $m \times n$ board of characters and a word, return `true` if the word can be constructed from letters of sequentially adjacent cells (horizontally or vertically neighbouring), where the same cell may not be used more than once.

11.2.1 B. Real-world analogy

Tracing a word in a word-search puzzle with your finger. Your finger may not touch the same square twice *in one word*, but starting a new word you may reuse any square.

11.2.2 C–G. Ingredients

State	(r, c, k) — current cell and how many characters of the word are matched; plus the board itself, mutated in place
Base case (accept)	<code>k == len(word) ⇒ return True</code>
Base case (reject)	out of bounds, or <code>board[r][c] != word[k]</code> , or cell already on this path
Choices	4 directions: up, down, left, right
Constraint	bounds + letter match + not on current path
Pruning	the letter-match test itself: one wrong letter kills a whole subtree
Undo	restore <code>board[r][c]</code> to its original character

11.2.3 Two ways to mark a cell

(a) Separate visited matrix	(b) Mutate the board
<code>visited[r][c] = True...False</code>	<code>board[r][c] = '#' ...restore</code>
$O(mn)$ extra memory	$O(1)$ extra memory
Non-destructive	Temporarily destructive; must restore exactly
Safer, clearer	Slicker; commonly expected in interviews

Both are correct. We show (b) and mention (a); note that (b) is only acceptable if the board may be modified transiently and is restored before returning — which our undo guarantees.

11.2.4 K. Visualising a search

Board and the word "ABCCED":

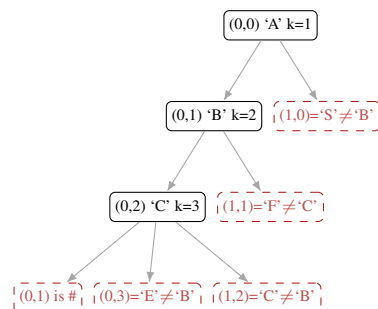
A	B	C	E
S	F	C	S
A	D	E	E

path for "ABCCED"

Now a *failing* attempt for the word "ABCB": we match A (0, 0), B (0, 1), C (0, 2), then look for B. The neighbours of (0, 2) are (0, 1)=B — but it is already on our path, marked # — and (0, 3)=E, (1, 2)=C. Nothing matches, so the call returns *False* and we undo back up.

#	#	#	E
S	F	C	S
A	D	E	E

cells on the current path are #
no neighbour of (0,2) equals 'B'
⇒ return false, restore 'C', 'B', 'A'



11.2.5 L. Python

```

1 class Solution:
2     def exist(self, board, word):
3         if not board or not board[0]:
4             return False
5         rows, cols = len(board), len(board[0])
6
7         def backtrack(r, c, k):
8             if k == len(word):          # ACCEPT: whole word matched
9                 return True
10            if r < 0 or r ≥ rows or c < 0 or c ≥ cols:
11                return False            # REJECT: off the board
12            if board[r][c] ≠ word[k]:    # REJECT: wrong letter (or '#')
13                return False
14
15            board[r][c] = "#"            # CHOOSE: mark as on-path
16            found = (backtrack(r + 1, c, k + 1) or      # EXPLORE 4 directions
17                    backtrack(r - 1, c, k + 1) or
18                    backtrack(r, c + 1, k + 1) or
19                    backtrack(r, c - 1, k + 1))

```

```

20         board[r][c] = word[k]                # UNDO: restore the letter
21         return found
22
23     for r in range(rows):
24         for c in range(cols):
25             if backtrack(r, c, 0):            # try every starting cell
26                 return True
27     return False

```

Listing 11.1: LeetCode 79 — Word Search

Key Insight

`board[r][c] = word[k]` is the correct restore because we only reach that line after verifying `board[r][c] == word[k]`. Writing `board[r][c] = '?'` or forgetting to save the character are the two ways to corrupt the board permanently.

11.2.6 M. Java

```

1  class Solution {
2      private static final int[][] DIRS = {{1,0},{-1,0},{0,1},{0,-1}};
3
4      public boolean exist(char[][] board, String word) {
5          if (board.length == 0 || board[0].length == 0) return false;
6          for (int r = 0; r < board.length; r++)
7              for (int c = 0; c < board[0].length; c++)
8                  if (backtrack(board, word, r, c, 0)) return true;
9          return false;
10     }
11
12     private boolean backtrack(char[][] b, String w, int r, int c, int k) {
13         if (k == w.length()) return true;                // ACCEPT
14         if (r < 0 || r ≥ b.length || c < 0 || c ≥ b[0].length) return false;
15         if (b[r][c] ≠ w.charAt(k)) return false;        // REJECT
16
17         char saved = b[r][c];
18         b[r][c] = '#';                                   // CHOOSE
19         for (int[] d : DIRS) {
20             if (backtrack(b, w, r + d[0], c + d[1], k + 1)) {
21                 b[r][c] = saved;                          // restore before
22                 return true;                               // early return!
23             }
24         }
25         b[r][c] = saved;                                   // UNDO
26         return false;
27     }
28 }

```

Listing 11.2: LeetCode 79 — Word Search, Java

Common Mistake

Returning true without restoring the board. On LeetCode you usually get away with it, but the board is left mutated — a real bug if the caller reuses it, and an instant follow-up question from a sharp interviewer. Restore on *every* exit path, or structure the code so there is exactly one exit (as in the Python version).

11.2.7 N. Dry run, board `[ [A,B] , [C,D] ]`, word `"ABD"`

Dry Run

#	call	board state	result
1	<code>bt(0,0,0)</code>	A matches 'A'; mark #	explore
2	<code>bt(1,0,1)</code>	C vs 'B'	false
3	<code>bt(-1,0,1)</code>	out of bounds	false
4	<code>bt(0,1,1)</code>	B matches; mark #	explore
5	<code>bt(1,1,2)</code>	D matches; mark #	explore
6	<code>bt(2,1,3)</code>	<code>k==3==len</code>	true
7	unwind	each frame restores its letter	true

Step 6 shows an important subtlety: the accept check comes *before* the bounds check, so a match on the last character returns true even though the next call's coordinates are off-board. Order those two `ifs` the other way round and correct words at the edge of the board are rejected.

11.2.8 O. Complexity

Let $L = \text{len}(\text{word})$, board $m \times n$.

- From each cell the first step has 4 options; every later step has at most 3 (we never immediately go back to the cell we came from, since it is marked).
- So one start explores $O(4 \cdot 3^{L-1}) = O(3^L)$ paths.
- There are mn starts. **Time:** $O(m \cdot n \cdot 3^L)$.
- **Space:** $O(L)$ recursion depth (board mutated in place).

11.2.9 P. Common mistakes

Common Mistake

Forgetting the undo $\Rightarrow$ cells stay # forever, so later starting cells find a board full of holes and the function returns `false` for words that exist.

Common Mistake

Checking `k == len(word)` after the bounds check. As shown in the dry run, this rejects valid words ending on the boundary.

Common Mistake

Using a global `visited` that is never reset between starting cells. Same symptom as forgetting the undo.

Common Mistake

Using `|` instead of `or` in Python (`or |` instead of `||` in Java). The non-short-circuiting operator explores all four directions even after success, which is slower and, worse, mutates state you assumed untouched.

11.2.10 Useful prunes

- If `len(word) > rows * cols`, return `False` immediately.

- Count letter frequencies in the board; if any letter of `word` occurs more often in the word than on the board, return `False`.
- If the last letter of the word is rarer on the board than the first, search for the *reversed* word — fewer starting cells to try.

11.2.11 Q/R. Practice

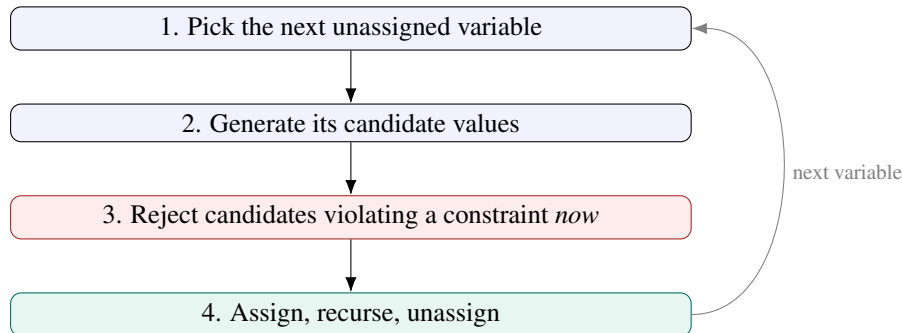
LeetCode 200 — Number of Islands (DFS with permanent visited — the contrast case), **LeetCode 212 — Word Search II** (the same search plus a trie, so that one traversal handles all words), **LeetCode 980 — Unique Paths III** (grid backtracking where you must cover every empty cell).

Chapter 12

Constraint Backtracking — N-Queens

12.1 A. Constraint satisfaction as a search pattern

A constraint-satisfaction problem (CSP) gives you variables, domains, and constraints, and asks for an assignment satisfying all constraints. The backtracking recipe is always:



Key Insight

The art is in step 3: checking constraints as *early* and as *cheaply* as possible. Every constraint you can test at depth k instead of depth n removes an entire b^{n-k} -sized subtree.

12.2 LeetCode 51 — N-Queens

Problem. Place n queens on an $n \times n$ board so that no two attack each other. Return all distinct solutions.

12.2.1 The first modelling decision (and it is the whole problem)

Naively, the choice is “which of the n^2 squares gets the next queen”, giving $\binom{n^2}{n}$ candidates ($n = 8$: 4.4×10^9). But two queens on the same row attack each other, so *every solution has exactly one queen per row*. Therefore:

Variable = row. Domain = which column. Depth = n . Branching $\leq n$.

That reduces the space to n^n before constraints, and to $n!$ after the column constraint alone. Modelling beat brute force by six orders of magnitude before we wrote a single `if`.

Interview Tip

State this reduction out loud before coding: “Since no two queens share a row, I will place exactly one queen per row and let the recursion index be the row.” It is the insight the interviewer is waiting for.

12.2.2 C–G. Ingredients

State	row, plus three sets: cols, diag1, diag2, plus the partial placement <code>path[row] = col</code>
Base case	<code>row == n</code> $\Rightarrow$ render the board and record
Choices	<code>col</code> in <code>0..n-1</code>
Constraints	<code>col</code> unused; both diagonals through <code>(row, col)</code> unused
Pruning	the constraints themselves; optionally symmetry breaking
Undo	remove <code>col</code> and both diagonal keys from their sets

12.3 The diagonal formulas

Two squares (r_1, c_1) and (r_2, c_2) lie on the same

- **main diagonal** (“\”, top-left to bottom-right) iff $r_1 - c_1 = r_2 - c_2$;
- **anti-diagonal** (“/”, top-right to bottom-left) iff $r_1 + c_1 = r_2 + c_2$.

Why. Moving one step down-right changes r by $+1$ and c by $+1$, so $r - c$ is invariant while $r + c$ increases by 2. Moving one step down-left changes r by $+1$ and c by -1 , so $r + c$ is invariant. Each quantity is therefore constant on — and uniquely labels — one family of diagonals.

$r - c$: main diagonals “\”

0	-1	-2	-3
1	0	-1	-2
2	1	0	-1
3	2	1	0

$r + c$: anti-diagonals “/”

0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6

Equal numbers form exactly one diagonal line. Note $r - c \in [-(n-1), n-1]$, so if you use an array instead of a set you must offset by $n-1$: `diag1[r - c + n - 1]`.

Common Mistake

Using $r - c$ as an array index without the offset $\Rightarrow$ negative index. In Python that silently wraps around to the end of the list and produces wrong answers; in Java it throws. Prefer a `set` (Python) / `HashSet<Integer>` (Java) and the problem disappears.

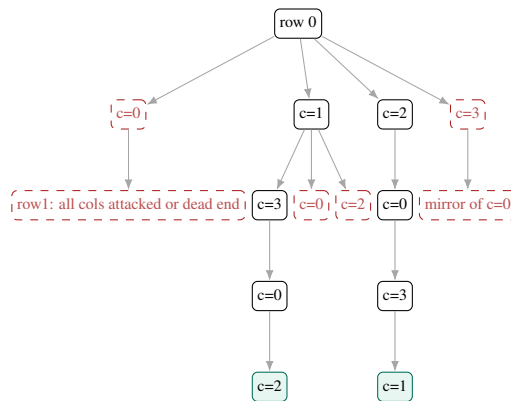
12.3.1 A 4-queens solution board

	Q		
			Q
Q			
		Q	

solution `[1, 3, 0, 2]`: row i has its queen in column `path[i]`

Check: columns $\{1, 3, 0, 2\}$ all distinct; $r - c = \{-1, -2, 2, 1\}$ distinct; $r + c = \{1, 4, 2, 5\}$ distinct. Valid.

12.3.2 K. The search tree for $n = 4$



Only 2 solutions for $n = 4$: $[1, 3, 0, 2]$ and $[2, 0, 3, 1]$. The full unpruned tree would have $4^4 = 256$ leaves; with the three constraints we visit a few dozen nodes.

12.3.3 L. Python

```

1 class Solution:
2     def solveNQueens(self, n):
3         res = []
4         path = [-1] * n          # path[r] = column of the queen in row r
5         cols, diag1, diag2 = set(), set(), set() # diag1: r-c, diag2: r+c
6
7     def backtrack(row):
8         if row == n:             # ACCEPT
9             res.append(["." * c + "Q" + "." * (n - c - 1) for c in path])
10            return
11        for col in range(n):
12            if (col in cols or
13                (row - col) in diag1 or
14                (row + col) in diag2):
15                continue          # PRUNE this branch
16            cols.add(col)          # CHOOSE
17            diag1.add(row - col)
18            diag2.add(row + col)
19            path[row] = col
20            backtrack(row + 1)      # EXPLORE next row
21            cols.remove(col)        # UNDO (all three!)
22            diag1.remove(row - col)
23            diag2.remove(row + col)
24
25        backtrack(0)
26        return res

```

Listing 12.1: LeetCode 51 — N-Queens

12.3.4 M. Java

```

1 import java.util.*;
2
3 class Solution {
4     public List<List<String>> solveNQueens(int n) {
5         List<List<String>> res = new ArrayList<>();
6         int[] path = new int[n];
7         Set<Integer> cols = new HashSet<>();
8         Set<Integer> diag1 = new HashSet<>(); // r - c

```

```

9      Set<Integer> diag2 = new HashSet<>(); // r + c
10     backtrack(0, n, path, cols, diag1, diag2, res);
11     return res;
12 }
13
14 private void backtrack(int row, int n, int[] path, Set<Integer> cols,
15                       Set<Integer> d1, Set<Integer> d2,
16                       List<List<String>> res) {
17     if (row == n) { res.add(render(path, n)); return; }
18
19     for (int col = 0; col < n; col++) {
20         if (cols.contains(col) || d1.contains(row - col)
21             || d2.contains(row + col)) continue;
22         cols.add(col); d1.add(row - col); d2.add(row + col); // CHOOSE
23         path[row] = col;
24         backtrack(row + 1, n, path, cols, d1, d2, res); // EXPLORE
25         cols.remove(col); d1.remove(row - col); // UNDO
26         d2.remove(row + col);
27     }
28 }
29
30 private List<String> render(int[] path, int n) {
31     List<String> board = new ArrayList<>();
32     for (int c : path) {
33         char[] row = new char[n];
34         Arrays.fill(row, '.');
35         row[c] = 'Q';
36         board.add(new String(row));
37     }
38     return board;
39 }
40 }

```

Listing 12.2: LeetCode 51 — N-Queens, Java

Common Mistake

In Java, `Set<Integer>.remove(row - col)` is ambiguous-looking but fine — `Set` has no `remove(int index)` overload, so autoboxing applies. This is the one place where the `List` pitfall does *not* bite. Know the difference: it is a `List` problem, not a collections problem.

12.3.5 The bitmask version (for the follow-up)

```

1  def totalNQueens(n):
2      full = (1 << n) - 1
3
4      def solve(cols, d1, d2):
5          if cols == full:
6              return 1
7          count = 0
8          available = ~(cols | d1 | d2) & full # bits that are safe
9          while available:
10             bit = available & -available # lowest set bit
11             available -= bit
12             count += solve(cols | bit, (d1 | bit) << 1 & full,
13                           (d2 | bit) >> 1)
14             return count
15
16     return solve(0, 0, 0)

```

Listing 12.3: N-Queens II (counting) with bitmasks — $O(1)$ constraint checks

Key Insight

Shifting `d1` left by one each row expresses “a diagonal threat moves one column right as you go one row down”. This eliminates hashing entirely; each check is a single AND. It is a classic senior-level follow-up, but never lead with it — lead with the readable sets version and offer this as an optimisation.

12.3.6 N. Dry run, $n = 4$, first branch**Dry Run**

#	row / col	sets after CHOOSE	outcome
1	r0, c0	cols{0}, d1{0}, d2{0}	descend
2	r1, c0	col taken	skip
3	r1, c1	$r - c = 0 \in d1$	skip (diagonal)
4	r1, c2	cols{0,2}, d1{0,-1}, d2{0,3}	descend
5	r2: c0 col, c1 $r + c = 3 \in d2$, c2 col, c3 $r - c = -1 \in d1$	—	dead end
6	backtrack to r1, undo all three sets	cols{0}	try c3
7	r1,c3 $\rightarrow$ r2 c1 $\rightarrow$ r3: no safe column	—	dead end
8	backtrack to r0, undo	{}	try c1 $\Rightarrow$ leads to [1, 3, 0]

Step 5 is pruning doing its job: an entire depth-2 subtree (4^2 leaves) is eliminated by three cheap set lookups.

12.3.7 O. Complexity

- Without diagonals, the column constraint alone bounds the leaves by $n!$.
- Diagonals cut it much further; the actual node count grows roughly like $n!/c^n$ but has no closed form. Standard answer: $O(n!)$ **time**, with the caveat that pruning makes the practical constant far smaller.
- Rendering each solution costs $O(n^2)$, so producing all boards is $O(n! \cdot n^2)$; the counting variant (LC 52) avoids that factor.
- **Space:** $O(n)$ for the sets, path, and recursion.

12.3.8 P. Common mistakes**Common Mistake**

Undoing only two of the three sets. A stale entry in `diag2` silently forbids legal placements later, and you return too few solutions. Group the three adds and the three removes so they are visually symmetric.

Common Mistake

Checking “does any earlier queen attack this square” with an $O(n)$ loop each time. Correct, but $O(n)$ per candidate instead of $O(1)$. Fine as a first pass — say “I will start with the $O(n)$ check and then optimise to sets”.

Common Mistake

Scanning all n^2 squares as candidates. You will produce each solution $n!$ times and time out. One queen per row is not an optimisation; it is the model.

Common Mistake

Anti-diagonal written as $c - r$. It works (it is just $-(r - c)$) but only if you use it consistently for the *main* diagonal. The two families are $r - c$ and $r + c$; $c - r$ is the main diagonal again, so using $r - c$ and $c - r$ as your two sets leaves anti-diagonal attacks undetected.

12.3.9 Q/R. Practice

LeetCode 52 — N-Queens II (count only — try the bitmask version), **LeetCode 2001 — Number of Pairs of Interchangeable Rectangles** is unrelated; instead try *Knight's Tour* and *Graph Colouring* as classic CSP practice, then Sudoku in the next chapter.

Chapter 13

Sudoku Solver

13.1 Why Sudoku is not “simply brute force”

A 9×9 Sudoku with 51 empty cells has $9^{51} \approx 4 \times 10^{48}$ fillings. At a billion boards per second, checking them all takes longer than the age of the universe by many orders of magnitude. Yet a naive backtracking solver finishes a newspaper puzzle in milliseconds. The difference is entirely due to *incremental constraint checking*:

- Brute force assigns all 51 cells and *then* validates. Every one of the 9^{51} assignments is generated.
- Backtracking validates *after each single assignment*. A cell typically has only 2–4 legal digits, and an illegal one is rejected before any of its 9^{50} descendants exists.

Key Insight

The branching factor is not 9 — it is “the number of digits currently legal in this cell”, which is usually small and shrinks as the board fills. Backtracking exploits a branching factor that *collapses under constraints*. That is the whole game.

13.2 LeetCode 37 — Sudoku Solver

Problem. Fill the empty cells (marked ‘.’) of a 9×9 board so that every row, every column and each of the nine 3×3 boxes contains the digits 1–9 exactly once. A solution is guaranteed unique; modify the board in place.

13.2.1 The box index formula

$$\text{box}(r, c) = \left\lfloor \frac{r}{3} \right\rfloor \times 3 + \left\lfloor \frac{c}{3} \right\rfloor$$

$\lfloor r/3 \rfloor$ picks the band (0,1,2), $\lfloor c/3 \rfloor$ picks the stack (0,1,2), and the $\times 3$ flattens the pair into 0..8 — exactly row-major numbering of the boxes.

0	0	0	1	1	1	2	2	2
0	0	0	1	1	1	2	2	2
0	0	0	1	1	1	2	2	2
3	3	3	4	4	4	5	5	5
3	3	3	4	4	4	5	5	5
3	3	3	4	4	4	5	5	5
6	6	6	7	7	7	8	8	8
6	6	6	7	7	7	8	8	8
6	6	6	7	7	7	8	8	8

13.2.2 C–G. Ingredients

State	the board itself, plus three arrays of sets: <code>rows[9]</code> , <code>cols[9]</code> , <code>boxes[9]</code>
Base case	no empty cell remains $\Rightarrow$ return <code>True</code> (propagate success up)
Choices	digits '1'..'9' for the chosen empty cell
Constraint	digit not already in <code>rows[r]</code> , <code>cols[c]</code> , <code>boxes[b]</code>
Pruning	the constraint; plus MRV cell ordering (below)
Undo	clear the cell and remove the digit from all three sets
Return type	<code>bool</code> — because we need <i>one</i> solution and must stop as soon as we find it

Key Insight

Why this function returns a boolean. Enumeration problems return `void` and collect into `res`. Decision problems (“is there a solution?”) return `bool` so that a successful deep call can unwind the whole stack immediately without undoing the assignments that produced the answer. Getting this right is the structural difference between LC 37 and LC 51.

13.2.3 L. Python

```

1  class Solution:
2      def solveSudoku(self, board):
3          rows = [set() for _ in range(9)]
4          cols = [set() for _ in range(9)]
5          boxes = [set() for _ in range(9)]
6          empties = []
7
8          for r in range(9):
9              for c in range(9):
10                 ch = board[r][c]
11                 if ch == ".":
12                     empties.append((r, c))
13                 else:
14                     rows[r].add(ch)
15                     cols[c].add(ch)
16                     boxes[(r // 3) * 3 + c // 3].add(ch)
17
18         def backtrack(k):
19             if k == len(empties):          # ACCEPT: no cell left
20                 return True
21             r, c = empties[k]
22             b = (r // 3) * 3 + c // 3
23             for d in "123456789":
24                 if d in rows[r] or d in cols[c] or d in boxes[b]:
25                     continue              # CONSTRAINT  $\rightarrow$  prune
26                 board[r][c] = d          # CHOOSE
27                 rows[r].add(d); cols[c].add(d); boxes[b].add(d)
28
29                 if backtrack(k + 1):      # EXPLORE
30                     return True          # success: do NOT undo
31
32                 board[r][c] = "."        # UNDO
33                 rows[r].remove(d); cols[c].remove(d); boxes[b].remove(d)
34             return False                  # no digit worked here
35
36         backtrack(0)

```

Listing 13.1: LeetCode 37 — Sudoku Solver

13.2.4 M. Java

```

1  import java.util.*;
2
3  class Solution {
4      private boolean[][] rows = new boolean[9][10];
5      private boolean[][] cols = new boolean[9][10];
6      private boolean[][] boxes = new boolean[9][10];
7      private List<int[]> empties = new ArrayList<>();
8
9      public void solveSudoku(char[][] board) {
10         for (int r = 0; r < 9; r++)
11             for (int c = 0; c < 9; c++) {
12                 if (board[r][c] == '.') { empties.add(new int[]{r, c}); }
13                 else {
14                     int d = board[r][c] - '0';
15                     rows[r][d] = cols[c][d] = boxes[(r / 3) * 3 + c / 3][d] = true;
16                 }
17             }
18         backtrack(board, 0);
19     }
20
21     private boolean backtrack(char[][] board, int k) {
22         if (k == empties.size()) return true; // ACCEPT
23         int r = empties.get(k)[0], c = empties.get(k)[1];
24         int b = (r / 3) * 3 + c / 3;
25
26         for (int d = 1; d ≤ 9; d++) {
27             if (rows[r][d] || cols[c][d] || boxes[b][d]) continue; // prune
28             board[r][c] = (char) ('0' + d); // CHOOSE
29             rows[r][d] = cols[c][d] = boxes[b][d] = true;
30
31             if (backtrack(board, k + 1)) return true; // EXPLORE
32
33             board[r][c] = '.'; // UNDO
34             rows[r][d] = cols[c][d] = boxes[b][d] = false;
35         }
36         return false;
37     }
38 }

```

Listing 13.2: LeetCode 37 — Sudoku Solver, Java

Common Mistake

Undoing after a successful recursive call. If you write `backtrack(k+1); board[r][c]='.'`; without the `if`, you erase the solution you just found. The rule: on success, return immediately *without* undoing; undo only on failure.

13.2.5 N. Dry run (conceptual)

Dry Run

Suppose the first empty cell is (0, 2) and its row already has {5, 3}, its column {8}, its box {5, 3, 6}.

1. Try 1: legal. Place, mark, recurse to cell 2.
2. Deep inside, cell 17 finds *no* legal digit ⇒ returns False.
3. Cell 16 undoes its digit, tries the next; eventually it too returns False.
4. The failure propagates back to (0, 2), which undoes 1 and tries 2, then 4, ...
5. Eventually a chain of assignments reaches `k == len(empties)` and True cascades up the whole

stack, each frame returning immediately and leaving its assignment in place. The board at the end holds exactly the assignments on the successful root-to-leaf path.

13.3 Optimisation techniques

13.3.1 1. Sets or boolean arrays instead of scanning

The naive validity check scans 9 row cells, 9 column cells and 9 box cells: 27 comparisons per candidate. With the three lookup structures it is 3 constant-time tests. This alone is typically a 5–10× speedup.

13.3.2 2. Bitmasks

Replace each set by a 9-bit integer. Then

```
1 candidates = ~(rows[r] | cols[c] | boxes[b]) & 0x1FF  # bits that are legal
2 while candidates:
3     bit = candidates & -candidates  # lowest legal digit
4     candidates -= bit
5     ...
```

Listing 13.3: Candidate generation in one expression

You never test illegal digits at all — the loop iterates exactly `popcount` times instead of 9. Undo becomes `rows[r] ^= bit` (XOR toggles the bit back).

13.3.3 3. MRV — Minimum Remaining Values

Instead of filling empty cells in scan order, always fill the cell with the *fewest* legal candidates.

Intuition

If some cell has only one candidate, filling it is free — branching factor 1, no risk. If some cell has zero candidates, the board is already dead and you discover it now instead of after a hundred fruitless assignments. Choosing the most constrained variable first is the single most effective CSP heuristic, and it is why human solvers scan for “only one possibility” squares.

```
1 def pick_cell(board, rows, cols, boxes):
2     best, best_count = None, 10
3     for r in range(9):
4         for c in range(9):
5             if board[r][c] != ".":
6                 continue
7             b = (r // 3) * 3 + c // 3
8             used = rows[r] | cols[c] | boxes[b]
9             n = 9 - len(used)
10            if n < best_count:
11                best, best_count = (r, c), n
12                if n ≤ 1:  # cannot do better; stop scanning
13                    return best, best_count
14    return best, best_count
```

Listing 13.4: MRV cell selection

If `best_count == 0` for any empty cell, return `False` immediately — a powerful early failure detection.

13.3.4 4. Constraint propagation

Two classic rules, applied repeatedly until nothing changes, before each branching decision:

- **Naked single:** a cell with exactly one candidate must take it.
- **Hidden single:** if a digit can go in only one cell of a row/column/box, it goes there.

Propagation is deterministic — no branching, no undo needed beyond recording what was filled so it can be rolled back if the branch fails. On typical puzzles, propagation alone solves most cells and reduces the search to a handful of nodes. This is the classic *Norvig* solver design.

Key Insight

The hierarchy of Sudoku solvers. (1) generate-and-test: hopeless. (2) backtracking with per-assignment checks: solves easy puzzles. (3) + sets/bitmasks: fast. (4) + MRV: handles adversarial puzzles. (5) + constraint propagation: near-instant on everything. Each level reduces *nodes*, except (3) which reduces *cost per node*. Being able to name which one you are doing is what separates a memorised solution from understanding.

13.3.5 O. Complexity

- Worst case $O(9^m)$ where m is the number of empty cells — an honest but very loose bound, since constraints make most branches die immediately.
- For the fixed 9×9 board the input size is constant, so strictly speaking the solver is $O(1)$; the interesting statement is about generalised $n^2 \times n^2$ Sudoku, which is NP-complete.
- **Space:** $O(m)$ recursion depth, $O(1)$ for the constraint structures.

13.3.6 P. Common mistakes

Common Mistake

Returning void. Then a deep success cannot stop the search, and later frames keep exploring and undoing until the board is wiped. Sudoku needs `bool`.

Common Mistake

Recomputing the empty-cell list inside the recursion (or rescanning the board from $(0, 0)$ every call). It turns each node into $O(81)$ work. Either precompute the list or pass an index and advance it.

Common Mistake

Forgetting to seed the sets from the pre-filled cells. The solver then happily duplicates given digits. Always initialise before recursing.

Common Mistake

Box index $r/3 + c/3$ instead of $(r/3)*3 + c/3$. The wrong formula maps several distinct boxes to the same index (e.g. box $(0,1)$ and $(1,0)$ both give 1), so constraints leak between boxes and you get invalid solutions.

13.3.7 Q/R. Practice

LeetCode 36 — Valid Sudoku (the constraint check alone — do this first), **LeetCode 37 — Sudoku Solver**, then implement the MRV version and compare node counts on a “hardest Sudoku” puzzle. The difference is dramatic and worth seeing with your own counter.

Part IV

Mastery

Chapter 14

Pruning

14.1 What is pruning?

Pruning is the act of *not creating* a subtree whose leaves you can prove are all useless. Formally: at node v you evaluate a predicate $P(v)$ such that

$$P(v) \text{ is true} \implies \text{no descendant leaf of } v \text{ is a solution.}$$

If $P(v)$ holds you return immediately. Correctness requires the implication to be *sound* (never prune a subtree that contains a solution); efficiency wants it to fire often and cost little.

14.2 Why it matters: the arithmetic

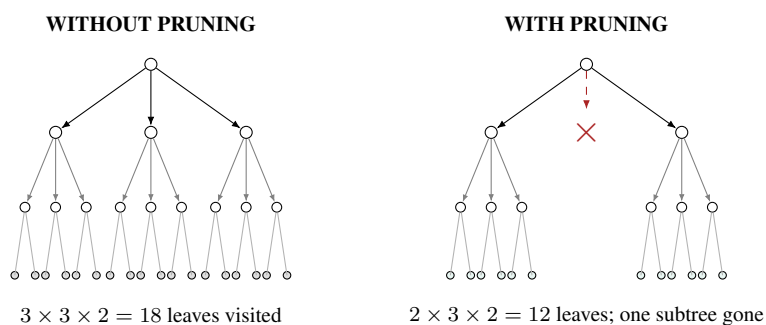
A node at depth k in a tree of branching factor b and depth d has b^{d-k} descendant leaves. Pruning it saves all of them.

Prune at depth	Leaves removed ($b=3, d=12$)	% of tree
1	$3^{11} = 177,147$	33.3%
4	$3^8 = 6,561$	1.23%
8	$3^4 = 81$	0.015%
11	$3^1 = 3$	0.0006%

Key Insight

Prune early or don't bother. A test that fires at depth 1 is worth b^7 times more than the same test at depth 8. When designing, always ask: “can I detect this failure sooner?”

14.3 Visualising it



14.4 Correctness vs optimisation vs pruning

These three words are often used loosely. Keep them apart:

Term	What it does	If you remove it
Constraint	Rejects choices that would make the candidate <i>invalid</i>	Wrong answers (invalid or duplicate output)
Pruning	Rejects choices that are valid <i>now</i> but provably cannot complete	Same answers, slower
Optimisation	Reduces the <i>cost per node</i> (bitmasks, precomputed tables, $O(1)$ checks)	Same answers, same tree, slower per node

Common Mistake

Calling everything “pruning”. In N-Queens, “this column is taken” is a *constraint* — drop it and you output attacking boards. In Combination Sum, $c > t$ is *pruning* — drop it and the output is still correct. Being precise here signals real understanding.

14.5 The four kinds of prune

1. **Feasibility prune** — the partial candidate already violates a monotone invariant. *Example:* `target < 0` in Combination Sum; `close > open` in Generate Parentheses.
2. **Capacity prune** — not enough material remains to finish. *Example:* `i <= n - need + 1` in Combinations; `len(word) > rows*cols` in Word Search.
3. **Ordering prune** — with a sorted candidate list, once one candidate fails a monotone test, all later ones do too, so `continue` upgrades to `break`. *Example:* `if c > t: break` in Combination Sum.
4. **Symmetry / duplicate prune** — two branches produce identical answer sets, so keep the canonical one. *Example:* `if i > start and nums[i] == nums[i-1]: continue`; in N-Queens, restricting row 0 to the left half and mirroring.

14.6 Worked examples

14.6.1 Combination Sum (LC 39)

```
if c > t: break           # ordering prune (needs sorted input)
if t == 0: record         # accept
```

Soundness: all candidates are positive, so t never increases; once $t < 0$ it can never return to 0. *Effect:* for $[2, 3, 6, 7]$, target 7, the tree drops from 33 nodes to 13.

14.6.2 Generate Parentheses (LC 22)

```
if open < n: ...          # capacity prune
if close < open: ...      # feasibility prune
```

Effect: at $n = 8$ the unpruned space is $2^{16} = 65,536$ candidates; the pruned search visits only the $C_8 = 1,430$ valid strings and their prefixes.

14.6.3 N-Queens (LC 51)

The three set lookups are constraints; the row-per-queen model is a *modelling* prune that removes $\binom{n^2}{n}/n^n$ of the space before the search starts. A further *symmetry* prune: for counting solutions, only try columns $0..[n/2] - 1$ in row 0 and double the count (adjusting for odd n).

14.6.4 Sudoku (LC 37)

Constraint: the three membership tests. Prune: MRV cell ordering, which is a *reordering* rather than a rejection — it makes failures surface at shallower depth. Zero-candidate detection is a pure feasibility prune.

14.6.5 Word Search (LC 79)

```
if board[r][c] != word[k]: return False # feasibility, fires at depth k
```

Plus two cheap global prunes computed before the search:

```
if len(word) > rows * cols: return False
if Counter(word) - Counter(ch for row in board for ch in row): return False
```

And a neat one: if the word's last letter is rarer on the board than its first, search for `word[::-1]` instead, cutting the number of viable starting cells.

14.7 How to invent a prune

Ask, in this order:

1. **Is some quantity monotone along a path?** (remaining target decreases, length increases, unmatched brackets never go negative.) A monotone quantity crossing a threshold is an instant prune.
2. **Is there a resource budget?** (elements left, cells left, characters left.) Compare “needed” against “available”.
3. **Is the candidate list sorted?** If yes, look for a `continue` you can upgrade to `break`.
4. **Do two branches produce the same subtree?** Sort, then keep the canonical representative.
5. **Can I reorder the choices so failures appear earlier?** (MRV, most-constrained variable first, least-constraining value first.)

Interview Tip

After you get a working brute-force backtracking solution, say: “Now let me add pruning.” Then walk through the five questions above out loud. Interviewers are explicitly scoring whether you optimise unprompted.

14.8 Over-pruning and under-pruning

Common Mistake

Over-pruning — the predicate is not sound and removes real solutions. *Example:* in Combination Sum with *unsorted* candidates, using `break` instead of `continue`. *Symptom:* too few answers, and typically only on some test cases — the worst kind of bug. *Defence:* state the invariant that justifies the prune before you write it.

Common Mistake

Under-pruning — correct but slow; the classic “validate only at the leaf”. *Symptom:* TLE. *Defence:* for every constraint, ask whether it can be tested on a prefix.

Common Mistake

Expensive pruning — a prune costing $O(n^2)$ that eliminates 5% of nodes is a net loss. Measure: a prune is worth it if $(\text{cost per node}) \times (\text{nodes}) < (\text{nodes saved}) \times (\text{cost per saved node})$.

14.9 Pruning checklist

Remember This

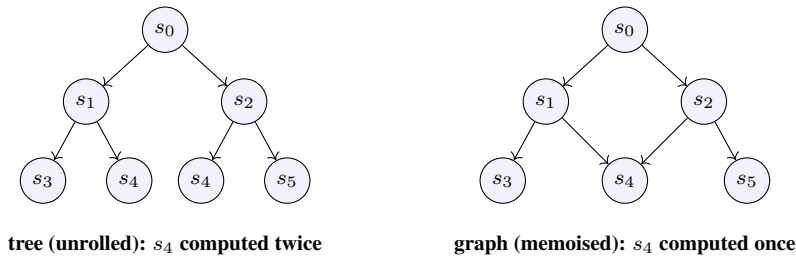
1. Is any quantity monotone? $\Rightarrow$ threshold prune.
2. Is there a remaining-capacity bound? $\Rightarrow$ loop-bound prune.
3. Is the input sorted? $\Rightarrow$ break instead of continue.
4. Are there duplicate values? $\Rightarrow$ same-level skip.
5. Are there symmetric solutions? $\Rightarrow$ canonical half only.
6. Can I order variables by fewest options? $\Rightarrow$ MRV.
7. Can I detect “zero options left” before descending? $\Rightarrow$ early failure.
8. Is my prune *sound*? State the invariant that proves it.

Chapter 15

Backtracking + Memoisation

15.1 State-space tree vs state graph

Everything so far assumed a *tree*: distinct decision paths lead to distinct nodes. But sometimes different paths arrive at *the same state*. Then the true search structure is a directed acyclic *graph*, and the tree we explore is that graph “unrolled” — with subtrees duplicated once per incoming path.



Key Insight

The memoisation test. Ask: *if two different prefixes lead to the same state, is the remaining work identical?* If yes, cache it. If the answer depends on the prefix — because the output must be prefixed with it — caching the *final answers* is wrong, though caching an aggregate over the suffix is still fine.

15.2 When memoisation applies (and when it does not)

Question	Memoisable?	Why
“List all subsets”	No	Every leaf is a distinct output; nothing repeats
“Count subsets summing to T ”	Yes, on (i, t)	The count of completions depends only on (i, t)
“Is there a subset summing to T ?”	Yes, on (i, t)	Boolean depends only on the state
“All partitions of s into palindromes”	Partly	The <i>set of suffix partitions</i> depends only on <i>start</i> , but each reuse needs a copy
“Minimum palindrome cuts”	Yes, on <i>start</i>	Aggregate over the suffix
“All permutations”	No	Every leaf distinct; $n!$ output
“Can I partition into k equal-sum subsets”	Yes, on the used-mask	Different fill orders reach the same mask

Remember This

Rule of thumb. If the problem asks you to *enumerate objects*, the output is the bottleneck and memoisation buys little. If it asks for a *number*, a *boolean*, or an *optimum*, look hard for a memoisable state — you are probably looking at DP wearing a backtracking costume.

15.3 Example 1 — Subset sum: from backtracking to DP

15.3.1 The backtracking version

```

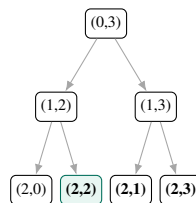
1 def count_subsets(nums, target):
2     n = len(nums)
3
4     def backtrack(i, t):
5         if t == 0:
6             return 1
7         if i == n or t < 0:
8             return 0
9         return backtrack(i + 1, t - nums[i]) + backtrack(i + 1, t)
10
11    return backtrack(0, target)

```

Listing 15.1: Count subsets with a given sum — pure search, $O(2^n)$

15.3.2 Spotting the repetition

With `nums = [1, 2, 1, 2]` and target 3, consider the paths *take 1, skip 2* and *skip 1, ...*. Several distinct prefixes reach the state ($i=2, t=2$). From that state onward, the number of completions is the same regardless of how we got there — because the future depends only on which elements remain and how much target is left.



15.3.3 The memoised version

```

1 from functools import lru_cache
2
3 def count_subsets(nums, target):
4     n = len(nums)
5
6     @lru_cache(maxsize=None)
7     def backtrack(i, t):
8         if t == 0:
9             return 1
10        if i == n or t < 0:
11            return 0
12        return backtrack(i + 1, t - nums[i]) + backtrack(i + 1, t)
13
14    return backtrack(0, target)

```

Listing 15.2: Same code, one decorator — $O(n \cdot T)$

```

1  import java.util.*;
2
3  class SubsetCount {
4      private int[] nums;
5      private Map<Long, Integer> memo = new HashMap<>();
6
7      public int count(int[] nums, int target) {
8          this.nums = nums;
9          return go(0, target);
10     }
11
12     private int go(int i, int t) {
13         if (t == 0) return 1;
14         if (i == nums.length || t < 0) return 0;
15         long key = ((long) i << 20) | (t & 0xFFFF); // pack (i, t)
16         Integer cached = memo.get(key);
17         if (cached != null) return cached;
18         int result = go(i + 1, t - nums[i]) + go(i + 1, t);
19         memo.put(key, result);
20         return result;
21     }
22 }

```

Listing 15.3: Java: explicit memo table

The tree had 2^n nodes; the graph has at most $n \times (T + 1)$ distinct states. For $n = 30$, $T = 100$ that is 10^9 versus 3,100.

15.4 Example 2 — Word Break (LC 139): the classic

```

1  def word_break(s, word_dict):
2      words = set(word_dict)
3      n = len(s)
4
5      from functools import lru_cache
6
7      @lru_cache(maxsize=None)
8      def can(start): # can s[start:] be segmented?
9          if start == n:
10             return True
11             for end in range(start + 1, n + 1):
12                 if s[start:end] in words and can(end):
13                     return True
14             return False
15
16     return can(0)

```

Listing 15.4: Word Break: backtracking that MUST be memoised

Without the cache, input like $s = \text{"aaaa...ab"}$ with dictionary $[\text{"a"}, \text{"aa"}, \text{"aaa"}, \text{"aaaa"}]$ takes exponential time. With it, $O(n^2)$ states $\times O(n)$ substring work $= O(n^3)$.

Key Insight

The state is *start* alone — *not* (*start*, *path*). The moment you notice that the answer for the suffix is independent of the prefix, drop the path from the state and memoise. Conversely, LC 140 (*Word Break II*, return all sentences) needs the path in the output, so you memoise the *list of suffix sentences* and prepend the current word to each — a copy per reuse, but still an enormous saving.

```

1 def word_break_ii(s, word_dict):
2     words = set(word_dict)
3     from functools import lru_cache
4
5     @lru_cache(maxsize=None)
6     def solve(start):
7         # all sentences for s[start:]
8         if start == len(s):
9             return [""]
10            # one empty sentence
11
12        out = []
13        for end in range(start + 1, len(s) + 1):
14            w = s[start:end]
15            if w in words:
16                for rest in solve(end):
17                    out.append(w if rest == "" else w + " " + rest)
18
19        return out
20
21    return solve(0)

```

Listing 15.5: Word Break II: memoising the answer lists

15.5 Example 3 — Partition to K Equal Sum Subsets (LC 698)

Here the memoisable state is a *bitmask* of used elements: many different fill orders reach the same set of used elements, and from there the remaining work is identical.

```

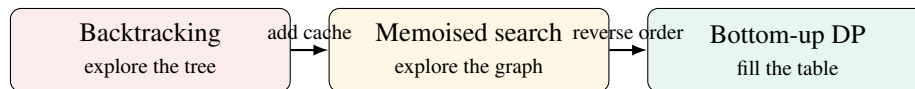
1 def can_partition_k(nums, k):
2     total = sum(nums)
3     if total % k:
4         return False
5     target = total // k
6     nums.sort(reverse=True)
7     # big elements first: fail fast
8     if nums[0] > target:
9         return False
10    n = len(nums)
11    seen = {}
12
13    def dfs(mask, current):
14        if mask == (1 << n) - 1:
15            return True
16        if (mask, current) in seen:
17            # same used-set, same partial bucket
18            return seen[(mask, current)]
19
20        ok = False
21        for i in range(n):
22            if mask & (1 << i):
23                continue
24            if current + nums[i] > target:
25                continue
26            # prune
27            nxt = current + nums[i]
28            if dfs(mask | (1 << i), 0 if nxt == target else nxt):
29                ok = True
30                break
31        seen[(mask, current)] = ok
32        return ok
33
34    return dfs(0, 0)

```

Listing 15.6: Bitmask memoisation over used elements

Note that `current` is derivable from `mask` and `target`, so keying on `mask` alone is also possible — a nice micro-exercise.

15.6 Backtracking, memoisation and DP: one continuum



Interview Tip

A very strong interview move: solve it with backtracking first, then say “notice that (i, t) fully determines the remaining work, so I can memoise, giving $O(nT)$ — and if you’d like, I can convert that to a bottom-up table with $O(T)$ space.” You have then shown search, DP, and space optimisation in one breath.

15.7 Common mistakes

Common Mistake

Memoising on an incomplete state. If you cache on i but the answer also depends on t , you return wrong values — and the bug is invisible on small tests. Write down every variable the return value depends on, and make the key exactly that tuple.

Common Mistake

Caching mutable objects and then mutating them. If `solve(start)` returns a list that the caller appends to, the cached entry is corrupted for the next reader. Return copies or treat cached values as immutable.

Common Mistake

Using `lru_cache` on a method that closes over mutable state (like a `path` list). The cache key does not include that state, so results leak between contexts. Only memoise pure functions of their arguments.

Common Mistake

Memoising an enumeration. Caching “all subsets from index i ” saves nothing asymptotically, because the output size dominates. Recognise when the exponential is irreducible.

Chapter 16

Complexity Analysis

16.1 The master formula

$$\text{Time} = \underbrace{N}_{\text{nodes explored}} \times \underbrace{w}_{\text{work per node}} + \underbrace{S \times c}_{\text{solutions} \times \text{copy cost}}$$

and, with branching factor b and depth d ,

$$N \leq \sum_{k=0}^d b^k = \frac{b^{d+1} - 1}{b - 1} = \Theta(b^d).$$

Key Insight

$O(b^d)$ is an *upper bound on a uniform tree*. Real backtracking trees are rarely uniform: b shrinks (permutations), branches die early (N-Queens), or the accepted leaves are a tiny fraction (Generate Parentheses). Always state the bound, then say how pruning changes it.

16.2 The six quantities to identify

Branching factor b	How many iterations does the choice loop make? Constant (2 for subsets), shrinking ($n - k$ for permutations), or data-dependent (legal digits in Sudoku)?
Depth d	How many decisions before a leaf? $n, k, 2n, T/\text{min}, L \dots$
Number of states N	The node count. Usually $\Theta(\text{leaves})$ when every internal node has ≥ 2 children or leads to a leaf.
Cost per state w	The loop itself plus constraint checks. $O(1)$ with sets, $O(n)$ with linear scans.
Copying cost c	Snapshotting the path at each solution: $O(\text{path length})$.
Number of solutions S	Often the true lower bound on the whole algorithm.

Remember This

Output-sensitivity. You can never beat $\Omega(S \cdot c)$ if you must print all solutions. For Subsets that is $\Omega(n2^n)$ — so $O(n2^n)$ is *optimal*, not merely “exponential and bad”. Saying this is a mark of maturity.

16.3 Worked analyses

16.3.1 Subsets (LC 78)

$b = 2, d = n$, so $N = 2^{n+1} - 1$. Work per node $O(1)$. Solutions $S = 2^n$, average path length $n/2$, so copying totals $\sum_k \binom{n}{k} k = n2^{n-1}$.

$$\text{Time } \Theta(n \cdot 2^n), \quad \text{Aux space } O(n).$$

16.3.2 Combinations (LC 77)

Leaves = $\binom{n}{k}$; with the capacity prune every internal node has a descendant leaf, so $N = O(\binom{n}{k})$. Copy cost $O(k)$.

$$\text{Time } O\left(k \binom{n}{k}\right), \quad \text{Aux space } O(k).$$

Without the prune, add a factor up to $O(n - k)$ from dead branches.

16.3.3 Permutations (LC 46)

Level k has $n(n - 1) \cdots (n - k + 1)$ nodes; summing, $N = \sum_{k=0}^n \frac{n!}{(n-k)!} \leq \lfloor e \cdot n! \rfloor = \Theta(n!)$. Per node the loop scans all n indices, so $w = O(n)$; each leaf copies $O(n)$.

$$\text{Time } O(n \cdot n!), \quad \text{Aux space } O(n).$$

16.3.4 Combination Sum (LC 39)

Depth is bounded by T/m where $m = \min(\text{candidates})$; branching $\leq n$. Loose bound

$$\text{Time } O\left(n^{T/m} \cdot \frac{T}{m}\right), \quad \text{Aux space } O(T/m).$$

The honest statement: the exact count is the number of restricted multisets summing to T , which has no simple closed form. Pruning makes the practical count far smaller.

16.3.5 Generate Parentheses (LC 22)

Pruning admits only valid prefixes, so leaves = C_n (Catalan) and every internal node is a prefix of some valid string. Since $C_n = \frac{1}{n+1} \binom{2n}{n} \sim \frac{4^n}{n^{3/2}\sqrt{\pi}}$ and each leaf costs $O(n)$:

$$\text{Time } O\left(\frac{4^n}{\sqrt{n}}\right), \quad \text{Aux space } O(n).$$

16.3.6 Palindrome Partitioning (LC 131)

At most 2^{n-1} partitions. Naive palindrome checks make each node $O(n)$; with the precomputed table, $O(1)$ per check and $O(n)$ per leaf copy.

$$\text{Time } O(n \cdot 2^n), \quad \text{Aux space } O(n^2) \text{ for the table.}$$

16.3.7 Word Search (LC 79)

From each of mn starts, the first move has 4 options and each subsequent move has at most 3 (the cell we came from is marked).

$$\text{Time } O(mn 3^L), \quad \text{Aux space } O(L).$$

16.3.8 N-Queens (LC 51)

The column constraint alone bounds leaves by $n!$; diagonals cut much more, but no closed form exists. Rendering a board costs $O(n^2)$.

$$\text{Time } O(n!) \text{ nodes, } O(n! \cdot n^2) \text{ with rendering,} \quad \text{Aux space } O(n).$$

The actual number of solutions grows roughly like $n!/c^n$ with $c \approx 2.54$ — empirical, not proven.

16.3.9 Sudoku (LC 37)

With m empty cells, $O(9^m)$ worst case. For the fixed 9×9 board this is a constant, so the meaningful statement concerns generalised Sudoku, which is NP-complete. Aux space $O(m)$.

16.4 Summary table

Problem	b	d	Time	Aux space
Binary strings	2	n	$O(n2^n)$	$O(n)$
Subsets (78)	2	n	$O(n2^n)$	$O(n)$
Subsets II (90)	$\leq n$	$\leq n$	$O(n2^n)$	$O(n)$
Combinations (77)	$\leq n$	k	$O(k \binom{n}{k})$	$O(k)$
Combination Sum (39)	$\leq n$	T/m	$O(n^{T/m} \cdot T/m)$	$O(T/m)$
Combination Sum II (40)	$\leq n$	$\leq n$	$O(n \cdot 2^n)$	$O(n)$
Permutations (46)	$n \rightarrow 1$	n	$O(n \cdot n!)$	$O(n)$
Permutations II (47)	$\leq n$	n	$O(n \cdot n!)$	$O(n)$
Letter Combos (17)	3–4	$ d $	$O(d \cdot 4^{ d })$	$O(d)$
Gen Parentheses (22)	2	$2n$	$O(4^n / \sqrt{n})$	$O(n)$
Palindrome Part. (131)	$\leq n$	$\leq n$	$O(n2^n)$	$O(n^2)$ table
Word Search (79)	4 then 3	L	$O(mn3^L)$	$O(L)$
N-Queens (51)	$\leq n$	n	$O(n!)$	$O(n)$
Sudoku (37)	≤ 9	m	$O(9^m)$	$O(m)$

16.5 Why exact complexity is often hard

1. **Pruning is data-dependent.** How many N-Queens branches die depends on arithmetic properties of n ; no formula is known for the number of solutions.
2. **The branching factor varies.** In Sudoku it is “number of legal digits”, which changes cell by cell and puzzle by puzzle.
3. **Solution counts are combinatorially deep.** Catalan numbers, derangements, integer partitions — each needs its own theory.
4. **Worst case $\neq$ typical case.** Sudoku is 9^m in theory and microseconds in practice.

Interview Tip

When you cannot give an exact bound, give the *structure* of the bound and be explicit about the looseness: “The upper bound is $O(n!)$ from the column constraint; the diagonal constraints prune far more, but there is no closed form for the exact node count.” That is a better answer than a confident wrong number.

16.6 Space complexity: three components

1. **Recursion stack:** $O(d)$ frames — almost always the dominant auxiliary term.
2. **Shared state:** `path` $O(d)$, `visited` $O(n)$, constraint sets $O(n)$, palindrome table $O(n^2)$.
3. **Output:** usually excluded by convention, but say so explicitly: “ $O(n)$ auxiliary, plus $O(n2^n)$ for the output.”

Common Mistake

Claiming $O(1)$ space for a recursive solution. The call stack is real memory; depth d means $O(d)$ space. Interviewers check for this.

16.7 Complexity checklist

Remember This

1. What is b ? Is it constant, shrinking, or data-dependent?
2. What is d ?
3. How many nodes — and does pruning make node count $\approx$ leaf count?
4. What is the per-node work (loop + constraint checks)?
5. What does each recorded solution cost to copy?
6. How many solutions are there? Is my algorithm output-optimal?
7. Auxiliary space: stack depth + shared state. State it separately from output.

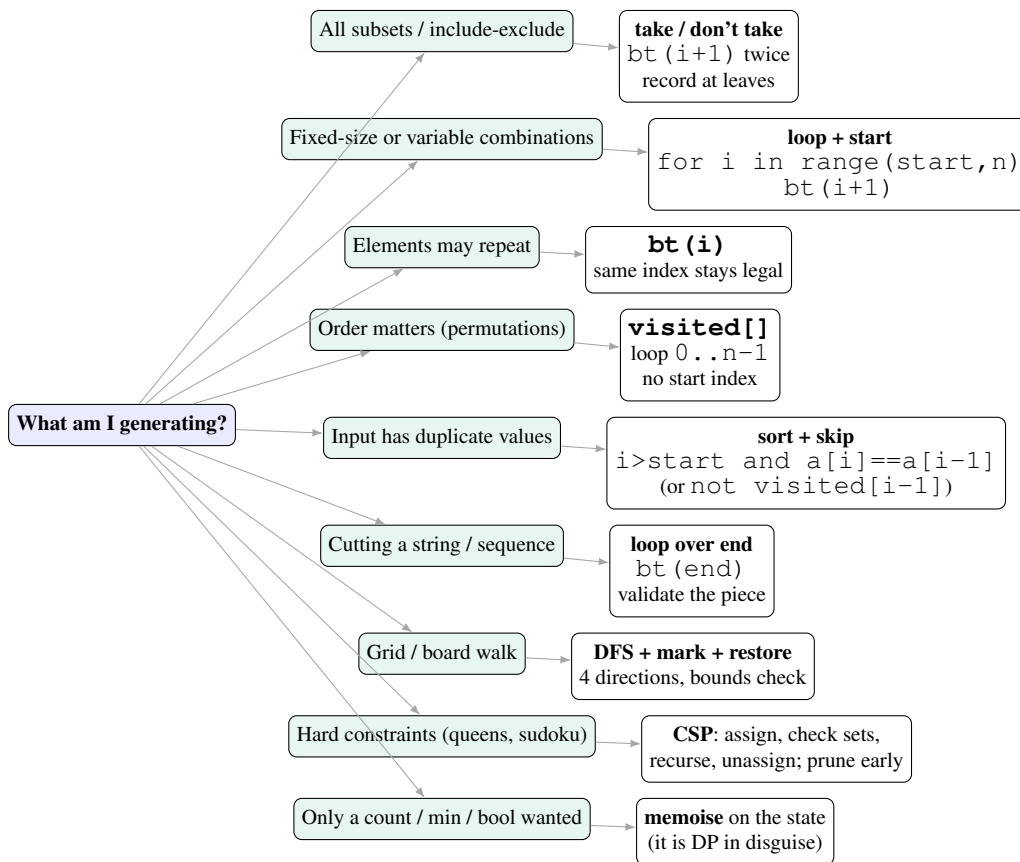
Chapter 17

Pattern Cheat Sheet

17.1 The master table

Problem	State	Choices	Recursion	Undo	Complexity
Subsets (78)	(i, path) or (start, path)	take / skip; or all $i \geq \text{start}$	bt (i+1)	pop	$O(n2^n)$
Combinations (77)	(start, path)	numbers start..n	bt (i+1)	pop	$O(k \binom{n}{k})$
Combination Sum (39)	(start, t, path)	candidates $\geq \text{start}$	bt (i) (reuse)	pop	$O(n^{T/m})$
Combination Sum II (40)	(start, t, path)	candidates, dedup siblings	bt (i+1)	pop	$O(n2^n)$
Permutations (46)	(path, visited)	all unused indices	bt ()	pop visited=F	+ $O(n \cdot n!)$
Permutations II (47)	(path, visited)	unused, skip non-canonical dups	bt ()	pop visited=F	+ $O(n \cdot n!)$
Subsets II (90)	(start, path)	$i \geq \text{start}$, skip dup siblings	bt (i+1)	pop	$O(n2^n)$
Letter Combos (17)	(index, path)	letters of digits[index]	bt (index+1)	pop	$O(d 4^{ d })$
Gen Parentheses (22)	(open, close, path)	' (' if open < n; if close < open	bt (o+1, c) / bt (o, c+1)	pop	$O(4^n / \sqrt{n})$
Palindrome Part. (131)	(start, path)	every prefix s[start:end]	bt (end)	pop	$O(n2^n)$
Word Search (79)	(r, c, k) board	+ 4 directions	bt (nr, nc, k+1)	restore board[r][c]	$O(mn3^L)$
N-Queens (51)	(row, cols, d1, d2)	columns 0..n-1	bt (row+1)	remove from 3 sets	$O(n!)$
Sudoku (37)	board rows/cols/boxes	+ digits 1-9 legal here	bt (k+1), turns bool	re-clear cell + 3 sets	$O(9^m)$

17.2 Which template should I use?



17.3 The three recursion calls, side by side

Call	Meaning	Used in
<code>bt (i)</code>	element i may be chosen again	Combination Sum (39), coin change enumerations
<code>bt (i + 1)</code>	element i is consumed; move on	Subsets, Combinations, Combination Sum II, Palindrome Partitioning (as <code>bt (end)</code>)
<code>bt ()</code> with <code>visited</code>	any unused element, in any order	Permutations (46, 47), N-Queens (by row)

17.4 Combination vs permutation, at a glance

COMBINATION — order irrelevant
 $[1, 2] == [2, 1] \Rightarrow$ one answer
 device: start index, strictly forward
 loop: `for i in range(start, n)`
 count: $\binom{n}{k}$ or 2^n

PERMUTATION — order matters
 $[1, 2] != [2, 1] \Rightarrow$ two answers
 device: `visited[]` array
 loop: `for i in range(0, n)`
 count: $n!$ or $n!/(n-k)!$

17.5 Recognition signals

Phrase in the problem	Reach for
“return all possible ...”	backtracking enumeration
“how many ways ...” with big n	DP, not backtracking
“all combinations”	loop + start
“all permutations”, “all arrangements”	visited[]
“may reuse an element unlimited times”	bt(i)
“each element used at most once”	bt(i+1)
“the array may contain duplicates”	sort + same-level skip
“partition the string/array”	loop over piece length, bt(end)
“in the grid, adjacent cells”	DFS + mark/restore
“no two ... may attack/conflict”	CSP + constraint sets
“find a valid configuration”	return bool, stop on success
“minimum / maximum / count”	memoise the state

Chapter 18

Python vs Java

18.1 The translation table

Role	Python	Java
Path container	<code>path = []</code>	<code>List<Integer> path = new ArrayList<>()</code>
Choose	<code>path.append(x)</code>	<code>path.add(x)</code>
Undo	<code>path.pop()</code>	<code>path.remove(path.size() - 1)</code>
Snapshot	<code>path[:] / list(path)</code>	<code>new ArrayList<>(path)</code>
Result list	<code>res = []</code>	<code>List<List<Integer>> res = new ArrayList<>()</code>
String buffer	<code>path = [] + "".join(path)</code>	<code>StringBuilder sb + sb.toString()</code>
String undo	<code>path.pop()</code>	<code>sb.deleteCharAt(sb.length()-1)</code>
Visited flags	<code>visited = [False]*n</code>	<code>boolean[] visited = new boolean[n]</code>
Constraint set	<code>cols = set()</code>	<code>Set<Integer> cols = new HashSet<>()</code>
Set add/remove	<code>s.add(x) / s.remove(x)</code>	<code>s.add(x) / s.remove(x)</code>
Sort	<code>nums.sort()</code>	<code>Arrays.sort(nums)</code>
Grid	<code>list[list[str]]</code>	<code>char[][]</code>
Memo	<code>@lru_cache</code>	<code>HashMap<Key, Value></code>

18.2 Side-by-side: the same solution twice

Python	Java
<pre>def subsets(nums): res, path = [], [] def bt(start): res.append(path[:]) for i in range(start, len(nums)): path.append(nums[i]) bt(i + 1) path.pop() bt(0) return res</pre>	<pre>List<List<Integer>> res = new ArrayList<>(); void bt(int start, int[] nums, List<Integer> path) { res.add(new ArrayList<>(path)); for (int i = start; i < nums. length; i++) { path.add(nums[i]); bt(i + 1, nums, path); path.remove(path.size() - 1); } }</pre>

Structurally identical. Every Java line has a Python twin. If your two versions diverge in structure, one of them is probably wrong.

18.3 Java pitfalls

Common Mistake

1. Storing the same `ArrayList` reference.

```
res.add(path); // WRONG: all entries alias one list
res.add(new ArrayList<>(path)); // RIGHT
```

Symptom: the result is a list of 2^n identical (usually empty) lists.

Common Mistake

2. `remove(int)` vs `remove(Object)`. For `List<Integer>`, `path.remove(3)` removes *index* 3. To remove the value 3 you must write `path.remove(Integer.valueOf(3))`. For the undo always use `path.remove(path.size() - 1)` — unambiguous and $O(1)$.

Common Mistake

3. `StringBuilder` mutation without restore.

```
sb.append(c);
backtrack(...);
// missing: sb.deleteCharAt(sb.length() - 1);
```

Symptom: outputs grow monotonically ("a", "ab", "abc" instead of "a", "b", "c"). Also note `sb.setLength(len)` is a clean alternative: record `int len = sb.length()` before the loop and restore with `sb.setLength(len)` — especially handy when a choice appends more than one character.

Common Mistake

4. Static/global result state. A static `List<...> res` persists between test cases in the same JVM, so the second call returns the first call's answers too. Use instance fields initialised in the public method, or pass `res` as a parameter.

Common Mistake

5. Autoboxing comparisons. `path.get(i) == path.get(j)` compares *references* for `Integer` outside the cache range $[-128, 127]$. Use `.equals()` or `unbox` to `int`.

Common Mistake

6. `int[]` vs `Integer[]` and `Arrays.asList`. `Arrays.asList(intArray)` yields a `List<int[]>` of size 1, not a list of integers. Build the list explicitly or use streams.

Common Mistake

7. Modifying a List while iterating it with a `for-each` loop throws `ConcurrentModificationException`. In backtracking you almost always iterate an *array* or an index range while mutating `path` — which is safe. Iterating `path` itself while mutating it is not.

18.4 Python pitfalls

Common Mistake

1. Appending the path reference.

```
res.append(path)      # WRONG
res.append(path[:])   # RIGHT
```

Same failure mode as Java's. This is the single most common backtracking bug in both languages.

Common Mistake

2. Forgetting `pop()`. Produces silently wrong (over-long) answers rather than a crash — much harder to spot than an exception.

Common Mistake

3. Shallow copies of nested structures. `path[:]` copies the outer list only. If `path` contains lists that you also mutate, use `copy.deepcopy`. For the usual list-of-ints case, the shallow copy is correct and cheap.

Common Mistake

4. Mutable default arguments.

```
def bt(i, path=[]):    # WRONG: the SAME list is reused across calls
```

The default is evaluated once at definition time, so state leaks between invocations of the outer function. Use `path=None` and create it inside, or close over an outer variable.

Common Mistake

5. Rebinding instead of mutating. Inside a nested function, `path = path + [x]` creates a *local* name and the outer `path` is unchanged — so your undo has nothing to undo and the recursion behaves unpredictably. Either mutate (`append/pop`) or pass the new list explicitly as a parameter. Never mix.

Common Mistake

6. Recursion depth. Python's default limit is 1000. Deep backtracking (e.g. a 100×100 grid walk) can exceed it. `sys.setrecursionlimit(10**6)` helps, but note the C stack is the real limit.

Common Mistake

7. `list.remove(x)` instead of `pop()`. `remove` deletes the *first* occurrence by value and is $O(n)$; with duplicate values it removes the wrong one. The undo must always be `pop()`.

18.5 Performance notes

	Python	Java
Typical constant factor	30–100× slower	baseline
Recursion overhead	high (function calls are expensive)	moderate; JIT inlines well
String concat in a loop	$O(n^2)$ — avoid	$O(n^2)$ with <code>String</code> — use <code>StringBuilder</code>
Set membership	<code>in</code> on a set is $O(1)$	<code>HashSet.contains</code> $O(1)$; <code>boolean[]</code> is faster still
Practical advice	prefer <code>list+join</code> , avoid slicing in hot loops	prefer primitive arrays over boxed collections in hot loops

Interview Tip

Write the version you are fluent in. Interviewers care about the recursion, the state, and the undo — not the language. But do know the two or three pitfalls of your language cold, because “why did you write `new ArrayList<>(path)?`” is a standard probe.

Chapter 19

Why My Backtracking Solution Doesn't Work

A debugging catalogue. Find your symptom in the table, then read the matching section.

Symptom	Likely cause (section)
RecursionError	/ 1, 5, 16
StackOverflowError	
All results are identical (often empty)	8
Results are too long / contain extra elements	3
Duplicate results	6, 9
Too few results	7, 10, 15
Results include invalid candidates	11
Correct but times out	11 (under-pruning)
Wrong answers only on some tests	4, 13, 14, 15
Second test case is polluted	12

19.1 Missing base case

```
def bt(i):  
    for x in choices:           # no stopping condition  
        path.append(x); bt(i + 1); path.pop()
```

Listing 19.1: Broken

Why it fails: nothing ever stops the descent; *i* grows forever.

```
def bt(i):  
    if i == n:  
        res.append(path[:]); return  
    for x in choices:  
        path.append(x); bt(i + 1); path.pop()
```

Listing 19.2: Fixed

19.2 Wrong base case

```
if open == n:                   # records "(" as a solution  
    res.append("".join(path)); return
```

Listing 19.3: Broken — Generate Parentheses

Why: the string is complete only when *both* counts reach *n*.

```
if len(path) == 2 * n:          # or: if open == n and close == n  
    res.append("".join(path)); return
```

Listing 19.4: Fixed

19.3 Forgetting the undo

```
for i in range(start, n):
    path.append(nums[i])
    bt(i + 1)
    # missing path.pop()
```

Listing 19.5: Broken

Why: `path` keeps growing, so every sibling branch inherits the previous branch's choices. Output is too long and wrong — and no exception is raised.

```
for i in range(start, n):
    path.append(nums[i])
    bt(i + 1)
    path.pop() # exact inverse, immediately after
```

Listing 19.6: Fixed

19.4 Wrong start index

```
for i in range(0, n): # ignores 'start'
    ...
    bt(i + 1)
```

Listing 19.7: Broken — always restarts from 0

Why: the ordering invariant collapses; you generate permutations of every subset and therefore massive duplication.

```
for i in range(start, n):
    ...
    bt(i + 1)
```

Listing 19.8: Fixed

19.5 Using `i` when you need `i + 1`

```
path.append(nums[i]); bt(i); path.pop() # element never consumed
```

Listing 19.9: Broken — Subsets

Why: `nums[i]` stays available forever $\Rightarrow [1, 1, 1, \dots]$ and infinite recursion in unbounded problems.

```
path.append(nums[i]); bt(i + 1); path.pop()
```

Listing 19.10: Fixed

19.6 Using `i + 1` when you need `i`

```
backtrack(i + 1, t - c) # forbids reuse
```

Listing 19.11: Broken — Combination Sum (LC 39)

Why: LC 39 explicitly allows unlimited reuse; `[2, 2, 3]` disappears.

```
backtrack(i, t - c)
```

Listing 19.12: Fixed

19.7 Forgetting to reset `visited`

```
visited[i] = True
path.append(nums[i])
bt()
path.pop()
# missing visited[i] = False
```

Listing 19.13: Broken — Permutations

Why: elements are consumed permanently, so after the first root-to-leaf path everything is marked and no further permutation can be built. Returns 1 result instead of $n!$.

```
path.pop()
visited[i] = False # both mutations undone
```

Listing 19.14: Fixed

19.8 Not copying the path

```
res.append(path) # Python
res.add(path);   // Java
```

Listing 19.15: Broken

Why: all entries alias the one live buffer, which is empty at the end.

```
res.append(path[:]) # Python
res.add(new ArrayList<>(path)); // Java
```

Listing 19.16: Fixed

19.9 Duplicate results

```
nums.sort()
for i in range(start, n):
    path.append(nums[i]); bt(i + 1); path.pop()
```

Listing 19.17: Broken — Subsets II without the guard

Why: equal values at the same level generate identical subtrees.

```
for i in range(start, n):
    if i > start and nums[i] == nums[i - 1]:
        continue
    path.append(nums[i]); bt(i + 1); path.pop()
```

Listing 19.18: Fixed

19.10 Over-pruning

```
if i > 0 and nums[i] == nums[i - 1]:
    continue # also forbids [2,2] across levels
```

Listing 19.19: Broken — $i > 0$ instead of $i > \text{start}$

Why: it kills legitimate multi-level repeats. Output too small.

```
if i > start and nums[i] == nums[i - 1]:
    continue
```

Listing 19.20: Fixed

Another over-prune: break on an unsorted array in Combination Sum.

19.11 Under-pruning

```
if len(path) == 2 * n:
    if is_valid("".join(path)):
        res.append("".join(path))
    return
for ch in "()":
    path.append(ch); bt(); path.pop()
```

Listing 19.21: Broken — validate only at the leaf

Why: correct but explores 2^{2n} candidates instead of C_n . TLE.

```
if open < n: ...
if close < open: ...
```

Listing 19.22: Fixed — test the prefix

19.12 Mutating shared state incorrectly

```
static List<List<Integer>> res = new ArrayList<>(); // survives across calls
```

Listing 19.23: Broken — Java static state

Why: the second test case sees the first case's results.

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> res = new ArrayList<>(); // fresh per call
    backtrack(0, nums, new ArrayList<>(), res);
    return res;
}
```

Listing 19.24: Fixed

The Python twin is the mutable default argument `def bt(path=[])`.

19.13 Wrong recursion state

```
path.append(s[start:end])
bt(start + 1) # should continue from the cut
```

Listing 19.25: Broken — Palindrome Partitioning

Why: pieces overlap; the partition is not a partition.

```
bt(end)
```

Listing 19.26: Fixed

General rule: the new state must describe exactly the situation *after* the choice. Write the state on paper before the call.

19.14 Incorrect loop bounds

```
for i in range(start, n):      # LC 77 uses numbers 1..n, not indices
```

Listing 19.27: Broken — Combinations

Why: drops the value n entirely.

```
for i in range(start, n + 1): # values 1..n inclusive
```

Listing 19.28: Fixed

Similarly in Palindrome Partitioning, `for end in range(start+1, n+1)` — the final piece must be allowed to reach the end of the string.

19.15 Incorrect diagonal calculation

```
if (row - col) in diag1 or (col - row) in diag2:    # same family twice!
```

Listing 19.29: Broken — N-Queens

Why: $c - r = -(r - c)$, so both sets track main diagonals; anti-diagonal attacks go undetected and invalid boards are emitted.

```
if (row - col) in diag1 or (row + col) in diag2:
```

Listing 19.30: Fixed

Related: using `diag1[row - col]` as an array index without the $+ n - 1$ offset.

19.16 Infinite recursion

Three usual causes:

1. No base case, or a base case that cannot be reached (`if i == n` but i increases by 2).
2. Recursing with an unchanged state (`bt(start)` instead of `bt(i+1)`).
3. A zero-cost choice: e.g. Combination Sum where a candidate equals 0, so $t - c == t$ forever. Filter out zeros, or assert positivity.

```
def bt(i, t):
    assert 0 ≤ i ≤ n and t ≥ 0      # states your invariants
    if t == 0: ...
    if i == n: return
```

Listing 19.31: Defensive pattern

19.17 A five-minute debugging routine

Remember This

1. **Print the tree.** Add `print(" "*depth, i, path)` at the top of the function. Two levels of output usually reveal the bug instantly.
2. **Check the invariant.** After the loop finishes, is `path` back to what it was on entry? Assert it.
3. **Count.** Compare the number of results against the formula (2^n , $\binom{n}{k}$, $n!$, C_n). Too many $\Rightarrow$ duplicates

or a wrong `start`. Too few $\Rightarrow$ missing undo or over-pruning.

4. **Shrink the input.** Debug with $n = 2$ or $n = 3$; the tree fits on paper.

5. **Read the choose/undo pair.** Are they exact inverses? Is every mutation undone?

Chapter 20

How to Derive a Backtracking Solution

20.1 The twelve questions

Whenever you meet a new problem, walk this list in order. Do not write code until you have answered questions 1–5.

1. **Am I generating possibilities?** Look for “all”, “every”, “generate”, “list”, “find a valid”, or a suspiciously small n . If the answer is a single number over a large input, stop — this is DP or greedy, not backtracking.
2. **What is my state?** Write it as a tuple. The test: *if I handed this tuple to a fresh copy of the function, could it finish the job correctly?* If not, something is missing.
3. **What are my choices?** One of: binary (take/skip), positional (`for i in range(start, n)`), alphabet (digits, letters, columns), directional (grid neighbours), or length (where to cut).
4. **What is my base case?** Distinguish the *accepting* case (complete candidate $\Rightarrow$ record) from the *rejecting* case (dead end $\Rightarrow$ return).
5. **What makes a choice invalid?** These are constraints. Removing them changes the *answers*.
6. **Can I prune?** Monotone quantity? Capacity bound? Sorted input ($\Rightarrow$ `break`)? Duplicate siblings? Symmetry? Reordering (MRV)?
7. **What do I modify?** List every mutation: `path`, `visited`, `board`, constraint sets.
8. **What do I undo?** Exactly the list from question 7, in inverse form, immediately after the recursion.
9. **Can choices be reused?** Yes $\Rightarrow$ `bt(i)`. No $\Rightarrow$ `bt(i+1)`.
10. **Does order matter?** Yes $\Rightarrow$ drop `start`, add `visited`. No $\Rightarrow$ keep `start`.
11. **Are duplicates present in the input?** Yes $\Rightarrow$ sort, then `if i > start and a[i]==a[i-1]: continue` (or `not visited[i-1]` in the permutation framework).
12. **Do multiple paths reach the same state?** Yes, *and* the required output is an aggregate $\Rightarrow$ memoise. Yes but the output is the objects themselves $\Rightarrow$ the exponential is irreducible.

20.2 Worked derivation: a problem you have not seen

Problem. Given an array of distinct positive integers and an integer k , return all subsets of size exactly k whose elements sum to a multiple of 3.

Q1	“return all subsets” $\Rightarrow$ enumeration, backtracking.
Q2	(start, path, s) where s is the running sum. Could I drop s? Yes, recompute at the leaf — but carrying it is $O(1)$ and enables pruning.
Q3	positional: for i in range(start, n).
Q4	accept when len(path) == k <i>and</i> s % 3 == 0; stop when len(path) == k regardless.
Q5	no per-choice constraint — any unused later element is legal.
Q6	capacity prune: need k - len(path) more, so i <= n - need.
Q7	path.append, s += nums[i] (the latter as a parameter, so free).
Q8	path.pop().
Q9	no reuse $\Rightarrow$ bt(i+1).
Q10	order irrelevant $\Rightarrow$ keep start.
Q11	input distinct $\Rightarrow$ no duplicate guard needed.
Q12	counting such subsets would memoise on (i, taken, s mod 3); listing them does not.

The code now writes itself:

```

1 def subsets_k_div3(nums, k):
2     n, res, path = len(nums), [], []
3
4     def bt(start, s):
5         if len(path) == k:                # Q4: complete
6             if s % 3 == 0:
7                 res.append(path[:])
8             return
9         need = k - len(path)              # Q6: capacity prune
10        for i in range(start, n - need + 1):
11            path.append(nums[i])           # Q7
12            bt(i + 1, s + nums[i])        # Q9: no reuse
13            path.pop()                    # Q8
14
15    bt(0, 0)
16    return res

```

Listing 20.1: Derived directly from the twelve answers

```

1 private void bt(int start, int s, int k, int[] nums,
2     List<Integer> path, List<List<Integer>> res) {
3     if (path.size() == k) {
4         if (s % 3 == 0) res.add(new ArrayList<>(path));
5         return;
6     }
7     int need = k - path.size();
8     for (int i = start; i <= nums.length - need; i++) {
9         path.add(nums[i]);
10        bt(i + 1, s + nums[i], k, nums, path, res);
11        path.remove(path.size() - 1);
12    }
13 }

```

Listing 20.2: The same derivation in Java

Key Insight

Notice that at no point did we ask “what is the trick?”. There is no trick. The twelve questions produce the solution mechanically. That reliability is exactly what you want under interview pressure.

20.3 The interview script

What to say, in order, when a backtracking problem appears:

1. *“The output is a collection of candidates and n is small, so this is an exhaustive search — backtracking.”*
2. *“Let me define the state as ... and the choice at each level as ...”*
3. *“Let me draw the decision tree for a size-3 input.”* (Draw it.)
4. *“The base case is ... and I record a copy of the path there, because the path is a shared mutable buffer.”*
5. *“I’ll pass $i+1$ because ...”* (Give the reason, not the rule.)
6. Write the code.
7. *“Let me dry-run it on $[1, 2]$.”*
8. *“Complexity: the tree has ... nodes, each costs ..., so ...; space is $O(\text{depth})$ plus output.”*
9. *“Now let me look for pruning.”* (Walk the five prune questions.)
10. *“Edge cases: empty input, $k = 0$, all duplicates, target unreachable.”*

20.4 One-page interview checklist

BACKTRACKING — INTERVIEW CHECKLIST

RECOGNISE

- ☐ “all / every / generate / list” ☐ output is a set of sequences ☐ $n \leq 20$ (subsets) or $n \leq 10$ (permutations)

DEFINE (before coding)

- ☐ State tuple: _____
☐ Choices at a level: _____
☐ Accepting base case: _____
☐ Rejecting base case: _____

DECIDE

- ☐ Reuse allowed? `bt(i)` vs `bt(i+1)`
☐ Order matters? `visited[]` vs `start`
☐ Duplicates in input? `sort + i > start` and `a[i]==a[i-1]`
☐ Need one answer or all? `bool return` vs `void + res`

WRITE

- ☐ Base case *before* the loop, with `return`
☐ Record a **copy**: `path[:]` / `new ArrayList<>(path)`
☐ CHOOSE → EXPLORE → UNDO adjacent and symmetric
☐ Every mutation has exactly one inverse

PRUNE

- ☐ Monotone quantity crossing a threshold
☐ Capacity: enough elements left to finish?
☐ Sorted input ⇒ `break` instead of `continue`
☐ Duplicate siblings / symmetric branches
☐ Reorder variables by fewest options (MRV)

VERIFY

- ☐ Dry run on $n = 2$
☐ Result count matches the formula $(2^n, \binom{n}{k}, n!, C_n)$
☐ Path is empty again when the top call returns
☐ Complexity: $\text{nodes} \times \text{work-per-node} + \text{solutions} \times \text{copy cost}$
☐ Space: $O(\text{depth})$ auxiliary, output stated separately

Chapter 21

Practice Set

Remember This

Solutions are in Appendix A. **Do not look yet.** For each problem, first fill in the four blanks: state, choices, base case, undo. If you cannot fill them in, re-read the relevant chapter — not the solution.

21.1 Level 1 — Warm-ups (custom problems)

P1. All binary strings of length n

Pattern: take/don't take. **Prerequisite:** Ch. 3. **Statement:** print every string of length n over $\{0, 1\}$.

- *State?* How much of the string is built so far.
- *Choices?* Two characters.
- *Base case?* Length reached.
- *Undo?* Remove the last character.
- *Hint:* the number of outputs must be exactly 2^n — check it.

P2. All strings of length n over $\{a, b, c\}$ with no two adjacent equal

Pattern: take/don't take + a constraint. **Prerequisite:** P1.

- *What makes a choice invalid?* It equals the previous character.
- *Hint:* this is your first prune. Count the answers: $3 \cdot 2^{n-1}$.

P3. All subsets of an array (no library tricks)

Pattern: Ch. 4. Do it *both* ways — take/don't take and loop+start — and confirm both give 2^n answers.

P4. Choose exactly k elements from an array

Pattern: loop + start. **Prerequisite:** P3.

- *Base case?* `len(path) == k`.
- *Hint:* add the capacity prune `i <= n - need`. Verify the count is $\binom{n}{k}$.

P5. All subsets whose sum equals a target (positive integers, each used once)

- *State?* `(start, remaining_target, path)`.
- *Prune?* `target < 0`. Why is that sound?

P6. Print all paths from top-left to bottom-right of an $m \times n$ grid moving only right or down

- *Choices?* Two. *Constraint?* Stay inside the grid.
- *Hint:* the count is $\binom{m+n-2}{m-1}$ — a good self-check.

21.2 Level 2 — LeetCode core

78. Subsets

Easy · take/don't take or loop+start

Prerequisite: P3. **Identify first:** that every node (loop version) or every leaf (binary version) is an answer; that the path must be *copied*.

- *State? Choices? Base case? Undo?*
- *Hint:* no pruning is possible. Why not?

77. Combinations

Medium · loop + start

Prerequisite: 78. **Identify first:** the tree is the same as Subsets; only the recording rule changes.

- *Hint:* derive the loop bound $i \leq n - \text{need} + 1$ yourself before coding.
- *Check:* the answer count is $\binom{n}{k}$.

39. Combination Sum

Medium · reusable choices

Prerequisite: 77. **Identify first:** reuse is allowed $\Rightarrow \text{bt}(i)$; ordering invariant is non-decreasing, not strictly increasing.

- *What is the base case, and what is the reject case?*
- *Hint:* sort first so `continue` can become `break`. State the invariant that makes `break` sound.

46. Permutations

Medium · visited[]

Prerequisite: 78. **Identify first:** there is *no* start index; two mutations means two undos.

- *Why can't the start-index technique work here?* Answer this before coding.
- *Check:* $n!$ results.

17. Letter Combinations of a Phone Number

Medium · string building

Prerequisite: P1. **Identify first:** the input string *is* the depth; the empty-input edge case.

- *Undo?* Depends on whether you use a buffer or immutable concatenation.
- *Trap:* empty input returns `[]`, not `[""]`.

22. Generate Parentheses

Medium · constraint pruning

Prerequisite: P1. **Identify first:** the two guards, and *why* an invalid prefix can never be repaired.

- *State?* Not an index — two counters.
- *Hint:* do not generate-then-filter. Prove that `close < open` is a sound prune.

90. Subsets II

Medium · duplicates

Prerequisite: 78. **Identify first:** sort; the difference between same-level and different-level duplicates.

- *Why is the guard $i > \text{start}$ and not $i > 0$?* Answer before coding.

40. Combination Sum II

Medium · duplicates + no reuse

Prerequisite: 39 and 90. **Identify first:** this uses `bt(i+1)` *and* the duplicate guard — the only problem here that needs both.

131. Palindrome Partitioning

Medium · partitioning

Prerequisite: 77. **Identify first:** the choice is a *piece length*; the recursion continues at the cut.

- *Trap:* recursing with `start+1` instead of `end`.
- *Follow-up:* precompute the palindrome table. Does it reduce nodes or cost per node?

79. Word Search

Medium · grid

Prerequisite: P6. **Identify first:** `visited` is path-local and must be cleared on the way out; the accept check must precede the bounds check.

- *Undo?* Restore the exact original character.
- *Hint:* return `bool`, and short-circuit with `or / ||`.

21.3 Level 3 — Hard

51. N-Queens

Hard · constraint satisfaction

Prerequisite: 46. **Identify first:** one queen per row is the *model*, not an optimisation; the diagonal keys $r - c$ and $r + c$.

- *State?* Three constraint sets plus the row.
- *Undo?* All three sets — forgetting one is the classic bug.
- *Follow-up:* LC 52 with bitmasks.

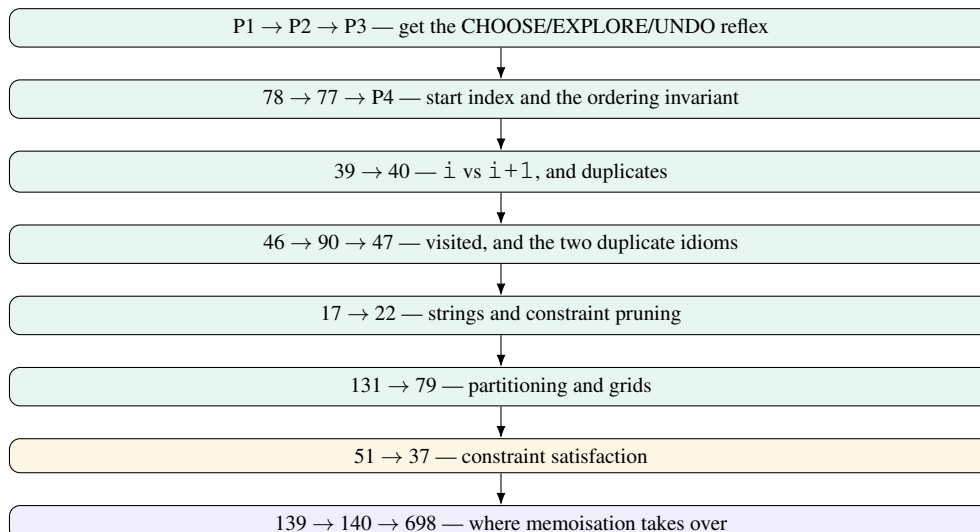
37. Sudoku Solver

Hard · CSP + early termination

Prerequisite: 51 and LC 36. **Identify first:** the function returns `bool`; on success you must *not* undo.

- *Box index?* Derive $(r//3)*3 + c//3$ yourself.
- *Optimisations, in order:* sets $\rightarrow$ bitmasks $\rightarrow$ MRV $\rightarrow$ constraint propagation. Implement at least the first two.

21.4 Suggested progression



Interview Tip

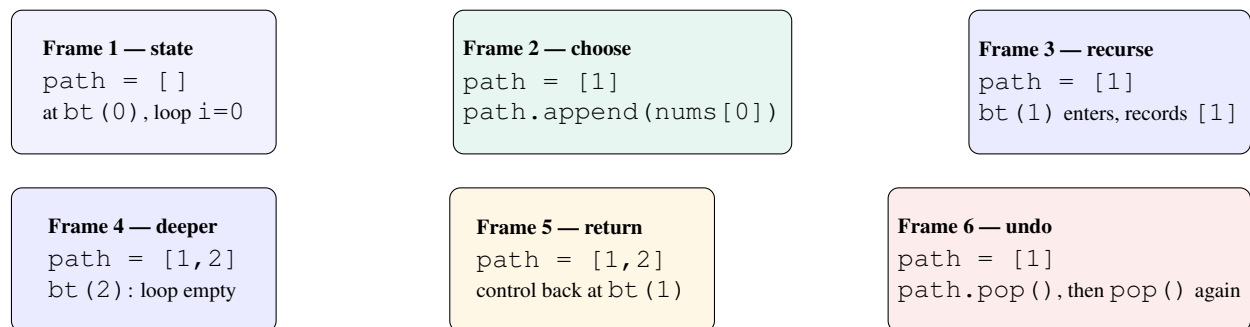
Solve each problem twice: once from scratch, and once a week later from memory. The second attempt is what actually builds the reflex. If the second attempt still needs the solution, the gap is in the *derivation*, not the code — go back to Chapter 20.

Chapter 22

Diagrams and Animations

22.1 Sequential frames: animating the cycle on paper

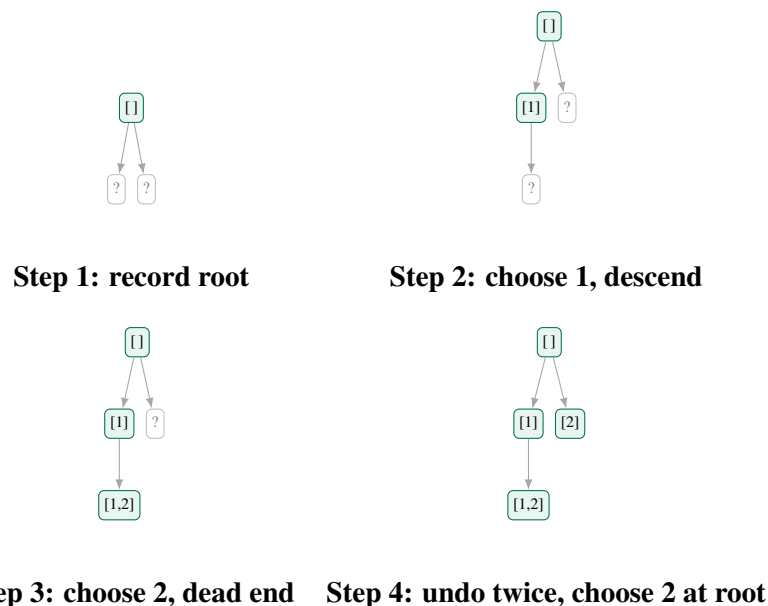
Real PDF animation needs a viewer that supports it. A more reliable technique — and the one that actually helps revision — is a *sequence of frames* laid out side by side, each showing one step of the **CHOOSE** → **EXPLORE** → **UNDO** cycle. Below is the cycle for `nums=[1, 2]`, loop+start Subsets, drawn as six frames.



Read them left to right, top to bottom. The one thing to watch is the `path` field: it grows in frames 2 and 4, and shrinks in frame 6 back to exactly what it was in frame 1.

22.2 The same idea as a growing tree

Four snapshots of the Subsets tree for `[1, 2]` being built, one level of exploration at a time. Grey nodes are not yet created.



Between step 3 and step 4 lies the entire content of backtracking: two `pop()`s that return `path` from `[1, 2]` to `[]`.

22.3 Beamer overlays — a compilable presentation

If you want genuine click-through animation, compile the following as a *separate* Beamer document. Each `\pause` or overlay specification produces one PDF page, so clicking forward in any viewer animates the recursion.

```

1 \documentclass{beamer}
2 \usepackage{tikz}
3 \usetikzlibrary{arrows.meta}
4 \usecolortheme{seahorse}
5 \title{Backtracking: CHOOSE - EXPLORE - UNDO}
6 \begin{document}
7
8 \begin{frame}{Subsets of [1,2]: the state buffer}
9   \begin{itemize}
10    \item<1→ \texttt{bt(0)}: path = [ ] -- record [ ]
11    \item<2→ CHOOSE 1: path = [1]
12    \item<3→ EXPLORE \texttt{bt(1)} -- record [1]
13    \item<4→ CHOOSE 2: path = [1,2] -- record [1,2]
14    \item<5→ UNDO: path = [1]
15    \item<6→ UNDO: path = [ ]
16    \item<7→ CHOOSE 2: path = [2] -- record [2]
17    \item<8→ UNDO: path = [ ] \alert{invariant restored}
18   \end{itemize}
19 \end{frame}
20
21 \begin{frame}{The tree grows and retreats}
22   \begin{tikzpicture}[level distance=13mm,
23     level 1/.style={sibling distance=28mm},
24     every node/.style={draw,rounded corners,font=\small}]
25     \node (root) { [ ] }
26     child { node[visible on=<2→] { [1] } }
27     child { node[visible on=<4→] { [1,2] } }
28     child { node[visible on=<7→] { [2] } };
29     \onslide<5-6>{\node[draw=none,red] at (3.4,-1.3) {UNDO};}
30   \end{tikzpicture}
31 \end{frame}
32
33 \begin{frame}{N-Queens: place, conflict, backtrack}
34   \begin{tikzpicture}[scale=0.7]
35     \foreach \r in {0,...,3} \foreach \c in {0,...,3} {
36       \draw[fill=black!8] (\c,-\r) rectangle ++(1,-1); }
37     \node[visible on=<2→] at (0.5,-0.5) {\textbf{Q}};
38     \node[visible on=<3→,red] at (1.5,-1.5) {\textbf{times}};
39     \node[visible on=<4→] at (2.5,-1.5) {\textbf{Q}};
40     \node[visible on=<5→,red] at (0.5,-2.5) {\textbf{times}};
41     \node[visible on=<5→,red] at (1.5,-2.5) {\textbf{times}};
42     \node[visible on=<5→,red] at (2.5,-2.5) {\textbf{times}};
43     \node[visible on=<5→,red] at (3.5,-2.5) {\textbf{times}};
44   \end{tikzpicture}
45   \begin{itemize}
46     \item<3→ column and diagonal conflicts are rejected immediately
47     \item<5→ row 2 has no safe square: \alert{dead end}
48     \item<6→ undo row 1, try the next column
49   \end{itemize}
50 \end{frame}
51
52 \end{document}

```

Listing 22.1: animation.tex — compile with `pdflatex animation.tex`

Add this helper to the preamble if your Beamer version lacks it:

```

1 \tikzset{
2   visible on/.style={alt={#1}{invisible}}},
3   alt/.code args={<#1>#2#3}{%
4     \alt<#1>{\pgfkeysalso{#2}}{\pgfkeysalso{#3}}},
5   invisible/.style={opacity=0}
6 }

```

Listing 22.2: visible on for TikZ nodes

Key Insight

The overlay technique generalises: put the *state* on one slide with \onslide steps for CHOOSE, EXPLORE, RETURN, UNDO, NEXT CHOICE. Building one such deck yourself for a problem you find hard is, in my experience, the fastest way to make the undo step stop feeling mysterious.

22.4 Notation used throughout this book

green node

A solution / accepting leaf, or a node that is itself an answer.

red dashed

A pruned branch: generated conceptually but never explored (or killed on arrival).

i=2 | path=[1]

Node annotations always list the *call parameters*: index or start, remaining target, path, and `visited` where relevant.

CHOOSE → **EX-
PLORE** → **UNDO**

The cycle, highlighted in green / blue / red respectively everywhere in the book.

Interview Tip

Use the same colour convention on the whiteboard: one colour for the choice you are making, one for the recursive descent, one for the undo. Interviewers follow a coloured tree far more easily than a monochrome one, and it forces you to keep the three steps distinct in your own head.

Chapter 23

Final Summary

23.1 Backtracking in 10 rules

Remember This

1. Backtracking is DFS over an implicit state-space tree, with constraint pruning and state restoration.
2. Define the **state** before writing a single line. If a fresh copy of the function could not finish from your state tuple, the state is incomplete.
3. The base case goes **before** the loop and always ends with `return`.
4. Record a **copy** of the path, never the live buffer.
5. Every mutation in CHOOSE has exactly one inverse in UNDO, placed immediately after the recursive call.
6. A call must leave shared state exactly as it found it.
7. `i` means reuse allowed; `i+1` means the element is consumed; no index at all (with `visited`) means order matters.
8. Duplicates in the input: sort, then skip duplicate *siblings* — never duplicate *ancestors*.
9. Prune as early as possible; a cut at depth k saves b^{d-k} leaves.
10. If the answer is a count, a boolean, or an optimum, look for a memoisable state — it is probably DP in disguise.

23.2 Template — Python

```
1 def solve(inputs):
2     res, path = [], []
3
4     def backtrack(state):
5         if is_solution(state):           # 1. base case, BEFORE the loop
6             res.append(path[:])         # 2. snapshot, not the reference
7             return                       # 3. stop descending
8
9         for choice in choices(state):    # 4. this level's options
10            if invalid(choice):           # 5. constraint / prune
11                continue
12
13            make(choice)                  # 6. CHOOSE
14            backtrack(next_state)         # 7. EXPLORE
15            undo(choice)                  # 8. UNDO
16
17     backtrack(initial_state)
18     return res
```

Listing 23.1: Memorise this shape

23.3 Template — Java

```

1 public List<List<Integer>> solve(int[] inputs) {
2     List<List<Integer>> res = new ArrayList<>();
3     backtrack(initialState, inputs, new ArrayList<>(), res);
4     return res;
5 }
6
7 private void backtrack(State state, int[] inputs,
8                         List<Integer> path, List<List<Integer>> res) {
9     if (isSolution(state)) {
10        res.add(new ArrayList<>(path));    // snapshot
11        return;
12    }
13    for (Choice choice : choices(state)) {
14        if (invalid(choice)) continue;    // constraint / prune
15
16        make(choice, path);                // CHOOSE
17        backtrack(nextState, inputs, path, res);    // EXPLORE
18        path.remove(path.size() - 1);    // UNDO
19    }
20 }

```

Listing 23.2: The same shape

23.4 Pattern recognition table

The problem says	Pattern	Example
all subsets / power set	take-skip or loop+start	78
choose exactly k	loop + start + size base case	77
unlimited reuse of elements	bt (i)	39
each element at most once	bt (i+1)	40
all orderings / arrangements	visited[]	46
input contains duplicates	sort + same-level skip	90, 40, 47
build a string character by character	buffer + undo	17
prefix validity is checkable	prune at every node	22
split a string / array into pieces	loop over end, bt (end)	131
walk a grid, no cell reuse per path	mark + restore	79
no two items may conflict	constraint sets, CSP	51
fill a board subject to rules	CSP + bool return	37
count / min / max / feasibility	memoise the state	139, 698

23.5 i vs $i + 1$ — the one-liner

backtrack(i): “I may want this element again.”
 backtrack($i + 1$): “I am finished with this element forever.”
 backtrack() with visited: “Any unused element, in any order.”

23.6 Combination vs permutation

$\{1, 2\} = \{2, 1\}$
 combination: use start

$\{1, 2\} \neq \{2, 1\}$
 permutation: use visited

Counts: $\binom{n}{k}$ versus $\frac{n!}{(n-k)!}$. If you find yourself writing `range(start, n)` in a permutation problem, or a `visited` array in a combination problem, stop — you have picked the wrong framework.

23.7 Pruning checklist

Remember This

- ☐ Monotone quantity crossing a threshold (`t < 0, close > open`)
- ☐ Capacity bound (`i <= n - need + 1, len(word) > rows*cols`)
- ☐ Sorted input $\Rightarrow$ `break` instead of `continue`
- ☐ Duplicate siblings $\Rightarrow$ `i > start` and `a[i] == a[i-1]`
- ☐ Symmetric solutions $\Rightarrow$ explore the canonical half only
- ☐ Reorder variables by fewest remaining options (MRV)
- ☐ Detect zero options before descending
- ☐ Is the prune **sound**? Name the invariant that proves it.

23.8 Complexity checklist

Remember This

- ☐ Branching factor b : constant, shrinking, or data-dependent?
- ☐ Depth d ?
- ☐ Node count N — does pruning make it $\approx$ the number of solutions?
- ☐ Work per node (loop + constraint checks)?
- ☐ Copy cost per solution?
- ☐ Number of solutions S — am I output-optimal?
- ☐ Space: $O(d)$ stack + shared state; state the output separately.
- ☐ Never claim $O(1)$ space for a recursive solution.

23.9 Interview checklist

Remember This

- ☐ Say why it is backtracking (“all ...”, small n)
- ☐ Define the state tuple out loud
- ☐ Draw the decision tree for $n = 3$
- ☐ State the base case and the recording rule
- ☐ Justify `i` vs `i+1` vs `visited` *with a reason, not a rule*
- ☐ Write the code with CHOOSE/EXPLORE/UNDO adjacent
- ☐ Dry-run on a two-element input
- ☐ Give the complexity as `nodes × work + solutions × copy`
- ☐ Then say: “now let me look for pruning”
- ☐ Cover edge cases: empty input, $k = 0$, all duplicates, unreachable target

23.10 Complete problem progression

#	Problem	Pattern	Why it comes here
1	Binary strings	take/skip	the bare cycle
2	No-adjacent-equal strings	+ constraint	first prune
3	Subsets (78)	take/skip & loop+start	two dialects
4	Combinations (77)	loop + start	ordering invariant
5	Combination Sum (39)	bt (i)	reuse, target pruning
6	Combination Sum II (40)	bt (i+1) + dedup	both ideas at once
7	Permutations (46)	visited[]	order matters
8	Subsets II (90)	sort + skip	same-level duplicates
9	Permutations II (47)	visited[i-1]	the other dedup idiom
10	Letter Combinations (17)	string buffer	mutable vs immutable
11	Generate Parentheses (22)	prefix pruning	monotone invalidity
12	Palindrome Partitioning (131)	bt (end)	cuts as choices
13	Word Search (79)	grid + restore	path-local visited
14	N-Queens (51)	CSP + sets	modelling beats brute force
15	Sudoku (37)	CSP + bool return	early termination, MRV
16	Word Break II (140)	memoised search	where DP takes over

23.11 One-page cheat sheet

BACKTRACKING — ONE PAGE

THE CYCLE

CHOOSE → EXPLORE → UNDO

Every mutation has one inverse. A call restores the state it found.

THE TEMPLATE

```
def bt(state):
    if is_solution(state): res.append(path[:]); return
    for choice in choices:
        if invalid(choice): continue
        make(choice); bt(next); undo(choice)
```

THE FOUR DECISIONS

reuse allowed?	bt(i) vs bt(i+1)
order matters?	visited[] vs start
duplicates in input?	sort + i > start and a[i] == a[i-1]
one answer or all?	bool return vs void + res

COUNTS TO SANITY-CHECK

subsets 2^n · combinations $\binom{n}{k}$ · permutations $n!$ · parentheses C_n · partitions 2^{n-1}

COMPLEXITY

$$T = N \cdot w + S \cdot c, \quad N \leq O(b^d)$$

Space = $O(\text{depth})$ auxiliary; output stated separately.

TOP FIVE BUGS

1. `res.append(path)` without a copy
2. missing `pop()`
3. visited not cleared
4. `i` vs `i+1` confusion
5. `i > 0` instead of `i > start`

PRUNE

monotone · capacity · sorted → break · dedup · symmetry · MRV

Prune early: a cut at depth k removes b^{d-k} leaves.

Appendix A

Solutions

Interview Tip

Read a solution only after you have attempted the problem and written down your state, choices, base case and undo. Comparing your four answers against the ones below is worth more than reading the code.

A.1 Level 1 — Warm-ups

P1. All binary strings of length n

State $(i, \text{path}) \cdot \text{Choices } '0', '1' \cdot \text{Base } i == n \cdot \text{Undo } \text{pop}.$

```
1 def binary_strings(n):
2     res, path = [], []
3     def bt(i):
4         if i == n:
5             res.append("".join(path)); return
6         for ch in ("0", "1"):
7             path.append(ch); bt(i + 1); path.pop()
8     bt(0)
9     return res
```

```
1 void bt(int i, int n, StringBuilder sb, List<String> res) {
2     if (i == n) { res.add(sb.toString()); return; }
3     for (char ch : new char[]{'0', '1'}) {
4         sb.append(ch);
5         bt(i + 1, n, sb, res);
6         sb.deleteCharAt(sb.length() - 1);
7     }
8 }
```

Count 2^n ; time $O(n2^n)$; space $O(n)$.

P2. Strings over $\{a, b, c\}$, no two adjacent equal

The only addition is one constraint. Note it is tested *before* the mutation, so no undo is wasted.

```
1 def no_adjacent(n):
2     res, path = [], []
3     def bt(i):
4         if i == n:
5             res.append("".join(path)); return
6         for ch in "abc":
7             if path and path[-1] == ch:
8                 continue # CONSTRAINT
9             path.append(ch); bt(i + 1); path.pop()
10    bt(0)
11    return res
```

Count $3 \cdot 2^{n-1}$ — 3 choices at the first position, 2 at every later one.

P3. All subsets

Both dialects appear in Chapter 4; the loop version is repeated here for completeness.

```

1 def subsets(nums):
2     res, path = [], []
3     def bt(start):
4         res.append(path[:])
5         for i in range(start, len(nums)):
6             path.append(nums[i]); bt(i + 1); path.pop()
7     bt(0)
8     return res

```

P4. Choose exactly k elements

```

1 def choose_k(nums, k):
2     n, res, path = len(nums), [], []
3     def bt(start):
4         if len(path) == k:
5             res.append(path[:]); return
6         need = k - len(path)
7         for i in range(start, n - need + 1): # capacity prune
8             path.append(nums[i]); bt(i + 1); path.pop()
9     bt(0)
10    return res

```

```

1 void bt(int start, int k, int[] nums,
2     List<Integer> path, List<List<Integer>> res) {
3     if (path.size() == k) { res.add(new ArrayList<>(path)); return; }
4     int need = k - path.size();
5     for (int i = start; i ≤ nums.length - need; i++) {
6         path.add(nums[i]);
7         bt(i + 1, k, nums, path, res);
8         path.remove(path.size() - 1);
9     }
10 }

```

Count $\binom{n}{k}$; time $O(k\binom{n}{k})$.

P5. Subsets summing to a target

```

1 def subsets_with_sum(nums, target):
2     nums.sort()
3     n, res, path = len(nums), [], []
4     def bt(start, t):
5         if t == 0:
6             res.append(path[:]); return
7         for i in range(start, n):
8             if nums[i] > t:
9                 break # sound: all values positive & sorted
10            path.append(nums[i]); bt(i + 1, t - nums[i]); path.pop()
11    bt(0, target)
12    return res

```

The prune is sound because every candidate is positive, so t is non-increasing along a path and can never climb back to 0 once it passes it.

P6. All right/down paths in a grid

```

1 def grid_paths(m, n):
2     res, path = [], []
3     def bt(r, c):
4         if r == m - 1 and c == n - 1:
5             res.append("".join(path)); return
6         if r + 1 < m:
7             path.append("D"); bt(r + 1, c); path.pop()
8         if c + 1 < n:
9             path.append("R"); bt(r, c + 1); path.pop()
10    bt(0, 0)
11    return res

```

Count $\binom{m+n-2}{m-1}$. The bounds tests are constraints, not prunes: without them the walk leaves the grid.

A.2 Level 2 — LeetCode core

78. Subsets

See Chapter 4 (both formulations, Python and Java). *State* (i, path) or (start, path) · *Base* i == n (or record at every node) · *Undo* pop · $O(n2^n)$. No pruning is possible because the output itself has 2^n entries.

77. Combinations

See Chapter 5. The loop bound $i \leq n - \text{need} + 1$ follows from requiring $n - i + 1 \geq \text{need}$.

39. Combination Sum

See Chapter 6. Key line: backtrack(i, t - c) — i, not i+1. break is sound only after sorting.

46. Permutations

See Chapter 7. Two mutations, two undos. The start-index technique fails because it can only ever emit the increasing arrangement.

17. Letter Combinations

See Chapter 9. Guard the empty input first: return [], not [""].

22. Generate Parentheses

See Section 9.3. Guards: open < n and close < open. Invalidity is monotone, which is what makes the prune sound.

90. Subsets II

See Chapter 8. $i > \text{start}$ kills duplicate *siblings*; $i > 0$ would wrongly kill duplicate *ancestors* and lose [2, 2].

40. Combination Sum II

See Chapter 8. The only problem in the set that needs bt(i+1) *and* the duplicate guard *and* the target prune simultaneously.

131. Palindrome Partitioning

See Chapter 10. Recurse with `end`, never `start + 1`. The precomputed table reduces cost per node from $O(n)$ to $O(1)$; it does not reduce the node count.

79. Word Search

See Chapter 11. Accept check before bounds check; restore the original character on every exit path.

A.3 Level 3 — Hard

51. N-Queens

See Chapter 12. One queen per row is the model. Diagonals are $r - c$ and $r + c$; using $r - c$ and $c - r$ tracks the same family twice and misses anti-diagonal attacks. All three sets must be undone.

37. Sudoku Solver

See Chapter 13. The function returns `bool` so a deep success unwinds the stack without undoing. Box index $(r//3)*3 + c//3$. Optimisation ladder: sets $\rightarrow$ bitmasks $\rightarrow$ MRV $\rightarrow$ constraint propagation.

A.4 Answers to the “Try Yourself” prompts

Count subsets with an even sum (Ch. 4)

```

1 from functools import lru_cache
2
3 def count_even_sum_subsets(nums):
4     n = len(nums)
5     @lru_cache(maxsize=None)
6     def bt(i, parity):
7         if i == n:
8             return 1 if parity == 0 else 0
9         return bt(i + 1, (parity + nums[i]) % 2) + bt(i + 1, parity)
10    return bt(0, 0)

```

State (i, parity) — only $2n$ states, so this is $O(n)$ rather than $O(2^n)$. It is the memoisation lesson of Chapter 15 in miniature. (The closed form, for $n \geq 1$ with at least one odd element, is 2^{n-1} .)

Subsets in increasing order of size (Ch. 4)

Run the `choose_k` routine for $k = 0, 1, \dots, n$. Total work is $\sum_k k \binom{n}{k} = n2^{n-1}$ — the same as sorting-free single-pass generation, but the output arrives already grouped. Sorting all 2^n subsets afterwards costs $O(n2^n \log 2^n) = O(n^2 2^n)$, which is strictly worse.

Combination Sum with $i+1$ instead of i (Ch. 6)

For $[2, 3, 6, 7]$, target 7, the output shrinks from $[[2, 2, 3], [7]]$ to $[[7]]$: $[2, 2, 3]$ needs the 2 twice, which $i+1$ forbids.

Why LC 90 uses `i > start` but LC 47 uses not `visited[i-1]` (Ch. 8)

Both express “use only the canonical copy among equal values”. With a `start` index, “same level” is exactly “`i` ranged over by this loop beyond its first value”, i.e. `i > start`. With `visited`, there is no level marker, so canonicity is stated directly: consume the leftmost *unused* copy. If `visited[i-1]` is false, the previous equal copy is still on the shelf, so taking this one would be out of order — skip. If it is true, the previous copy is an ancestor in the current path, so this is a genuine second occurrence and must be allowed.